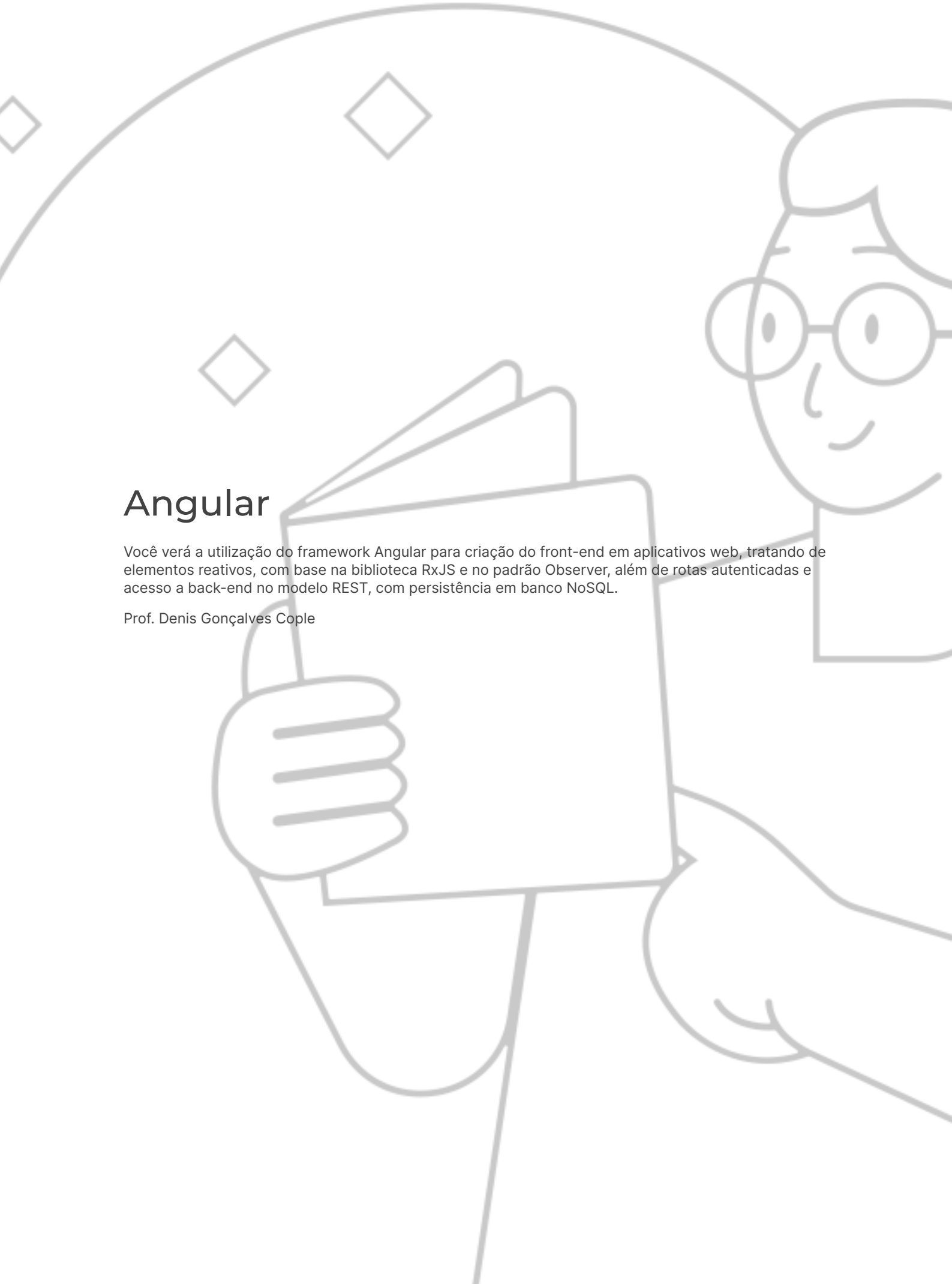


# Angular

Você verá a utilização do framework Angular para criação do front-end em aplicativos web, tratando de elementos reativos, com base na biblioteca RxJS e no padrão Observer, além de rotas autenticadas e acesso a back-end no modelo REST, com persistência em banco NoSQL.

Prof. Denis Gonçalves Cople

## Propósito

As instalações necessárias serão apresentadas ao longo deste conteúdo. Cabe destacar que a plataforma Windows é a utilizada, e os arquivos dos projetos estão disponíveis para download.

## Objetivos

- Reconhecer o framework Angular na estruturação do comportamento de sistemas web.
- Aplicar a arquitetura de componentes na criação do front-end de sistemas web.
- Empregar o paradigma reativo para criar front-end com elementos assíncronos.
- Aplicar o framework Angular na criação do front-end de um sistema completo.

## Introdução

Neste conteúdo, vamos abordar a construção do front-end através do framework Angular, de ampla aceitação no mercado, e que permite melhor organização do código, com o uso de TypeScript na construção de elementos voltados para a lógica do sistema, HTML para estruturar as páginas e CSS para estilização. Iremos compreender a construção da interface de usuário, segundo uma estrutura modular, baseada em componentes e serviços, além de implementar processamento assíncrono, seguindo o paradigma da programação reativa, através da biblioteca RxJS.

Veremos como funciona a navegação interna nos aplicativos Angular, nos quais as interfaces seguem o padrão SPA (single page application), e utilizaremos os componentes do Material Design para melhoria da usabilidade e acessibilidade do front-end.

Finalmente, iremos definir um back-end apropriado, baseado na tecnologia express, executando no NodeJS, com persistência efetuada no MongoDB, uma base NoSQL documental. Através da construção de um exemplo completo, em que a comunicação segue o padrão REST, teremos um modelo alinhado com as arquiteturas mais populares do mercado de desenvolvimento.

### Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

# Arquitetura do framework Angular

O framework Angular é de fácil utilização, com uma arquitetura baseada em módulos e sintaxe TypeScript, além da incorporação e combinação de funcionalidades através de decorações e injeção de dependências. Para que possamos estruturar corretamente um front-end baseado em Angular, devemos compreender a anatomia dos componentes, o modelo de roteamento e os relacionamentos entre os diversos elementos do framework.

Neste vídeo, você terá uma visão geral da arquitetura do framework angular, com destaque para os conceitos de arquitetura, módulo e fluxo de execução do angular.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Conceitos da arquitetura

O framework Angular é uma plataforma construída com base na linguagem **TypeScript**, ou seja, Java Script com acréscimo de **tipagem forte**, tendo como características principais:

- Estrutura baseada em componentes, viabilizando alto nível de reúso.
- Coleção de bibliotecas integradas que oferece, entre outras funcionalidades, roteamento, gerenciamento de formulários e comunicação cliente-servidor.
- Ferramentas para desenvolvedores, utilizadas na construção e no teste do aplicativo.

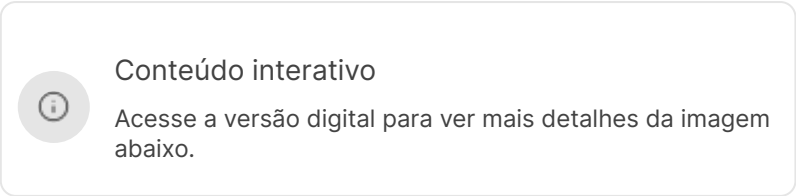
A arquitetura do framework Angular é baseada em quatro conceitos principais:

- Componentes.
- Modelos com ligação de dados (**data binding**) e diretivas.
- Módulos.
- Serviços com injeção de dependências.

## Componentes e modelos com ligação de dados

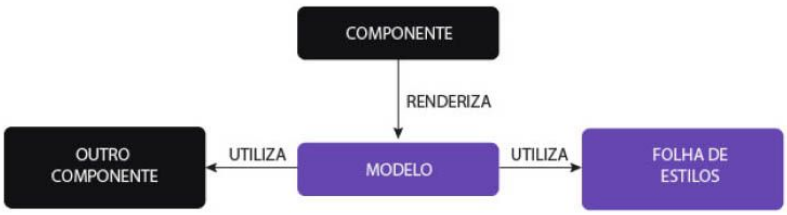
Um componente é, basicamente, um arquivo de código-fonte em TypeScript ou Java Script, com um modelo HTML (Hypertext Markup Language) associado e um nome para **exportação**. Outros componentes podem ser acessados no modelo a partir do nome, expresso na forma de um **seletor HTML**, e os dados que são

trabalhados ao nível do código-fonte podem ser recuperados no modelo, através de uma técnica denominada **data binding**.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Estrutura geral de um componente no Angular.

Um exemplo de componente é apresentado no código seguinte, no qual podem ser observados a definição do nome (selector), o modelo (templateUrl) e as folhas de estilos (styleUrls). Note que foi utilizada uma decoração para a classe **AppComponent**, a qual é **exportada** para outros arquivos e contém o atributo **title**.

```typescript
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'PrimeiroApp';
}
```

Ao nível do código HTML do modelo, a ligação com o atributo title, da classe AppComponent, exige apenas a utilização de chaves duplas para acesso ao seu valor.

```python
{{ title }} app is running!
```

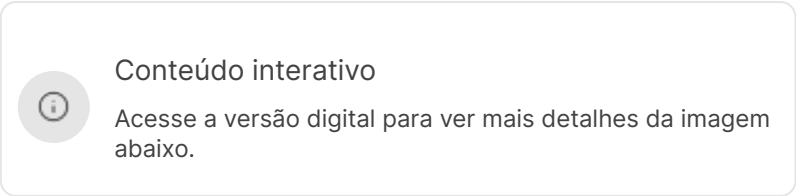
## Módulo

Para o Angular, refere-se a uma coleção de componentes e funcionalidades que apresentem alguma finalidade específica, dentro de um contexto de utilização. Por exemplo, o módulo **BrowserModule**, incluído nas bibliotecas do núcleo do Angular, oferece as funções para compatibilidade entre diferentes navegadores.

Podemos ter diversos módulos no projeto, mas apenas um pode ser marcado como raiz, ou **root**, ficando responsável pela inicialização do aplicativo, ou **bootstrap**, e pela invocação de outros módulos quando necessário.

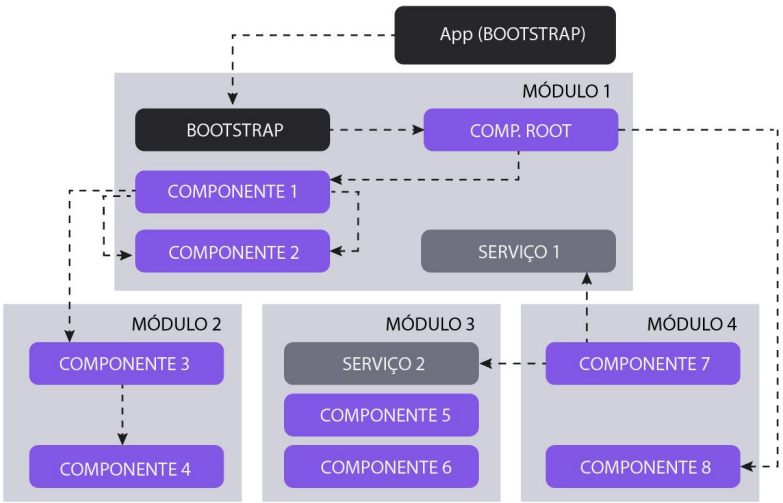
Um módulo pode ser configurado para acessar os recursos de outros módulos, ou seja, os componentes de

um módulo podem acessar os componentes e serviços de qualquer outro módulo, como exemplificado a seguir.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Exemplo de módulos com interação entre componentes e serviços.

A estrutura de módulo de um projeto Angular típico fica no arquivo **app.module.ts**, no qual temos uma classe com o nome **AppModule**, como no código replicado a seguir. Através da decoração **NgModule**, temos a inclusão dos componentes no atributo **declarations**, importação do módulo de compatibilidade e definição do componente raiz (AppComponent) em bootstrap; e no caso de serem utilizados serviços, eles seriam incluídos no atributo **providers**.

```typescript
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

## Fluxo de execução do Angular

Caso nosso aplicativo efetuasse o acesso a servidores externos, dentro de uma regra de negócios estabelecida, poderíamos criar **serviços**. Eles são elementos criados na sintaxe TypeScript, com **injeção de dependência**, nos componentes que os utilizam, através dos construtores.

Atenção

A injeção de dependência permite estabelecer o relacionamento entre serviços e componentes com baixo acoplamento e de forma pouco intrusiva.

O fluxo de execução do Angular é iniciado no arquivo **main.ts**, classificado como **entry point**, no qual são carregados todos os módulos do projeto e iniciado (bootstrap) o módulo principal. Para o exemplo apresentado seguir, o módulo principal seria **Appmodule**.

```typescript
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from
       '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';
import { environment } from './environments/environment';

if (environment.production) {
   enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule)
   .catch(err => console.error(err));
```

Após a execução do ponto de entrada do Angular, ocorre a construção da página a partir do arquivo **index.html**. Nele, temos um código HTML padrão, mas com a inclusão de uma chamada para o componente raiz através de seu seletor (**app-root**), como podemos observar no trecho do corpo da página, apresentado a seguir. Note que o desenho da página será possível apenas por causa da carga dos módulos efetuada anteriormente.

```python
```

# Atividade 1

Questão 1

Na engenharia de software, definimos acoplamento como o nível de dependência encontrada entre os diversos elementos de um sistema, e quanto maior for o acoplamento em um sistema, maior será a dificuldade de manutenção. Qualquer plataforma de desenvolvimento atual busca meios para diminuir o acoplamento nos sistemas. No Angular, garantimos o baixo acoplamento dos serviços através de uma técnica específica. Qual o nome da técnica utilizada?

A Modularização

B Injeção de dependências

C Uso de seletores

D Vinculação de dados

E Herança de classes

A alternativa B está correta.

No framework Angular, os serviços são utilizados pelos componentes a partir da injeção de dependências no construtor da classe do componente, o que garante um modelo de utilização com baixo acoplamento.

## Linguagem TypeScript

A linguagem TypeScript foi criada pela Microsoft como uma plataforma de código aberto, sendo definida a partir de uma extensão do Java Script com a adição de tipos fortes.

Ao contrário do Java Script padrão, que é apenas interpretado pelos navegadores, o TypeScript é uma linguagem que exige a compilação. Quando compilamos os arquivos de nossos programas em TypeScript, ocorre a conversão completa para Java Script, viabilizando a abertura a partir de qualquer navegador, o que faz parte da estratégia adotada pelo Angular.

Neste vídeo, você compreenderá a linguagem TypeScript ao observar pontos importantes, como: tipos de dados, funções, abordagem orientada a objetos, modelagem comportamental e modificadores. Confira!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

### Tipos de dados

Vejamos agora os tipos nativos que são oferecidos pelo TypeScript.

## number

Representa um número, sem diferenciar entre inteiro ou ponto fluente.

Exemplos:

- let val1: number;
- let val2: 10.5;

## string

São elementos de texto.

Exemplo:

- let f1: string = “TESTE”;

## boolean

É uma variável lógica, podendo assumir valores verdadeiro ou falso.

Exemplos:

- let c1: boolean = true;
- let c2 = false;

## any

Aceita qualquer tipo de dado.

Exemplo:

- let s: any = “XPTO”;

array
Cria os vetores.

Exemplos:

- let a: number[ ] = [1, 2, 3];

- let b: Array<number>;

Não é obrigatório definir a variável com o tipo explicitamente, podendo ser apenas atribuído um valor inicial. Isso ocorre porque temos a **inferência** de tipos.

## Funções

Elas devem indicar o **tipo** de retorno ou utilizar **void** quando não ocorre o retorno de um valor.

```typescript
function digaAlo(): void {
   console.log("Alo mundo");
}

function somarV1(x: number, y: number): number {
   return x + y;
}

let somarV2 = (x: number, y: number): number => {
   return x + y;
}

let somarV3 = (x: number, y: number) => x + y;
```

## Abordagem orientada a objetos

Uma característica essencial do TypeScript é sua abordagem robusta da **orientação a objetos**, com palavras específicas para a definição de **classes**, diferentemente do uso de function pelo Java Script. Também foi incluído um método próprio para inicialização do objeto, com a palavra **constructor**, possibilidade de **herança** e controle do **nível de acesso**, como pode ser observado na listagem seguinte.

```typescript
class Pessoa{
   private registro: number;
   protected nome: string;
   constructor(registro: number, nome: string){
      this.registro = registro;
      this.nome = nome;
   }
   exibir(): void {
      console.log(this.registro + ": " + this.nome);
   }
}

class Profissional extends Pessoa{
   private profissao: string;
   constructor(registro: number, nome: string, profissao: string){
      super(registro,nome);
      this.profissao = profissao;
   }
   exibir(): void{
      super.exibir();
      console.log("Trabalha como " + this.profissao)
   }
}

let p = new Profissional(1,"Ana","Balconista");
p.exibir();
```

Inicialmente, temos a definição da classe **Pessoa**, com os campos **registro** e **nome**, tendo nível de acesso privado (**private**) para o primeiro campo e protegido (**protected**) para o segundo. Com base nisso, ambos os campos ficam inacessíveis para código externo, mas enquanto registro só pode ser utilizado no código de Pessoa, nome pode ser acessado nos descendentes. Caso não seja definido o nível de acesso, ele é considerado como público (**public**), podendo ser utilizado também de forma externa.

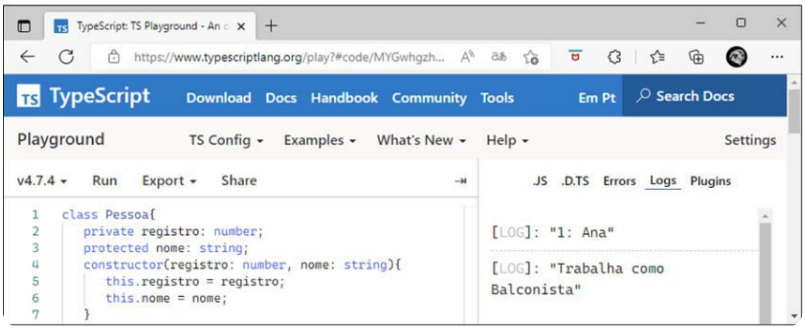
Enquanto a palavra **class** define uma classe, **extends** é utilizada para definir a herança, como no exemplo, onde temos Profissional como descendente de Pessoa. Os campos são acessados ao nível dos métodos internos, inclusive nos construtores, com o uso da palavra **this**, enquanto os métodos herdados são acessados com a utilização de **super**.

Podemos notar ainda um exemplo de **polimorfismo**, ao modificar o método **exibir** no nível do descendente. Originalmente, tínhamos apenas a impressão do registro e do nome no console, na classe Pessoa. Porém, em Profissional, acrescentamos a impressão da profissão. Note como o uso de **super** permite aproveitar o método exibir original, sendo apenas complementado.

Ao final do código, temos a definição de uma instância para a classe Profissional e invocação do método exibir. Para testar nosso exemplo, sem toda a complexidade de um projeto do tipo Angular, podemos executá-lo na ferramenta **TypeScript Playground**, que fica disponível no endereço a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Execução do exemplo no TypeScript Playground.

## Modelagem comportamental

Pode ser definida através de **elementos genéricos** ou baseada em decorações. Vamos à listagem!

```
typescript

function getArray(...items : T[] ) : T[] {
    return new Array().concat(items);
}

let vetor = getArray(1,2,3,4,5);
```

A função atua como um **template**, baseado em um tipo T qualquer, recebendo uma lista de valores do tipo determinado e retornando os elementos no formato de um array. Note ainda que o número de parâmetros é variável, devido ao uso de **reticências**, e que internamente os valores fornecidos são concatenados no array de retorno. Ao final, temos uma chamada com a passagem de valores **numéricos**, os quais são transformados em um **array** de **number**.

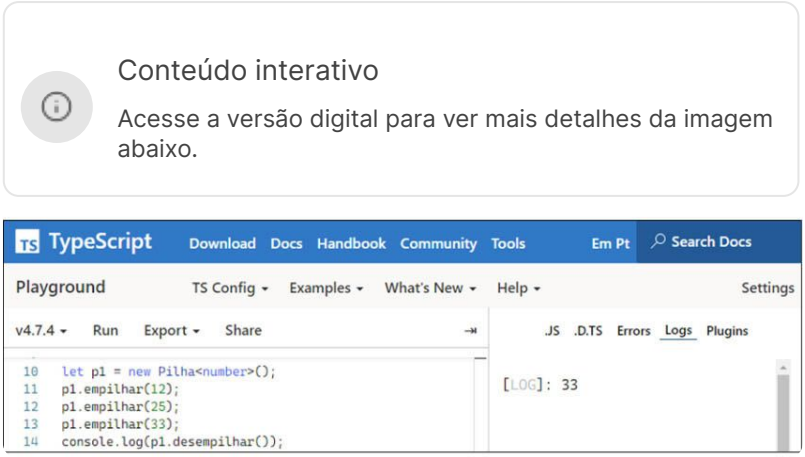
Mas não é apenas ao nível de funções que podemos definir templates. A próxima listagem irá demonstrar a criação de uma **classe template** para pilhas genéricas.

```
typescript

class Pilha {
   private repositorio: Array;
   constructor(){
      this.repositorio = new Array();
   }
   empilhar = (dado: T): void => { this.repositorio.push(dado); }
   desempilhar = (): T | undefined => this.repositorio.pop();
}

let p1 = new Pilha();
p1.empilhar(12);
p1.empilhar(25);
p1.empilhar(33);
console.log(p1.desempilhar());
```

Nossa classe **Pilha** é baseada em um tipo **T** e utiliza um repositório interno através de um array do tipo fornecido. No construtor, é instanciado o repositório, sendo utilizados os métodos **push** e **pop** do array, respectivamente, nos métodos **empilhar** e **desempilhar**. Note que o retorno de desempilhar pode ser do tipo fornecido ou indefinido, já que a pilha pode estar vazia, enquanto o método empilhar permite apenas a passagem de valores do tipo especificado.

Um exemplo de utilização é apresentado a seguir, com a criação de uma pilha de números, na qual são

empilhados os valores 12, 25 e 33. Em seguida, de acordo com o comportamento da pilha, é extraído o último valor e apresentado no console.




Execução do exemplo de pilha no TypeScript Playground.

Outra forma de modelar comportamentos é através de **anotações**, também conhecidas como **decorações**, que acrescentam funcionalidades a elementos já existentes, de uma forma pouco intrusiva. Na prática, temos **metadados** associados aos elementos do código, como atributos, métodos, classes, funções e até parâmetros, viabilizando o reconhecimento por ferramentas externas e injeção de funcionalidades. Um exemplo comum de utilização é na implementação do mapeamento objeto-relacional, facilitando a comunicação com o banco de dados.

Vejamos um exemplo de criação e utilização de decorações!

```
typescript

const impresso = (target: any, membro: string) => {
   console.log(membro);
};

class Produto {
   @impresso
   private nome: string = "Doug";
   @impresso
   getNome = () => this.nome;
}
```

A decoração é definida por meio de uma função, na qual o **target** recupera o elemento de código que foi decorado, e **membro** se refere ao nome desse elemento. Internamente, o que temos é a simples impressão do nome de uma propriedade ou método anotado.

Em seguida, temos uma classe com o atributo **nome** e o método **getNome**, ambos decorados com impresso. Sempre que o código for incluído em algum projeto, os nomes do atributo e do método serão impressos no console durante a execução.

Algo interessante no TypeScript é a abordagem **modular**, onde cada arquivo de código-fonte pode fornecer objetos, classes e funções para outros arquivos com o uso de **export**. Para que sejam utilizados em outro módulo, ou arquivo, devemos utilizar a palavra **import**.

Por conta do modelo de exportação e importação de funcionalidades adotado, podem ocorrer conflitos de nomes. Para resolver esse problema, temos os espaços de nomes (**namespaces**), que permitem diferenciar homônimos em diferentes módulos.

```typescript
namespace StringUtil {
   export const maiuscula = (str: string) => str.toUpperCase();

   export const minuscula = (str: string) => str.toLowerCase();
}
```

Os métodos exportados podem ser utilizados a partir de outro arquivo com o uso da diretiva para inclusão do **namespace**, como no exemplo apresentado a seguir. A diretiva é iniciada com **barra tríplice** e uma tag **reference**, tendo no atributo **path** o nome do arquivo TypeScript.

```typescript
///

console.log(StringUtil.maiuscula("Exemplo")); // A saída será EXEMPLO

console.log(StringUtil.minuscula("Exemplo")); // A saída será exemplo
```

## Modificadores

Para finalizar, temos ainda os modificadores **static**, que deixam o atributo ou método disponível a partir da classe, **readonly**, para definir um atributo que não pode ser escrito externamente, e abstract, utilizada para definir elementos abstratos no código. Aliás, tratando-se de elementos abstratos, temos também a possibilidade de definir conjuntos de assinaturas abstratas através da palavra reservada **interface**, exemplificada a seguir.

```typescript
interface IFuncionario {
    registro: number;
    nome: string;
    getSalario:() => number;
}

class Gerente implements IFuncionario {
    registro: number;
    nome: string;

    constructor(registro: number, nome: string) {
        this.registro = registro;
        this.nome = nome;
    }

    getSalario = () => 20000;
}

let emp: IFuncionario = new Gerente(1, "Steve");
console.log(emp.getSalario());
```

Aqui temos a definição da interface **IFuncionario**, com os atributos **registro** e **nome**, além de um método denominado **getSalario**, ocorrendo a implementação da interface na classe **Gerente**. Note que podemos receber uma instância de Gerente em uma variável do tipo IFuncionario.

# Atividade 2

Questão 1

Angular é um framework para desenvolver aplicações em diversas plataformas, mantido e desenvolvido pela Google. Ele vem com um conjunto de bibliotecas poderosas que podemos importar, possibilitando construir aplicações com uma qualidade e produtividade surpreendente. O Angular usa qual linguagem como padrão?

A Java

B Javascript

C TypeScript

D HTML

E XML

A alternativa C está correta.

O Angular é um framework JavaScript que simplifica não apenas a construção da interface de usuário, mas também o desenvolvimento de aplicações client-side diferenciadas, sejam elas para a web, mobile ou desktop. O Angular usa a linguagem TypeScript como padrão, ao invés do JavaScript.

# Roteamento no Angular

No modelo de **roteamento**, basicamente o que temos é uma **rota** definida a partir de um endereço inicial, que pode ser acessada através de algum método do protocolo **HTTP**, sendo interceptada por um **controlador**, para que seja retornado algum recurso do servidor.

Os projetos do Angular seguem o modelo Single Page Application (SPA), onde todo o aplicativo é descrito em uma única página, no caso index.html, ocorrendo a alternância dos componentes de visualização internos.

Neste vídeo, você compreenderá como funciona o roteamento no Angular, a partir da configuração de rotas e do componente de navegação.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Configuração de rotas

Um projeto com roteamento inclui o arquivo **app-routing.module.ts**, cujo código é replicado a seguir, definindo o módulo **AppRoutingModule**, que utilizará a biblioteca **RouterModule**, do núcleo do Angular, para o controle das rotas. No sistema do Angular, o tipo **Routes** serve para configurar as regras de navegação, e sua instância routes será utilizada na chamada do método **forRoot**, responsável pela inicialização do sistema de navegação.

```
typescript

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Configuramos as rotas no objeto **routes**, do arquivo **app-routing.module.ts**, como no exemplo seguinte, com a definição de três rotas, onde **home** leva para **HomeComponent**, **about** leva para **AboutComponent**, e a raiz direciona para home. Note que a segunda rota tem o parâmetro **id**, que poderá ser capturado ao nível do código TypeScript.

```
typescript

const routes: Routes = [
 { path: 'home', component: HomeComponent },
 { path: 'about/:id', component: AboutComponent },
 { path: '', redirectTo: '/home', pathMatch: 'full' },
];
```

## Componente de navegação

Para utilizar as rotas, devemos modificar o conteúdo de **app.component.html**, adotando algo mais simples e incluindo um menu de navegação, como na listagem seguinte.

```
python


  Principal
  Sobre


{{title}}
```

Temos um componente de navegação (nav), agrupando os links para as rotas especificadas, as quais são recuperadas a partir de **routerLink**. Cada vez em que ocorrer o clique sobre o link, teremos o componente alvo desenhado na área marcada com o seletor **router-outlet**.

Na chamada para a raiz do aplicativo, o conteúdo de **HomeComponent** é apresentado na área de **router-outlet**, devido ao direcionamento caracterizado pela terceira rota. A cada vez que clicamos em uma das

opções do menu, o conteúdo de **router-outlet** é modificado, enquanto o conteúdo de **AppComponent** permanece visível, permitindo a navegação.

# Atividade 3

Questão 1

Atualmente, temos meios para a definição de rotas de acesso aos recursos do sistema em diversas tecnologias de desenvolvimento, tanto no front-end quanto no back-end, e não seria diferente no Angular. Adotando o modelo de rotas, temos maior controle sobre a utilização dos recursos, permitindo que a autenticação e autorização sejam efetuadas de forma simples, já que ocorre um modo de acesso centralizado. Na estrutura do Angular, qual elemento é responsável pela exibição do conteúdo roteado?

A routerLink

B Routes

C ActivatedRoute

D Router

E router-outlet

A alternativa E está correta.

No modelo de roteamento do Angular, que é definido no módulo RouterModule, temos um objeto que efetua o redirecionamento, instância da classe Router, e configuramos as rotas disponíveis em uma coleção do tipo Routes. Os links que utilizam atributo navLink acessam as rotas, por meio do roteador, e direcionam o conteúdo roteado para a área especificada pelo seletor router-outlet, ficando este último responsável pela exibição. Podemos ainda transmitir valores pela rota, os quais são capturados na classe do componente a partir de ActivateRoute, ou rota ativa, que deve ser injetado no construtor e utilizado no método ngOnInit.

# Projeto PrimeiroApp

Nossa primeira prática envolverá a instalação do **Angular** e a criação de um aplicativo simples, com o nome **PrimeiroApp**, visando demonstrar os passos iniciais no uso do Angular.

Neste vídeo, você aprenderá a configuração do ambiente e a criação de app simples.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Roteiro de prática

1. A instalação do Angular utiliza o aplicativo **npm**, exigindo que o NodeJS já esteja instalado no ambiente. Para efetuar o download e instalação do NodeJS, basta acessar o endereço https://nodejs.org/en/, podendo ser utilizada a versão LTS.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

Site do NodeJS com opções de instalação.

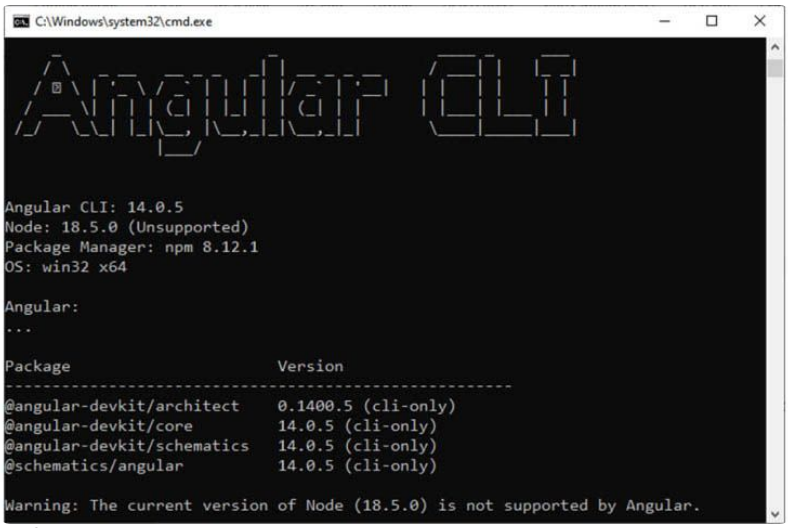
2. Nosso próximo passo será a instalação do **Angular CLI**, termo em que CLI significa interface de linha de comando. Utilizaremos o comando apresentado a seguir.

```
typescript

npm install -g @angular/cli
```

3. Para testar a instalação, basta executar o seguinte comando.

```
typescript

ng version
```

Vejamos o resultado!



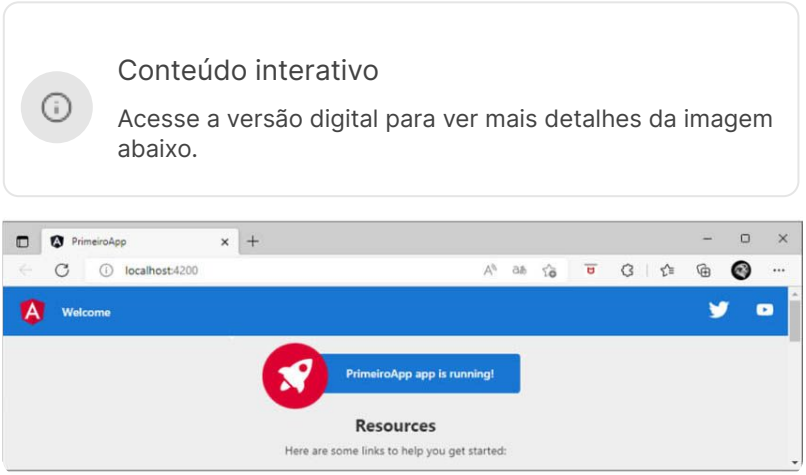
Verificação da instalação do Angular.

4. Instalação, para facilitar a programação, do **Visual Studio Code**, cujo download está disponível no endereço https://code.visualstudio.com/download.

5. Criação e execução do nosso primeiro projeto através da sequência de comandos apresentada na listagem seguinte. Todo projeto Angular será baseado em um diretório de mesmo nome, e por questões de organização, sugere-se definir um diretório de base para os projetos.

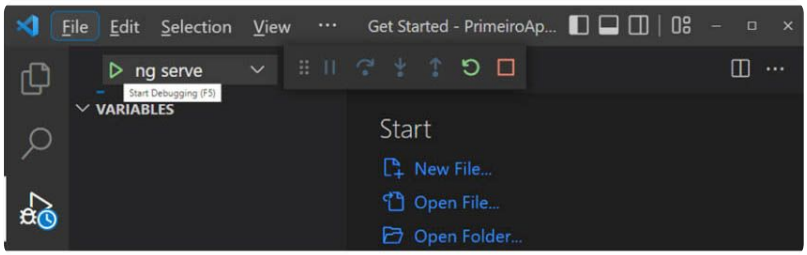
```
typescript

ng new PrimeiroApp
cd PrimeiroApp
ng serve --open
```

O acréscimo da opção **--open** ao comando irá abrir o navegador após a inicialização do servidor.



Execução do projeto PrimeiroApp.

6. Vamos tornar a edição simples com o **Visual Studio Code**, acessando o menu **File**, seguido da opção **Open Folder**, e escolhemos o diretório criado para o aplicativo.



Execução do projeto a partir do Visual Studio Code.

7. A execução do servidor poderá ser observada no painel inferior do editor, divisão **Terminal**, e o projeto será testado com o acesso ao endereço **localhost:4200** no navegador. Nosso ambiente está preparado para a codificação dos aplicativos.

8. Agora vamos alterar o projeto padrão do Angular para definir as características do aplicativo, a começar pelo componente principal, **AppComponent**, no arquivo **app.component.ts**, que fica no diretório **src/app**. Segundo o padrão adotado pelo Angular, todos os arquivos relacionados ao componente são agrupados em um diretório dentro de src. O novo conteúdo de **app.component.ts** é apresentado a seguir, em que o atributo **title** recebe o valor **PrimeiroApp**, sendo mantidas a anotação Component e os parâmetros originais.

```
typescript

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'PrimeiroApp';
}
```

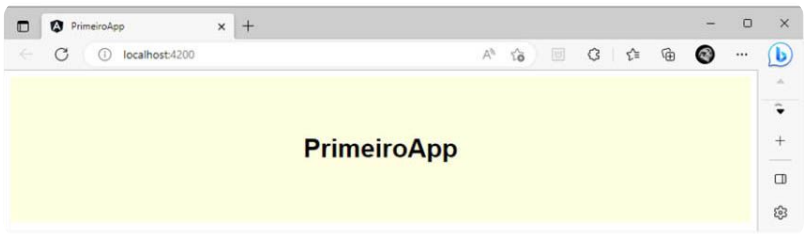
Encontraremos o modelo no arquivo **app.component.html**, nele devemos utilizar o conteúdo apresentado a seguir. Observe o uso de **chaves duplas** para a recuperação do valor que foi definido no atributo da classe AppComponent.

```
python


{{ title }}
```

9. Por último, vamos alterar a folha de estilos, no arquivo **app.component.css**, onde será definida a formatação utilizada na div identificada como **principal**.

```
python

*{font-family: Arial, Helvetica, sans-serif;}
#principal{
    background-color: lightyellow; padding: 50px;
    text-align: center;
}
```

Executando novamente o aplicativo, através comando **ng serve --open**, obteremos a saída a seguir.

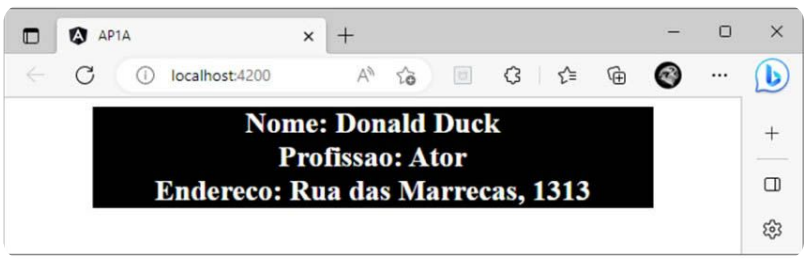



Execução do projeto PrimeiroApp.

# Atividade 4

Crie um projeto simples no Angular para apresentação dos seus dados pessoais. Os dados que serão exibidos deverão ser fornecidos a partir da classe do componente de exibição.

Ao final, você deverá fornecer uma saída como a que é apresentada a seguir.




Crie um projeto simples no Angular para apresentação dos seus dados pessoais. Os dados que serão exibidos deverão ser fornecidos a partir da classe do componente de exibição.

Ao final, você deverá fornecer uma saída como a que é apresentada a seguir.

## Chave de resposta

Comandos de linha para criação e configuração do projeto.

```
python

ng new AP1A
```

Arquivo app.component.ts.

```typescript
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  nome = 'Donald Duck';
  profissao = 'Ator';
  endereco = 'Rua das Marrecas, 1313';
}
```

Arquivo app.component.html.

```python
       Nome: {{ nome }}
       Profissao: {{ profissao }}
       Endereco: {{ endereco }}
```

Arquivo app.component.css.

```python
.container{
    display: flex; justify-content: center;
}
#pnlInfo{
    text-align: center;
    width: 500px; background: black;
}
.info{
    color: white; font-weight: bold;
    display: block; font-size: 26px;
}
```

## Projeto SegundoApp

Vamos à criação de um aplicativo que utiliza navegação, com o nome **SegundoApp**, visando demonstrar o roteamento.

Neste vídeo, aprenda como implementar um aplicativo com roteamento.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Roteiro de prática

1. Criação do projeto SegundoApp, com inclusão do sistema de roteamento do Angular.

```
typescript

ng new SegundoApp --routing
```

2. Criação de mais dois componentes em nosso aplicativo, **HomeComponent** e **AboutComponent**, entrando no diretório do projeto e executando os comandos apresentados a seguir. Serão criados os diretórios **home** e **about,** a partir de **src/app**, com todos os arquivos que fazem parte dos novos componentes.

```
typescript

cd SegundoApp
ng generate component Home
ng generate component About
```

3. Agora vamos modificar o objeto **routes**, no arquivo **app-routing.module.ts**.

```
typescript

const routes: Routes = [
 { path: 'home', component: HomeComponent },
 { path: 'about/:id', component: AboutComponent },
 { path: '', redirectTo: '/home', pathMatch: 'full' },
];
```

Temos três rotas, onde **home** leva para **HomeComponent**, **about** leva para **AboutComponent**, e a **raiz** direciona para **home**.

4. Modificação do conteúdo de **app.component.html**, definindo o menu de navegação e a área de exibição dos componentes.

```
python


  Principal
  Sobre


{{title}}
```

5. Modificação do conteúdo de **app.component.css**, para personalizar a formatação do **menu** e a fonte utilizada na página.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#menu {
    background-color: red; width:100%; height: 50px;
}
#menu a{
    color: white; text-decoration: none; width: 150px;
    margin-top: 15px; font-size: 20px;
    display: inline-block; text-align: center;
}
#menu a:hover{ text-decoration: underline; }
```

6. Modificação dos conteúdos de **about.component.css** e **home.component.css**, ambos adotando a listagem seguinte.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#principal{
    background-color: lightyellow; padding: 50px;
    text-align: center;
}
```

7. Modificação do modelo de **HomeComponent**, no arquivo **home.component.html** para que apresente um texto simples, como cabeçalho de nível dois, na div **principal**, formatada pela folha de estilo.

```python
<h2>VOCÊ ESTÁ NA HOME</h2>
```

8. Modificação do arquivo **about.component.ts** para a listagem apresentada a seguir, onde o controle da rota ativa é incluída, com a injeção de **ActivatedRoute** no construtor, permitindo a captura do parâmetro **id**.

```typescript
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-about',
  templateUrl: './about.component.html',
  styleUrls: ['./about.component.css']
})
export class AboutComponent implements OnInit {

  num: string | null = "";

  constructor(private route: ActivatedRoute) { }

  ngOnInit(): void {
    this.route.paramMap.subscribe(params => {
      this.num = params.get("id");
    });
  }
}
```

9. Modificação do arquivo **about.component.html**, de forma a refletir o valor capturado.

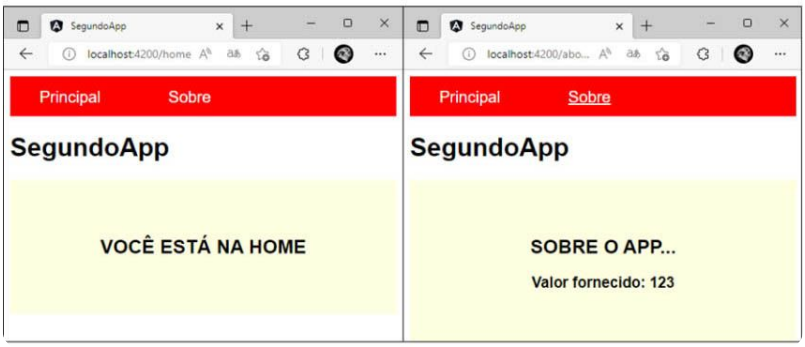
```python
# SOBRE O APP...

**Valor fornecido: {{num}}**
```

Temos mais um código HTML simples, no qual ocorre a ligação com o atributo **num**, definido no código TypeScript. Assim como para o atributo **title**, descrito para **AppComponent**, efetuamos o data binding com a utilização de chaves duplas.

10. Execução do aplicativo e verificação da navegação estabelecida.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



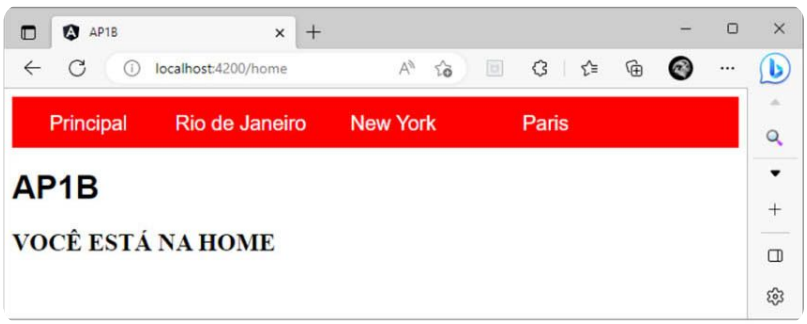

Telas do aplicativo Angular.

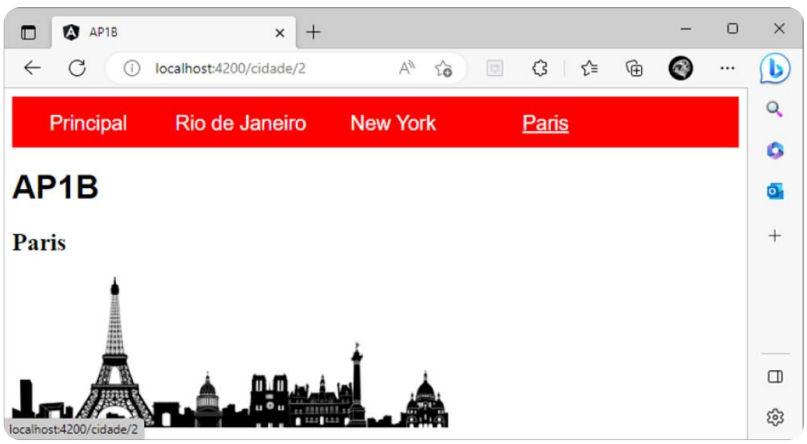
# Atividade 5

Nesta atividade, será explorado o sistema de roteamento do Angular. Crie um aplicativo com um menu de navegação apresentando o nome de 3 cidades e, para cada cidade escolhida, apresente na área principal o nome da cidade e uma foto dela.

As imagens referentes às cidades devem ficar no diretório **assets** do projeto, sendo acessadas por URLs iniciadas por /assests, como em **<img src='/assets/rio.png'/>**.

Ao final, você deverá fornecer uma saída como a que é apresentada nas duas telas a seguir.



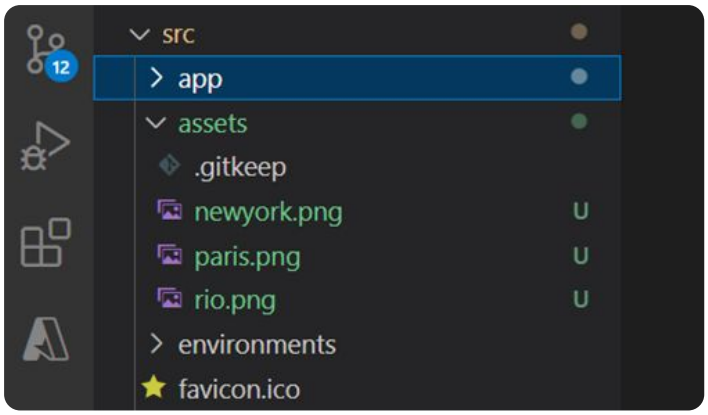




## Chave de resposta

Comandos de linha para criação e configuração do projeto.

```python
ng new AP1B --routing
cd AP1B
ng g c Home
ng g c TelaCidade
```

Baixe 3 imagens para as cidades, no formato png, e copie para o diretório assets do projeto.




Arquivo tela-cidade.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-tela-cidade',
  templateUrl: './tela-cidade.component.html',
  styleUrls: ['./tela-cidade.component.css']
})
export class TelaCidadeComponent implements OnInit {

  cidade = "";
  imagem = "";

  private nomes = ["Rio de Janeiro", "New York", "Paris"];
  private figuras = ["rio.png", "newyork.png", "paris.png"];

  constructor(private route: ActivatedRoute) { }

  ngOnInit(): void {
    this.route.paramMap.subscribe(params => {
      let id = params.get("id");
      const i = (id==null)?0:eval(id);
      this.cidade = this.nomes[i];
      this.imagem = this.figuras[i];
    });
  }
}
```

Arquivo tela-cidade.component.html.

```python
# {{cidade}}
```

Arquivo home.component.html.

```python
# VOCÊ ESTÁ NA HOME
```

Arquivo app-routing.module.ts.

```typescript
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { TelaCidadeComponent }
                    from './tela-cidade/tela-cidade.component';

const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'cidade/:id', component: TelaCidadeComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' },
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Arquivo app.component.html.

```python

  Principal
  Rio de Janeiro
  New York
  Paris

# {{title}}
```

Arquivo app.component.css.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#menu { background-color: red; width:100%; height: 50px; }
#menu a{
    color: white; text-decoration: none; width: 150px;
    margin-top: 15px; font-size: 20px;
    display: inline-block; text-align: center;
}
#menu a:hover{ text-decoration: underline; }
```

# Construção de componentes

O framework Angular oferece um conjunto de módulos que podem ser combinados, de acordo com as funcionalidades do sistema, oferecendo o suporte necessário para um front-end robusto e organizado, com base em componentes que utilizam lógica em classes **TypeScript**, estrutura visual em **HTML** e estilização com **CSS**. Também podem ser definidos serviços para tarefas executadas em background. Temos uma ligação de dados simples entre o modelo HTML e a classe, com uso de serviços via **injeção de dependências**, trazendo um modelo de construção simples e organizado, que devemos compreender para implementar nosso front-end.

Neste vídeo, você verá como realizar a construção de componentes, através de sua estrutura, ligação de dados e interface OnInit.

Conteúdo interativo

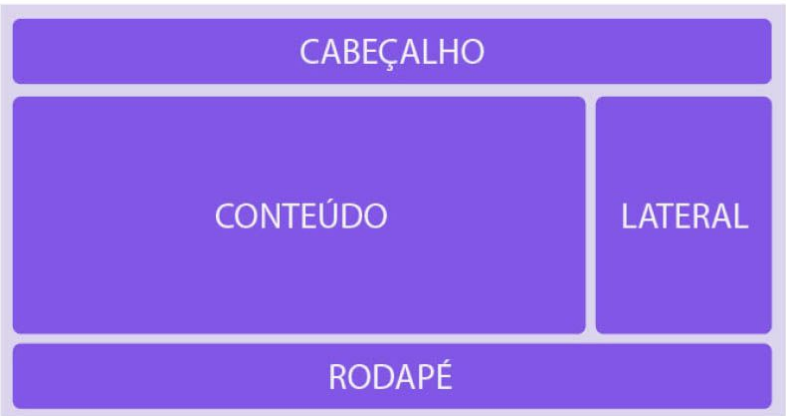
Acesse a versão digital para assistir ao vídeo.

## Estrutura dos componentes

Em termos práticos, um componente é construído com base em três arquivos, sendo um código de execução em **TypeScript**, um modelo em **HTML** e uma **folha de estilos**.

A utilização de componentes permite grande reutilização ao longo das páginas, facilitando a manutenção e diminuindo o esforço de programação. Por exemplo, um componente pode ser definido para apresentar informações gerais do aplicativo, na área do rodapé, permitindo o reaproveitamento em todas as páginas com o simples acréscimo de um **seletor** no HTML.

Suponha uma página com quatro áreas, segundo o padrão de mercado e as **tags semânticas** do HTML, as quais seriam: cabeçalho (**header**), lateral (**aside**), conteúdo (**main**) e rodapé (**footer**). O menu de navegação poderia ser fixado no cabeçalho, ao nível do componente principal, com o nome **AppComponente**, enquanto as demais áreas seriam alteradas dinamicamente.




Visão esquemática do aplicativo.

Adotando o componente raiz como responsável pelo cabeçalho e rodapé, enquanto o conteúdo restante varia de acordo com a rota escolhida, teríamos um modelo HTML como o seguinte.

```python


   Home
   Produtos
```

A arquitetura de componentes do Angular permite o reúso com a simples adição de **seletores** ao nível do modelo HTML. Observando o layout sugerido, com a navegação teríamos a alteração do conteúdo, enquanto a **lateral** e o **rodapé** seriam reutilizados, como pode ser visto já no nível do componente raiz, onde temos o seletor **app-rodape** na parte final do modelo.

## Ligação de dados

Além de permitir a composição de forma simples, os componentes do Angular permitem que o modelo HTML acesse os atributos da classe TypeScript, segundo o processo conhecido como **data binding**, ou ligação de dados. Por exemplo, a classe do rodapé poderia conter os atributos para o nome da empresa e o ano de desenvolvimento.

```typescript
@Component({
  selector: 'app-rodape',
  templateUrl: './rodape.component.html',
  styleUrls: ['./rodape.component.css']
})
export class RodapeComponent implements OnInit {
  empresa: string = "Calango";
  ano: number = 2022;

  constructor() { }
  ngOnInit(): void { }
}
```

Os atributos seriam recuperados facilmente no modelo, com a utilização de chaves duplas.

```python


©{{empresa}} {{ano}}
```

Com a folha de estilos adequada, teríamos um rodapé como o apresentado a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

Exemplo de ligação de dados no componente de rodapé.

Outro tipo de ligação de dados ocorre ao nível do seletor, mas agora no sentido inverso, em que o seletor fornece o valor do atributo. Por exemplo, para que o componente da divisão lateral exiba o nome da página corrente, devemos decorar o atributo com **Input**.

```
typescript

@Component({
  selector: 'app-lateral',
  templateUrl: './lateral.component.html',
  styleUrls: ['./lateral.component.css']
})
export class LateralComponent {
  @Input() pagina: string = "Indefinida"
}
```

Para exibir o atributo no modelo, a ligação de dados segue o mesmo padrão anterior.

```
python



Localização


    {{pagina}}
```

No entanto, o valor do atributo é preenchido através do **seletor**, como no fragmento de código apresentado a seguir, para um modelo HTML que englobe a lateral.

```
python



Bem-vindo!
```

É importante saber que, para definir atributos nos seletores, basta utilizar o decorador **Input** nos atributos da classe do componente.

Configurando as folhas de estilo dos componentes, teríamos como resultado uma saída como a apresentada a seguir.

Exemplo com fornecimento de dados via seletor.

## Interface OnInit

Permite a inicialização dos atributos de um componente por meio do método ngOnInit. Vejamos um exemplo!

```typescript
@Component({
  selector: 'app-exemplo',
  templateUrl: './exemplo.component.html',
  styleUrls: ['./exemplo.component.css']
})
export class ExemploComponent implements OnInit {
  ano: number = 0;

  constructor() { }

  ngOnInit(): void {
    let d: Date = new Date();
    this.ano = d.getFullYear();
  }
}
```

Para o exemplo, temos a criação de um objeto do tipo **Date**, com data e hora do sistema, e a extração do **ano** corrente para o atributo do componente.

A injeção de dependências ocorre através do construtor, na classe do componente, permitindo o acréscimo de funcionalidades de modo simples. Por exemplo, para utilizar o roteamento no código, precisamos apenas injetar um objeto do tipo **Router**.

Após injetar um Router, a navegação pode ser efetuada através do método **navigateByUrl**, como no exemplo seguinte, no qual o método **incluir** navegaria para um componente de inclusão.

```typescript
@Component({
  selector: 'app-produto',
  templateUrl: './produto.component.html',
  styleUrls: ['./produto.component.css']
})
export class ProdutoComponent {
  router: Router;

  constructor(router: Router) { this.router = router; }

  incluir():void{
    this.router.navigateByUrl("/novoprod")
  }
}
```

Assim como ocorre para os atributos, efetuamos a ligação do modelo HTML com os métodos da classe TypeScript, mas agora a partir dos eventos do HTML.

```python
Listagem de Produtos
                                                Novo Produto
```

No exemplo que foi apresentado, um botão do modelo HTML estaria acionando o método **incluir** quando uma ação de clique ocorrer.

Saiba que, para chamar um método da classe, podemos associá-lo ao clique sobre um botão ou outro componente HTML, através da inclusão do atributo click, colocado entre parênteses.

## Atividade 1

Questão 1

Não são raras as situações em que uma página precisa receber parâmetros para sua exibição, como no uso de parâmetros em uma requisição HTTP, ou como parte do endereço de acesso. Em termos da sintaxe HTML, os parâmetros de configuração de um elemento são fornecidos por meio de seus atributos, e o Angular permite a mesma abordagem no uso de seletores, mantendo o modelo de construção padrão do HTML. Qual anotação deve ser utilizada em campo da classe TypeScript para que seja alimentado por um atributo do seletor?

A Input

B Component

C Output

D Injectable

E NgModule

A alternativa A está correta.

Adicionando a anotação (decoração) Input a um campo da classe TypeScript, são eliminadas as indicações de erro por não inicializar a variável, e o valor passa a ser fornecido por meio de um atributo do seletor. O fluxo de dados inverso, ou seja, da classe para o modelo, seria viabilizado pela anotação Output, como no caso dos emissores de eventos. As demais opções são anotações estruturais, utilizadas para a definição de módulos (NgModule), componentes do Angular (Component) e serviços (Injectable).

# Serviços no Angular

Na arquitetura do Angular, os **serviços** são responsáveis por concentrar a lógica de **negócios**, separando-a dos **componentes**, que devem se concentrar apenas na **interface**. Nada impede que os dados sejam manuseados ao nível das classes TypeScript de nossos componentes, mas o uso de serviço melhora a organização do código e a especialização por perfil de desenvolvedor.

Neste vídeo, você entenderá como funcionam os serviços no Angular, observando a arquitetura para injeção de dependências, as diretivas e o uso de serviços, além das rotas e da autenticação.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Arquitetura do Angular para injeção de dependências

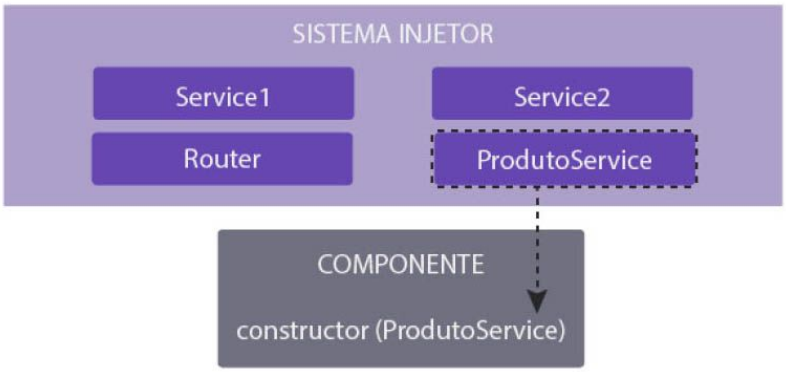
Além das possibilidades de **injeção de dependências** de outras plataformas, com base no uso de anotações, no Angular temos a possibilidade de injetar dependências no **construtor** da classe, a partir de seus parâmetros, algo que já fizemos ao trabalhar com o **router**. Esse é o modelo de utilização de serviços do Angular, no qual eles são decorados como **Injectable**, sendo recuperados nos componentes através de seus construtores.

A partir do sistema injetor, ou **Injector**, os diversos serviços registrados são disponibilizados para os componentes, e quando o construtor tem um parâmetro do tipo do serviço, ele é recuperado automaticamente. Em outras palavras, ocorre a injeção do serviço no parâmetro do construtor, caracterizando o fluxo de execução do Angular no que se refere à injeção de dependência.

Toda essa arquitetura é apresentada a seguir, de modo que temos a utilização de um serviço, com o nome **ProdutoService**, que é fornecido a partir do **Injector**, através da injeção de dependência no parâmetro do construtor em um componente qualquer.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Arquitetura do Angular para injeção de dependências.

Vamos exemplificar o uso de serviços, iniciando com a definição da classe **Produto**.

```
typescript

export class Produto {
   codigo: string = "";
   nome:   string = "";
   quantidade: number = 0;
}
```

Nossa classe **Produto** tem apenas os campos para representar **código**, **nome** e **quantidade** de cada produto de um repositório qualquer, sendo os dois primeiros do tipo texto e o último um valor numérico. Podemos configurar o repositório, ao nível de nosso serviço, deixando disponível para todo o aplicativo através da injeção de dependência.

```
typescript

import { Injectable } from '@angular/core';
import { Produto } from './produto';

@Injectable({
   providedIn: 'root'
})
export class ProdutoService {
   private produtos: Array

   constructor() { this.produtos = new Array(); }

   adicionar(produto: Produto): void{
      this.produtos.push(produto);
   }
   obterTodos = ():Array => this.produtos;
}
```

Nosso **repositório** interno será um elemento do tipo **Array**, como um atributo privado da classe **ProdutoService**, utilizando o nome **produtos**. O repositório é instanciado vazio no construtor do serviço, devendo ser observado que adotamos a tipagem para **Produto** na definição do **Array**.

A inclusão ocorre no método **adicionar**, tendo como parâmetro um **produto**, onde o método **push**, da classe **Array**, acrescenta o produto no final de **produtos**. A consulta é fornecida pelo método **obterTodos**, com o retorno do conjunto interno para o chamador.

Devemos ter atenção especial para a decoração **Injectable**, adotada para que o serviço possa ser

disponibilizado através do sistema de injeção de dependência. No caso, o serviço está sendo habilitado ao nível da raiz (**root**), devendo ficar disponível para todo o aplicativo.

Embora a decoração habilite a injeção de dependência, ainda é necessário configurar o serviço no módulo, com a modificação do arquivo **app.module.ts**, no atributo **providers** da decoração **NgModule**, de acordo com o fragmento de código apresentado a seguir, onde nosso serviço é adicionado ao conjunto de provedores do módulo. Também temos que adicionar a **importação** de **ProdutoService**, a partir do arquivo **produto.service**.

```
typescript

// As demais importações são mantidas!

import { ProdutoService } from './produto.service';

@NgModule({

// As demais configurações são mantidas!

  providers: [ProdutoService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Para acessar o serviço, basta referenciar no **construtor**.

```
typescript

servProd: ProdutoService;

constructor(servProd: ProdutoService) {
   this.servProd = servProd;
}
```

O processo é muito simples com a definição de um atributo interno para referenciar o serviço, e a configuração do atributo no construtor, a partir do parâmetro de mesmo tipo, que recebe o provedor configurado para o módulo através da injeção de dependência.

## Diretivas e uso de serviços

O acesso aos serviços será efetuado a partir de métodos construtores, ou seja, ocorrerá a partir da classe de cada componente. Para disponibilizar os dados, que são capturados ao nível da classe, no modelo HTML, trabalhamos com a vinculação de dados e algumas diretivas, as quais são particularmente úteis para conjuntos com quantidade variável de entidades.

As **diretivas** do Angular serão adotadas nos modelos para a tomada de decisão e ações especiais. Elas são marcadores aplicados aos elementos do HTML, na forma de atributos, para anexar algum comportamento específico, da mesma forma que as anotações, ou decorações, são utilizadas no código **TypeScript**. Algumas diretivas são oferecidas na forma de **seletores**, atuando como elementos do HTML.

Veja, agora, quais são as diretivas do Angular e suas respectivas utilizações!

- **nglf**: define uma condição para o desenho do componente HTML.

- **ngFor**: define a regra para a repetição do desenho de um componente do HTML.

- **ngSwitch**: condiciona o desenho de componentes HTML ao valor de uma variável, devendo ser usada em conjunto com **ngSwitchWhen** e **ngSwitchDefault**.

- **ngModel**: estabelece uma vinculação bidirecional com um atributo do controlador.

- **ngClass**: gerencia dinamicamente a classe CSS aplicada ao componente HTML.

- **ngStyle**: define o estilo dinamicamente, através da ligação de dados com a classe.

- **ng-app**: seletor que identifica o aplicativo Angular.

- **ng-template**: seletor que define um trecho reutilizável no modelo HTML.

É fácil imaginar que a diretiva **ngFor** deverá ser utilizada na listagem de produtos, exibindo as múltiplas ocorrências de produtos através de painéis. Devemos iniciar a alteração na classe do componente.

```typescript
@Component({
selector: 'app-produto',
templateUrl: './produto.component.html',
styleUrls: ['./produto.component.css']
})
export class ProdutoComponent implements OnInit {
servProd: ProdutoService;
produtos: Array = [];
constructor(servProd: ProdutoService) {
this.servProd = servProd;
}
ngOnInit(): void { this.produtos = this.servProd.obterTodos(); }
}
```

Temos a alimentação de **produtos** sempre que a rota da página é acessada, o que é viabilizado com a implementação da interface **OnInit**, através de seu método **ngOnInit**. Ocorre a chamada para o método **obterTodos**, a partir de **servProd**, substituindo o valor do atributo, o que será refletido diretamente na página por causa da vinculação de dados.

Para exibir os produtos no modelo HTML, será necessário utilizar a diretiva **ngFor**.

```python
        {{produto.codigo}}

        {{produto.nome}}

        Quantidade: {{produto.quantidade}}
```

No exemplo, a **div** com classe **pnlProduto** será responsável pela exibição dos dados para um produto específico. Nela foi adicionada a diretiva **ngFor**, onde percorremos todo o conjunto definido em **produtos**, refletindo cada elemento na variável **produto**, enquanto no conteúdo **interno** da div que é repetida, temos o preenchimento de dados a partir do objeto da iteração, seguindo a forma de vinculação padrão, com **chaves duplas**.

Confira um possível resultado da execução!

1. **Banana**

Quantidade: 1000

2. **Laranja**

Quantidade: 500

3. **Manga**

Quantidade: 800

Outro serviço é o gerenciador de **formulários**, que deve ser adicionado ao aplicativo no arquivo **app.module.ts**, como no fragmento seguinte, no qual vemos a importação de **FormsModule** e sua utilização no atributo**imports** da decoração **NgModule**.

```typescript
// As demais importações são mantidas!

import { FormsModule } from '@angular/forms';

@NgModule({

// As demais configurações são mantidas!

  imports: [
    BrowserModule,
    FormsModule,
    AppRoutingModule
  ],
  providers: [ProdutoService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

A configuração efetuada disponibiliza o suporte ao uso de formulários para todo o módulo, habilitando as diretivas necessárias para que ocorra a vinculação de dados **bidirecional** nos campos de entrada, já que na forma padrão temos apenas o modelo sendo alimentado pela classe do componente, e agora será possível atualizar os atributos a partir dos valores digitados.

A classe de um possível componente para inclusão de produtos pode ser observada a seguir. Veja!

```typescript
@Component({
  selector: 'app-nov-prod',
  templateUrl: './nov-prod.component.html',
  styleUrls: ['./nov-prod.component.css']
})
export class NovProdComponent implements OnInit {
  servProd: ProdutoService;
  produto: Produto = new Produto();

  constructor(servProd: ProdutoService) {
     this.servProd = servProd;
  }
  incluir():void{
     this.servProd.adicionar(this.produto);
  }
  ngOnInit(): void {
  }
}
```

Podemos observar a utilização de **ProdutoService** no método **incluir**, com a chamada para o método **adicionar**, tendo como parâmetro o atributo **produto**, o que causa a inserção de seus dados no repositório.

O **modelo** necessário, com inclusão da diretiva **ngModel**, para habilitar **vinculação bidirecional**, pode ser observado na listagem seguinte.

```python
Novo Produto

Código:

Nome:

Quantidade:

Incluir
```

Note que temos todos os campos de entrada configurados com a diretiva **ngModel**, onde cada um dos campos de entrada faz referência ao atributo equivalente de **produto**, definido na classe. Qualquer alteração efetuada no formulário será refletida nos campos de produto, da mesma forma que as alterações de produto via código serão refletidas nos campos do formulário, o que caracteriza a vinculação bidirecional.

> ⚠
>
> Atenção
>
> As classes dos componentes são classificadas como controladores, pois a lógica de negócios é implementada neles, enquanto o modelo se concentra na interface do usuário.

Outro elemento importante é o modelo de interação para envio, pois temos a chamada para o método **incluir** a partir de um evento **submit** do **formulário**. Com a configuração efetuada, o botão de envio deverá ser do **tipo** submit, iniciando a submissão do formulário no clique, com a consequente invocação do método incluir.

## Rotas e autenticação

Para o acesso às áreas restritas de um sistema, precisamos de um **usuário válido**, confirmado por algum método de **autenticação**. Na forma mais simples, temos uma tabela de usuários no banco de dados, onde a senha é armazenada com criptografia destrutiva, mas existem outros métodos, como autenticação por terceiros e utilização do usuário de rede via Active Directory.

Quando nosso usuário está armazenado em um banco de dados, precisamos de uma tela de login, já que a senha terá que ser digitada cada vez que for iniciada uma sessão. Outras formas de autenticação podem se basear em **tokens**, os quais também podem ser utilizados quando não há recursos para estabelecimento de sessão na tecnologia servidora utilizada, ou quando adotamos um autenticador intermediário, como o Google, segundo o protocolo **OAuth**.

Atenção

Token, que significa ficha, é uma identificação longa, normalmente criptografada, gerada em um servidor e fornecida ao cliente para acessar um grupo de recursos.

Mesmo um usuário válido pode não ter acesso a determinados recursos, como um aluno tentando alterar notas em um sistema acadêmico, enquanto o papel seria delegado a um professor. Nesse ponto, precisamos definir **perfis**, cada qual com **autorização** de acesso para um grupo de recursos específicos dentro do sistema.

Ao lidarmos com **rotas**, temos um padrão de acesso aos recursos do sistema, que inicialmente traz uma proteção efetiva contra o acesso indiscriminado aos diretórios internos. Como as rotas definem os pontos de acesso ao sistema, elas acabam se tornando o melhor local para proteger um recurso de qualquer acesso não autorizado.

No ambiente do Angular, as rotas podem ser protegidas de forma muito simples, com base em um **serviço** que implemente a interface **CanActivate**, vejamos!

```typescript
@Injectable({
  providedIn: 'root'
})
export class LoginService implements CanActivate{
  router: Router;
  isAuthenticated: boolean = false;

  constructor(router: Router) { this.router = router; }

  canActivate(route: ActivatedRouteSnapshot,
              state: RouterStateSnapshot):
         boolean | UrlTree | Observable |
         Promise {
    if(!this.isAuthenticated)
       return this.router.parseUrl("/login");
    else
       return true;
  }
  signin(login: string, senha: string): boolean {
    if((usuario=="admin")&&(usuario.senha=="1234")){
       this.usuarioLogado = usuario;
       this.isAuthenticated = true;
       return true;
    } else {
       return false;
    }
  }
}
```

O novo serviço permite gerenciar o login pelo método **signin**, controlando a autenticação, além de implementar a interface **CanActivate**, para a **autorização** de rotas. Já que o Angular é uma tecnologia de **front-end**, não temos gerência de sessão, mas como os serviços ficam ativos durante toda a execução do sistema, eles podem ser utilizados para manter dados no escopo global, sendo recuperado através da injeção de dependência.

A interface **CanActivate** foi implementada através do método **canActivate**, o qual apresenta um retorno bastante complexo, baseado em **tipos múltiplos**, sendo verificado se existe um usuário autenticado através do valor de **isAuthenticated**. Se ocorreu uma autenticação bem-sucedida, utilizando **admin** para o login e **1234** para a senha, temos o retorno do valor **true**, liberando a rota protegida, e no caso contrário é fornecida a rota para a **tela de login**.

Um exemplo de classe para o componente de login é apresentado a seguir. Ela deve receber os dados do usuário e chamar o método **signin**, de **LoginService**.

```typescript
@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent {
  router: Router;
  servLogin: LoginService;
  login: string = "";
  senha: string = "";
  mensagem: string = "";

  constructor(router: Router, servLogin: LoginService) {
    this.router = router;
    this.servLogin = servLogin;
  }

  signin(): void{
    if(this.servLogin.signin(this.login,this.senha))
       this.router.navigateByUrl("/produto");
    else
       this.mensagem = "Login ou senha inválido(s).";
  }
}
```

No construtor temos a injeção de dependências de **Router** e **LoginService** nos atributos **router** e **servLogin**, respectivamente. O processo de autenticação irá envolver a verificação do login e senha, com redirecionamento para a tela protegida em uma autenticação bem-sucedida.

Após a definição de LoginService e do componente de login, é necessário configurar as rotas que irão requerer autorização no objeto **routes**, do arquivo **app-routing.module.ts**, observe!

```typescript
const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  { path: 'produto', component: ProdutoComponent,
    canActivate: [LoginService] },
  { path: 'novoprod', component: NovProdComponent,
    canActivate: [LoginService] },
  { path: '', redirectTo: '/home', pathMatch: 'full' },
];
```

A diferença das rotas protegidas é o acréscimo da condição **canActivate**, tendo como base **LoginService**, efetuando o bloqueio automático e o redirecionamento para a tela de login, quando o usuário não estiver autenticado.

# Atividade 2

Questão 1

Um problema comum nas páginas HTML é a adoção de um modelo estático, exigindo muito esforço para modificação e poucas possibilidades em termos de comportamento dinâmico, mas o framework Angular resolve esse problema através do uso de diretivas. As diretivas oferecem muito controle sobre o gerenciamento e a criação dos elementos HTML, como no caso em que desejamos exibir os dados de uma coleção dinâmica, onde os elementos podem variar para cada chamada efetuada. Qual a diretiva correta para exibir os dados de uma coleção dinâmica?

A ngIF

B ngClass

C ngStyle

D ngFor

E ngSwitch

A alternativa D está correta.

A diretiva correta seria ngFor, que deve receber uma expressão para percorrer a coleção, com base na instrução for e no operador of do Java Script, viabilizando o uso dos dados do iterador no preenchimento do conteúdo de um trecho HTML que se repete.

# Projeto TerceiroApp

Criaremos um projeto com o nome **TerceiroApp**, no qual teremos um repositório de produtos em memória, gerenciado por meio de um serviço, além do acréscimo de proteção para as rotas que envolvem consulta e manuseio dos produtos.

Neste vídeo, você aprenderá a implementar um aplicativo.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Roteiro de Prática

1. Vamos criar o projeto **TerceiroApp** usando o comando apresentado a seguir.

```
typescript

ng new TerceiroApp --routing
```

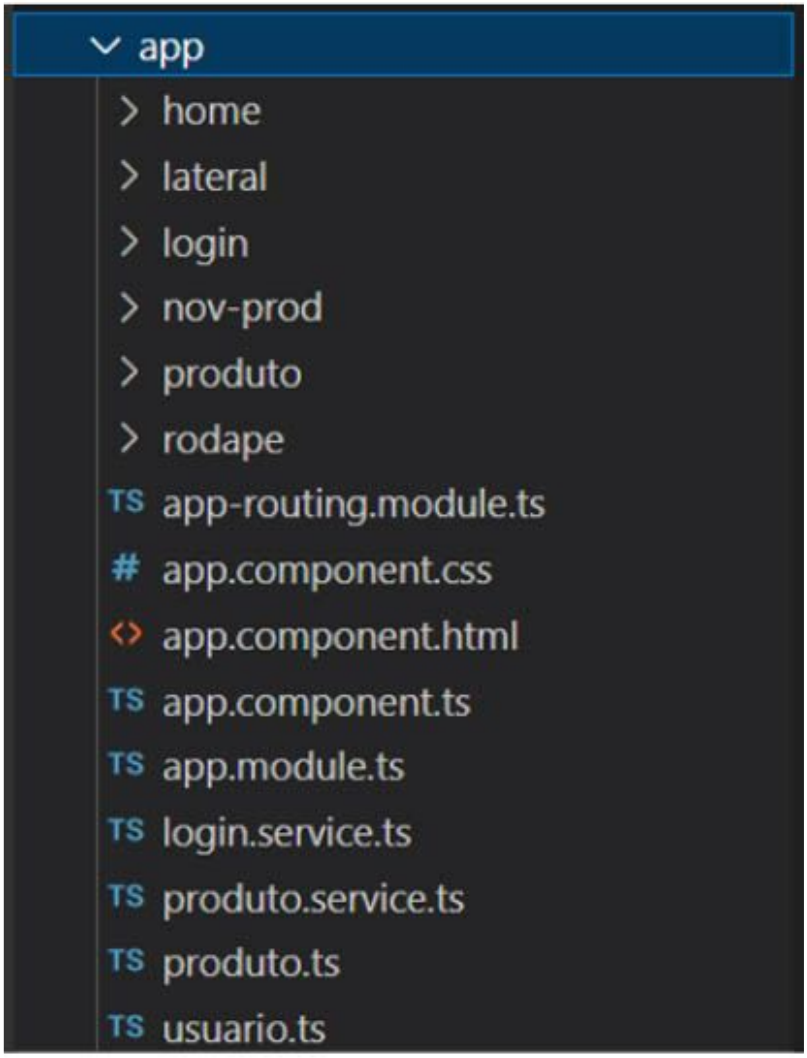
2. Vamos acrescentar os componentes do sistema, com a divisão das áreas de visualização em: **cabeçalho**, **conteúdo**, **lateral** e **rodapé**.

```
typescript

cd TerceiroApp
ng g c Rodape
ng g c Lateral
ng g c Home
ng g c Produto
ng g c NovProd
ng g c Login
```

3. Vamos adicionar os serviços e classes de entidade necessários para nosso projeto, com a execução, ainda no diretório **TerceiroApp**, dos comandos a seguir.

```
typescript

ng g s Produto --flat true --skip-tests true
ng g s Login --flat true --skip-tests true
ng g class Produto --skip-tests true
ng g class Usuario --skip-tests true
```

A estrutura de nosso projeto deverá estar desta forma:




Estrutura do projeto TerceiroApp.

4. Precisamos incluir os serviços e módulos complementares no módulo principal, modificando o conteúdo de **app.module.ts**.

```typescript
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { RodapeComponent } from './rodape/rodape.component';
import { LateralComponent } from './lateral/lateral.component';
import { HomeComponent } from './home/home.component';
import { ProdutoComponent } from './produto/produto.component';
import { NovProdComponent } from './nov-prod/nov-prod.component';
import { LoginComponent } from './login/login.component';
import { ProdutoService } from './produto.service';
import { LoginService } from './login.service';

@NgModule({
  declarations: [
    AppComponent,
    RodapeComponent,
    LateralComponent,
    HomeComponent,
    ProdutoComponent,
    NovProdComponent,
    LoginComponent
  ],
  imports: [
    BrowserModule,
    FormsModule,
    AppRoutingModule
  ],
  providers: [ProdutoService, LoginService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

As alterações necessárias são apenas a inclusão de **FormsModule** no atributo **imports** e dos serviços criados em **providers**, além das respectivas importações para as classes TypeScript.

5. Para criar uma identidade visual em nosso aplicativo, vamos baixar o ícone do Angular, a partir do endereço https://angular.io/presskit, no formato PNG (*Portable Network Graphics*), que já utiliza transparência e oferece compatibilidade com diversas plataformas. O arquivo baixado, com o nome **angular.png**, deve ser copiado para o diretório **assets** de nosso projeto. Também devemos baixar uma imagem para representar o usuário logado, onde uma opção interessante está disponível no endereço https://www.onlinewebfonts.com/icon/311846, renomeando o arquivo para **login.png** e copiando para o diretório **assets**.

6. Como teremos algumas rotas protegidas, vamos iniciar a programação pela entidade **Usuario**, no arquivo **usuario.ts**.

```typescript
export class Usuario {
   public login: string;
   public senha: string;
   public nome:  string;
   public email: string;
   constructor(login: string = "", senha: string = "",
               nome: string = "", email: string=""){
      this.login = login;
      this.senha = senha;
      this.nome = nome;
      this.email = email;
   }
}
```

7. Nosso próximo passo será a programação de **LoginService**, no arquivo **login.service.ts**, com base no conteúdo apresentado a seguir.

```typescript
import { Injectable } from '@angular/core';
import { ActivatedRouteSnapshot, CanActivate, Router,
         RouterStateSnapshot, UrlTree } from '@angular/router';
import { Observable } from 'rxjs';
import { Usuario } from './usuario';

@Injectable({
  providedIn: 'root'
})
export class LoginService implements CanActivate{
  private usuarios = [
    new Usuario("admin","1234","Administrador","admin@calango.com"),
    new Usuario("ana","1234","Ana Maria","ana@calango.com"),
    new Usuario("luiz","1234","Luiz Carlos","luiz@calango.com")]
  router: Router;
  isAuthenticated: boolean = false;
  usuarioLogado: Usuario = new Usuario();

  constructor(router: Router) { this.router = router; }

  canActivate(route: ActivatedRouteSnapshot,
              state: RouterStateSnapshot):
        boolean | UrlTree | Observable |
        Promise {
    if(!this.isAuthenticated)
       return this.router.parseUrl("/login");
    else
       return true;
  }

  signin(login: string, senha: string): boolean {
    let usuario = this.usuarios.filter((value)=>
                       value.login==login).pop();
    if((usuario!=null)&&(usuario.senha==senha)){
       this.usuarioLogado = usuario;
       this.isAuthenticated = true;
       return true;
    } else {
       return false;
    }
  }

  signout(){
    this.usuarioLogado = new Usuario();
    this.isAuthenticated = false;
  }
}
```

8. O serviço oferece as opções de **signin** e **signout**, além de implementar a interface **CanActivate**, para a **autorização** de rotas. Para oferecer um conjunto de usuários válidos em nossos testes, vamos utilizar um repositório local, configurado no atributo **usuários** de **LoginService**, onde temos três opções de login.

Podemos configurar as rotas via objeto **routes**, já utilizando a proteção do **canActivate**, no arquivo **app-routing.module.ts**.

```typescript
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { LoginComponent } from './login/login.component';
import { ProdutoComponent } from './produto/produto.component';
import { LoginService } from './login.service';
import { NovProdComponent } from './nov-prod/nov-prod.component';

const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  { path: 'produto', component: ProdutoComponent,
    canActivate: [LoginService] },
  { path: 'novoprod', component: NovProdComponent,
    canActivate: [LoginService] },
  { path: '', redirectTo: '/home', pathMatch: 'full' },
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

9. Vamos codificar **LateralComponent**, relacionado à divisão lateral (**aside**), no arquivo **lateral.component.ts**.

```typescript
import { Component, Input, OnInit } from '@angular/core';

@Component({
  selector: 'app-lateral',
  templateUrl: './lateral.component.html',
  styleUrls: ['./lateral.component.css']
})
export class LateralComponent implements OnInit {
  @Input() pagina: string = "Indefinida"
  acesso: string = ""

  constructor() { }

  addZero = (x: number) => (x>9) ? `${x}`:`0${x}`;

  formataData(d: Date): string {
    var dataStr = "";
    dataStr += this.addZero(d.getDate())+"/"+
               this.addZero(d.getMonth()+1)+"/"+
               d.getFullYear()+" "+
               this.addZero(d.getHours())+":"+
               this.addZero(d.getMinutes())+":"+
               this.addZero(d.getSeconds());
    return dataStr;
  }

  ngOnInit(): void {
    let d: Date = new Date();
    this.acesso = this.formataData(d);
  }
}
```

10. Vamos à configuração do modelo, no arquivo **lateral.component.html**.

```python
## Localização

    {{pagina}}

## Acesso

    {{acesso}}
```

Completaremos a programação do componente para a lateral, configurando a folha de estilos, no arquivo **lateral.component.css**.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#lateral{
   display:flex; flex-wrap: wrap; width:20%; min-height: 300px;
   justify-content: center; background-color: lightblue;
}
.mensagem{ width:100%; text-align:center; margin:0px; }
```

11. Vamos codificar a classe de **LoginComponent**, no arquivo **login.component.ts**, de acordo com a listagem seguinte. Ela será responsável por receber os dados da tela de login e acionar o método **signin**, de **LoginService**, com o fornecimento desses dados.

```typescript
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { LoginService } from '../login.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent implements OnInit {
  router: Router;
  servLogin: LoginService;
  login: string = "";
  senha: string = "";
  mensagem: string = "";

  constructor(router: Router, servLogin: LoginService) {
    this.router = router;
    this.servLogin = servLogin;
  }

  signin(): void{
    if(this.servLogin.signin(this.login,this.senha))
       this.router.navigateByUrl("/produto");
    else{
       this.mensagem = "Login ou senha inválido(s).";
       this.login = "";
       this.senha = "";
       this.router.navigateByUrl("/login");
    }
  }
  ngOnInit(): void {
  }
}
```

Vamos alterar o conteúdo do **modelo**, no arquivo **login.component.html**, para oferecer o suporte necessário à classe.

```python
Acesso Autenticado

{{mensagem}}

Login:

Senha:

Login
```

12. Vamos alterar o arquivo **login.component.css**.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#conteudo{
   display:flex; flex-wrap: wrap; width:80%; float:left;
   justify-content: center; align-items: center;
}
label, button{
   margin:5px; width: 100px; display: inline-block;
}
#mensagem{
   color:red; width:98%; background-color:lightyellow;
   padding:5px; border:red 1px dotted; font-weight: bold;
}
```

13. Vamos configurar o **rodapé** no arquivo **rodape.component.html**, onde temos apenas a tag semântica **footer**, com id **rodape**, o símbolo de copyright, a identificação de uma empresa fictícia e o ano de desenvolvimento.

```python
©Calango 2022
```

A folha de estilos do rodapé, no arquivo **rodape.component.css**, utilizará o conteúdo que é apresentado a seguir.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#rodape{
    background-color: red; color: white; font-weight: bold;
    text-align: center; padding: 5px; border: black dashed;
}
```

14. Vamos especificar o componente principal de nosso sistema, no qual temos o cabeçalho com o menu de navegação, a área dinâmica ao centro e o rodapé na parte inferior. Modificaremos a classe **AppComponent**, no arquivo **app.component.ts**.

```typescript
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { LoginService } from './login.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'TerceiroApp';
  router: Router;
  servLogin: LoginService;

  constructor(router: Router, servLogin: LoginService){
    this.router = router;
    this.servLogin = servLogin;
  }
  signout(): void{
    this.servLogin.signout();
    this.router.navigateByUrl("/home");
  }
}
```

Como estratégia para ativação do **signout**, vamos utilizar o clique sobre a imagem de usuário, o que será feito utilizando o conteúdo da listagem seguinte no arquivo **app.component.html**.

```
python

# {{title}}


    Usuário Logado:


  {{servLogin.usuarioLogado.nome}}

  {{servLogin.usuarioLogado.email}}




  Home
  Produtos
```

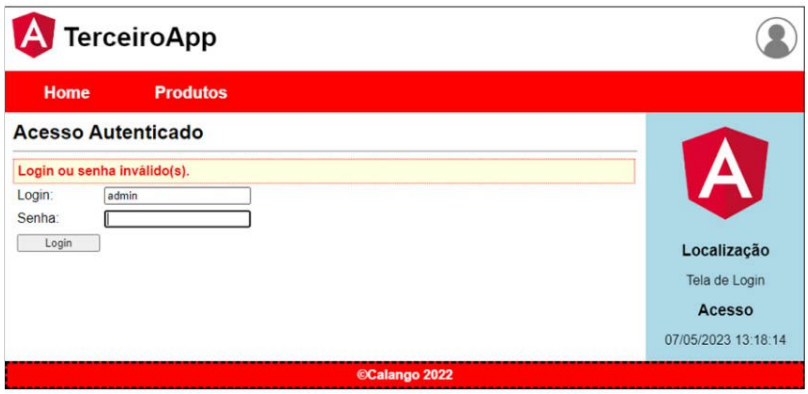
Vamos modificar nossa folha de estilos, no arquivo **app.component.css**.

```
python

*{font-family: Arial, Helvetica, sans-serif;}

#menu { background-color: red; width:100%; height: 50px; }
#menu a{
    color: white; text-decoration: none; width: 150px;
    margin-top: 15px; font-size: 20px; font-weight: bold;
    display: inline-block; text-align: center;
}
#menu a:hover{ text-decoration: underline; }

#topo{ display: flex; margin: 0 auto; }

#login{
    margin-left: auto; align-self:
    center; margin-right: 10px; opacity: 50%;
}
#logged{
    align-self: center; margin-right: 10px;
    margin-left:10px;
}
#descUsu{
    width:150px; align-self:center; margin-left: auto;
    padding: 2px; border-left: dotted black 1px;
}
```

15. Vamos executar nosso sistema e testar a autenticação, abrindo a tela de login com o clique sobre o menu de produtos, como pode ser observado a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Teste inicial do sistema de autenticação em TerceiroApp.

16. Para finalizar nosso projeto, vamos implementar os elementos relacionados à listagem, inserção e exclusão de produtos. Começaremos pela classe **Produto**, no arquivo **produto.ts**.

```
typescript

export class Produto {
   codigo: string = "";
   nome:   string = "";
   quantidade: number = 0;
}
```

17. Vamos voltar para a classe **ProdutoService**, no arquivo **produto.service.ts**, onde será utilizado o código seguinte.

```
typescript

import { Injectable } from '@angular/core';
import { Produto } from './produto';

@Injectable({
   providedIn: 'root'
})
export class ProdutoService {
   private produtos: Array

   constructor() { this.produtos = new Array(); }

   private getPos(codigo: string): number{
      for(let i=0; i-1)
         this.produtos.splice(pos,1);
   }
   obterTodos = ():Array => this.produtos;
}
```

18. Vamos implementar os componentes relacionados, a começar pela classe **NovoProdComponent**, no arquivo **nov-prod.component.ts**.

```typescript
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { Produto } from '../produto';
import { ProdutoService } from '../produto.service';

@Component({
  selector: 'app-nov-prod',
  templateUrl: './nov-prod.component.html',
  styleUrls: ['./nov-prod.component.css']
})
export class NovProdComponent implements OnInit {
  router: Router;
  servProd: ProdutoService;
  produto: Produto = new Produto();

  constructor(router: Router, servProd: ProdutoService) {
     this.router = router;
     this.servProd = servProd;
  }

  incluir():void{
     this.servProd.adicionar(this.produto);
     this.router.navigateByUrl("/produto")
  }
  ngOnInit(): void {
  }
}
```

19. Para dar suporte à funcionalidade, temos de definir um formulário no **modelo**, com utilização da diretiva **ngModel**, habilitando a **vinculação bidirecional**. Vamos utilizar o código da listagem seguinte em **nov-prod.component.html**.

```python
Novo Produto

Código:

Nome:

Quantidade:

Incluir
```

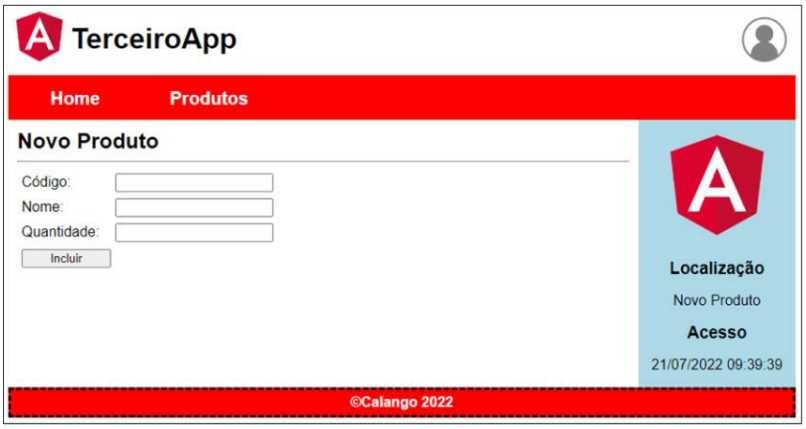
20. A folha de estilos, no arquivo **nov-prod.component.css**, deve ter seu conteúdo alterado para:

```python
*{font-family: Arial, Helvetica, sans-serif;}
#conteudo{
   display:flex; flex-wrap: wrap; width:80%; float:left;
   justify-content: center; align-items: center;
}
label, button{
    margin:5px; width: 100px; display: inline-block;
}
```

21. Ao executar o sistema completo, teremos a seguinte tela:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Formulário de inclusão.

22. Vamos implementar a visualização de produtos, utilizando o conteúdo seguinte para **ProdutoComponent**, no arquivo **produto.component.ts**.

```
typescript

import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { Produto } from '../produto';
import { ProdutoService } from '../produto.service';

@Component({
  selector: 'app-produto',
  templateUrl: './produto.component.html',
  styleUrls: ['./produto.component.css']
})
export class ProdutoComponent implements OnInit {
  router: Router;
  servProd: ProdutoService;
  produtos: Array = [];

  constructor(router: Router, servProd: ProdutoService) {
      this.router = router;
      this.servProd = servProd;
  }
  incluir():void{ this.router.navigateByUrl("/novoprod"); }
  excluir(codigo: string):void{
    this.servProd.remover(codigo);
    this.router.navigateByUrl("/produto");
  }
  ngOnInit(): void { this.produtos = this.servProd.obterTodos(); }
}
```

23. Vamos codificar o modelo, no arquivo **produto.component.html**, de acordo com o código da listagem seguinte.

```python
Listagem de Produtos
                                                                    Novo Produto
---
{{produto.codigo}}

{{produto.nome}}

Quantidade: {{produto.quantidade}}
---
Excluir
```

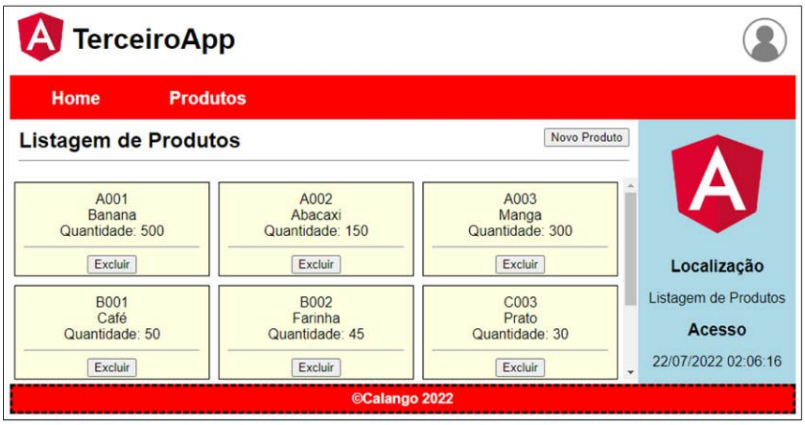
24. Vamos ajustar nossa folha de estilos que deve controlar os demais aspectos funcionais da página, como a exibição dos produtos na forma de um grid de painéis. O conteúdo de **produto.component.css** deve ser alterado para:

```python
*{font-family: Arial, Helvetica, sans-serif;}
#conteudo{
   display:flex; flex-wrap: wrap; width:80%; float:left;
   justify-content: center; align-items: center;
}
.container{
   display: flex; flex-direction: row; flex-wrap: wrap;
   width: 100%; overflow-y: scroll; height:230px;
}
.pnlProduto{
   width:200px; height:85px; background-color: lightyellow;
   border: black solid 1px; text-align: center;
   margin: 5px; padding: 10px;
}
```

25. Vamos incluir um conjunto de produtos como apresentado a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Listagem com produtos cadastrados através do serviço.

26. Precisamos melhorar **HomeComponent**, o que será realizado com a alteração de **home.component.html**.

```python
Bem-vindo!

Este é nosso sistema de exemplo.
Empresas ©Calango
```

27. A formatação, no arquivo **home.componente.css**, deve ser alterada para o conteúdo seguinte.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#conteudo{
   display:flex; flex-wrap: wrap; width:80%;
   float:left; justify-content: center; align-items: center;
}
```

28. A versão final do sistema está pronta, veja!

Conteúdo interativo

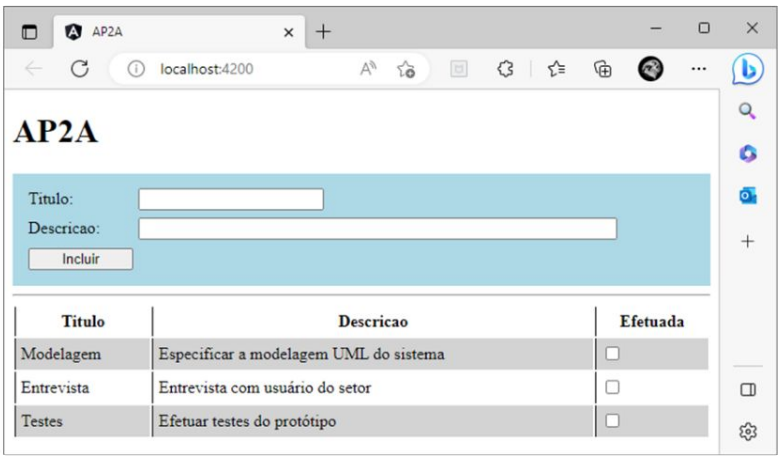
Acesse a versão digital para ver mais detalhes da imagem abaixo.

Página principal de TerceiroApp.

# Atividade 3

Crie um aplicativo que permita o gerenciamento de tarefas, com persistência em memória, onde cada tarefa deverá ter um título, uma descrição e um marcador indicando se já foi efetuada.

Ao final, você deverá fornecer uma saída como a que é apresentada a seguir.




## Chave de resposta

Comandos de linha para criação e configuração do projeto.

```
python

ng new AP2A
cd AP2A
ng g class Tarefa
ng g s Tarefa
```

Arquivo tarefa.ts.

```typescript
export class Tarefa {
    titulo: string = "";
    descricao: string = "";
    completa: boolean = false;
}
```

Arquivo tarefa.service.ts.

```typescript
import { Injectable } from '@angular/core';
import { Tarefa } from './tarefa';

@Injectable({
  providedIn: 'root'
})
export class TarefaService {
  private tarefas = new Array();
  constructor() { }
  inserir = (titulo: string, descricao: string) => {
    let t = new Tarefa();
    t.titulo = titulo;
    t.descricao = descricao;
    this.tarefas.push(t);
  }
  alternar = (posicao: number) => {
    this.tarefas[posicao].completa = !this.tarefas[posicao].completa;
  }
  obter = (): Array => this.tarefas;
}
```

Arquivo app.module.ts.

```typescript
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { TarefaService } from './tarefa.service';
import { FormsModule } from '@angular/forms';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    FormsModule
  ],
  providers: [
    TarefaService
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Arquivo app.component.ts.

```typescript
import { Component } from '@angular/core';
import { Tarefa } from './tarefa';
import { TarefaService } from './tarefa.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'AP2A';
  tarefa = new Tarefa();
  tarefas = new Array();
  constructor(private gestor: TarefaService) { }
  incluir = () => {
    this.gestor.inserir(this.tarefa.titulo, this.tarefa.descricao);
    this.tarefa = new Tarefa();
    this.tarefas = this.gestor.obter();
  }
  alternar = (posicao: number) => {
    console.log(posicao);
    this.gestor.alternar(posicao);
```

Arquivo app.component.html.

```python
# {{title}}

    Titulo:

    Descricao:

    Incluir

---

| Titulo | Descricao | Efetuada |
|---|---|---|
| {{t.titulo}} | {{t.descricao}} | |
```

Arquivo app.component.css.

```python
    width: auto; background: lightblue; padding: 10px;
}
label, button{
    margin:5px; width: 100px; display: inline-block;
}
table {
    width: 100%;
}
td,th {
    border-left: 1px solid black; padding: 5px;
}
th {
    font-weight: bold;
}
tr:nth-child(even) {
    background: lightgray
}
```

# Padrões de desenvolvimento

Trabalhar com elementos **assíncronos** pode ser algo complexo, mas necessário para garantir que um componente não paralise todo o sistema em operações demoradas. Com base no padrão **Observer**, temos uma alternativa simples para o controle assíncrono. A biblioteca **RxJS** irá oferecer para o Angular um modelo para tratamento assíncrono, baseado no paradigma de **programação reativa**, que segue os princípios do padrão Observer.

Neste vídeo, você terá uma visão geral dos padrões de desenvolvimento, além de soluções reutilizáveis, padrões de desenvolvimento mais comuns e padrões Observer.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Soluções reutilizáveis

Os padrões de desenvolvimento surgiram no intuito de organizar o modelo de programação, com base em soluções reutilizáveis, contendo nome, descrição da finalidade, modelagem UML e modo de utilização, o que permitiu recuperar as vantagens da abordagem orientada a objetos, trabalhando com soluções eficazes para problemas conhecidos.

## Padrões de desenvolvimento mais comuns

Embora não tenhamos a utilização de uma modelagem formal em nossos exemplos, estamos adotando alguns padrões de maneira conceitual, como no roteador, que relaciona as rotas com os componentes (Service Locator), ou na resposta aos eventos da interface pelos métodos do componente (Command). Outro aspecto interessante é a forma como os serviços são utilizados, através de instâncias globais, assemelhando-se ao padrão Singleton.

## Padrão Observer

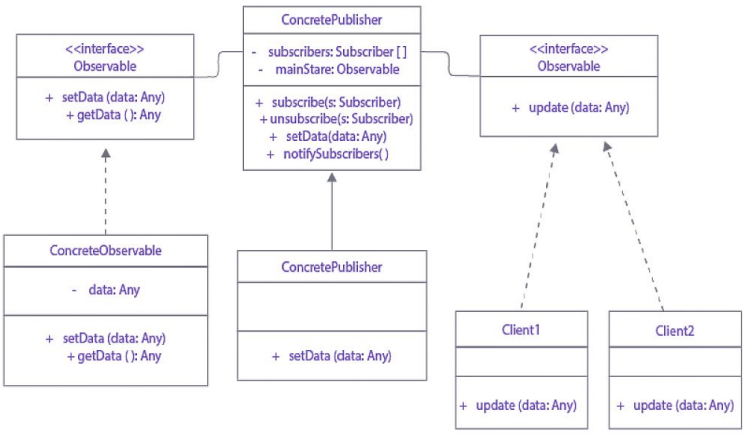
Em termos práticos, nosso objetivo é permitir a atualização dos componentes que refletem os dados de uma base comum, de forma assíncrona, com base em um sistema de assinaturas e notificações.

Segundo o padrão, precisamos de um gerenciador, denominado **Publisher**, que deve gerenciar uma fonte de dados com acesso controlado, classificada como **Observable**, além de manter um conjunto de assinantes, ou **Subscribers**. Cada vez que um assinante solicita ao gerenciador uma modificação na fonte de dados, os dados atualizados são enviados para todos os assinantes.

Confira um modelo UML que pode ser utilizado na implantação do padrão Observer.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Modelagem do padrão de desenvolvimento Observer.

Note que a classe abstrata **Publisher** prevê a utilização de uma fonte de dados que implemente a interface **Observable**, bem como um conjunto de assinantes (**subscribers**). Os assinantes, ou clientes, são classes que implementam a interface **Subscriber**, devendo efetuar o tratamento dos dados enviados, a cada notificação recebida, no método **update**.

O gerenciador concreto (**ConcretePublisher**) irá gerenciar a fonte de dados (**ConcreteObserver**), com a modificação dos dados ocorrendo a partir de **setData**. Sempre que um assinante invoca o método setData, ocorre a execução de **notifySubscribers**, após a conclusão das modificações, invocando o método **update** de cada assinante, com o fornecimento dos dados atualizados.

Por fim, um gerenciador deve permitir a adição de um assinante na coleção através do método **subscribe**, assim como sua remoção utilizando o método **unsubscribe**.

Utilizando o padrão Observer, temos uma alternativa para o processamento **assíncrono**, com a substituição por métodos de tratamento (**callback**), sem o uso de palavras reservadas e controle explícito de tarefas concorrentes (**threads**). Note que nossos clientes e gerenciadores podem estar executando em threads distintas, permitindo que as atualizações sejam efetuadas sem que o restante das funcionalidades do sistema seja bloqueado.

# Atividade 1

Questão 1

Em engenharia de software, um padrão de projeto é uma solução geral para um problema que ocorre com frequência dentro de determinado contexto no projeto de software. Qual padrão de projeto comportamental permite que um objeto notifique outros objetos sobre alterações em seu estado?

A Observer

B Strategy

C Iterator

D Interpreter

E Facade

A alternativa A está correta.

O Observer é um padrão de projeto de software que define uma dependência um-para-muitos entre objetos, de modo que, quando um objeto muda o estado, todos os seus dependentes são notificados e atualizados automaticamente.

# Biblioteca RxJS

A biblioteca **RxJS** (*Reactive Extensions Library for Java Script*) foi criada com o objetivo de gerar sistemas assíncronos e baseados em eventos, com base em sequências observáveis, utilizando o padrão **Observer**.

Este vídeo traz a apresentação dos componentes principais da biblioteca RxJS e a sua utilização no Angular. Confira!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Componentes principais

Temos um conjunto de componentes principais na arquitetura RxJS, que combinam os padrões **Observer** e **Iterator**, além da programação funcional para coleções, buscando um modelo ótimo para o tratamento de requisições assíncronas. Vamos entender melhor esses componentes!

Observable
Representa a fonte de informação, com uma coleção observável de dados e eventos futuros.

Observer
Representa os observadores, no caso um grupo de funções de resposta para receber os valores informados pelo Observable.

Subscription
É o meio para efetuar a assinatura na fonte de informação (Observable).

### Operators
São funções puras, como map, filter, concat e reduce, para o tratamento de coleções segundo o paradigma funcional.

### Subject
Define um canal de comunicação, equivalente a um componente do tipo EventEmitter, que gera eventos para os seletores do Angular.

### Schedulers
São despachantes centralizados, que especificam o agendamento das operações, coordenando a execução paralela.

Antes de analisarmos a utilização de RxJS, vamos observar o código da listagem seguinte, no qual temos um componente emissor de eventos.

```typescript
import { Component } from '@angular/core';
import { Tarefa } from './tarefa';
import { TarefaService } from './tarefa.service';
import { Component, EventEmitter, Output } from "@angular/core";

@Component({
  selector: 'zippy',
  template: `



Toggle




`})
export class Zippy {
  visible: boolean = true;
  @Output() open: EventEmitter = new EventEmitter();
  @Output() close: EventEmitter = new EventEmitter();
  toggle() {
    this.visible = !this.visible;
    if (this.visible) { this.open.emit(null); }
    else { this.close.emit(null); }
  }
}
```

Nesse código, temos um componente com o nome **Zippy**, no qual o modelo envolve uma div com o clique acionando o método **toggle**, e outra com a visibilidade controlada pela diretiva **hidden**, tendo como valor o inverso do atributo **visible**. Note que segunda div utiliza o seletor **ng-content**, que permite incluir qualquer conteúdo no interior do seletor.

De acordo com a funcionalidade estabelecida, o clique sobre a primeira div irá alternar (**toggle**) a visibilidade do conteúdo da segunda div. Analisando o método de resposta, podemos observar que o atributo **visible** é

alternado na primeira linha, através do operador de negação, seguido da emissão do evento correto (**open** ou **close**), de acordo com o valor de visible.

Por se tratar de um fluxo de execução que ocorre do componente para o modelo, o gerador de eventos é decorado com **Output**. Os atributos **open** e **close** são objetos do tipo **EventEmitter**, com utilização de parâmetro genérico (**any**).

Um exemplo de utilização pode ser observado no fragmento de código seguinte.

```
typescript


   Conteúdo com a visibilidade alternada através do clique em toggle.
```

Adicionando o componente Zippy ao módulo, e utilizando o seletor do fragmento em qualquer modelo HTML do projeto, teríamos a ocultação e reexibição da frase através do **clique** no texto **Toggle**, da primeira div do componente. Ocorrendo a mudança, será emitido o evento correto, close ou open, iniciando a execução do método de resposta configurada no componente, que nesse caso seria **onOpen** para o evento **open** e **onClose** para responder ao close.

O exemplo descrito aqui demonstra o processo de emissão e tratamento de eventos do Angular, que segue um modelo similar ao que é adotado pelo componente **Subject**, ou **assunto**, o qual determina um canal de comunicação compartilhado entre os assinantes. Antes de analisarmos a utilização de Subject, precisamos compreender o funcionamento de **Observable** e **Observers**.

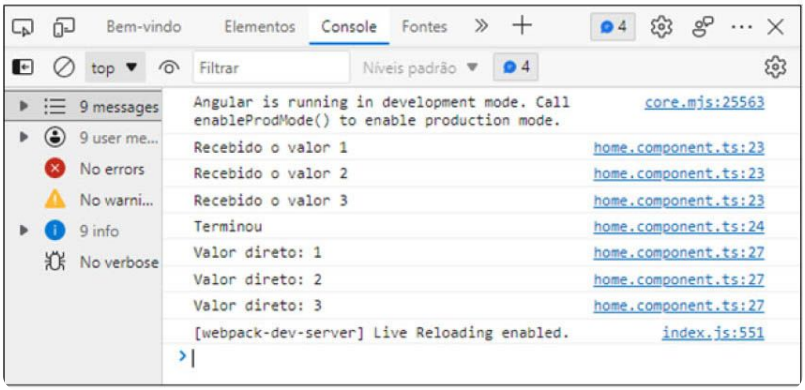
Podemos observar um exemplo básico de código com RxJS.

```
typescript

import { Observable } from 'rxjs';

const observable = new Observable((subscriber)=>{
   subscriber.next(1);
   subscriber.next(2);
   subscriber.next(3);
   subscriber.complete();
});
observable.subscribe({
   next(x){console.log(`Recebido o valor ${x}`);},
   complete(){console.log("Terminou");}
});
observable.subscribe((x)=>{ console.log(`Valor direto: ${x}`); });
```

Foi criada uma fonte de dados (**Observable**), com o fornecimento de três valores (**next**) para o assinante (**subscriber**) e fechamento do fluxo (**complete**). Em seguida, temos uma assinatura com o tratamento de next e complete separadamente, e outra apenas com o valor fornecido, sempre exibindo mensagens no console, como pode ser observado a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Saída no console para o exemplo de Observable.

Como descrito anteriormente, a biblioteca RxJS oferece diversos operadores (**Operators**) para lidar com coleções no modelo funcional. Na listagem seguinte, temos um exemplo de como podemos criar um **Observable** a partir de uma coleção, com base no operador **from**.

```
typescript

import { from } from 'rxjs';

const observable = from ([10,20,30]);

observable.subscribe((x)=>{ console.log(`Valor direto: ${x}`); });
```

Um observador (**Observer**) permite definir os métodos que são utilizados pelo assinante de forma reutilizável. Na prática, temos apenas a implementação dos métodos **next**, **complete** e **error**, os quais são responsáveis, respectivamente, pelo tratamento do valor recebido, detecção do encerramento do fluxo e gerenciamento de erros ocorridos.

No exemplo a seguir, temos a especificação de um **Observer**, que na verdade lida apenas com um objeto **JSON** (Java Script Object Notation), e não uma classe da biblioteca.

```
typescript

import { from } from 'rxjs';

const observable = from ([10,20,30]);
const observer = {
   next: (x:any) => console.log(`Recebido o valor ${x}`),
   error: (err:any) => console.log(`Erro: ${err}`),
   complete: () => console.log("Terminou")
}
observable.subscribe(observer);
```

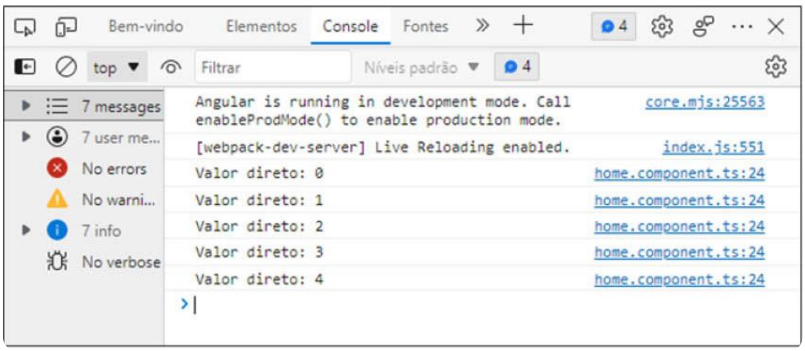
Com relação à assinatura (**Subscription**), recebemos o objeto a partir do método **subscribe**, e sua principal função é permitir que a assinatura em si seja cancelada, com a chamada para seu método **unsubscribe**. Vejamos um exemplo!

```typescript
import { interval } from 'rxjs';

const observable = interval(1000);
const subscription = observable.subscribe(
      (x)=>console.log(`Valor direto: ${x}`));
setTimeout(()=>{subscription.unsubscribe()},5000);
```

Aqui temos uma fonte de dados (**Observable**) que gera números sequenciais, em **intervalos** de um segundo, e a assinatura (**Subscription**) é cancelada após cinco segundos, através da chamada para **unsubscribe**, dentro de um agendamento com **setTimeout**. Apenas os números dos cinco primeiros ciclos serão exibidos:

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Utilização de unsubscribe para cancelar a assinatura.

Finalmente, podemos analisar o componente **Subject**.

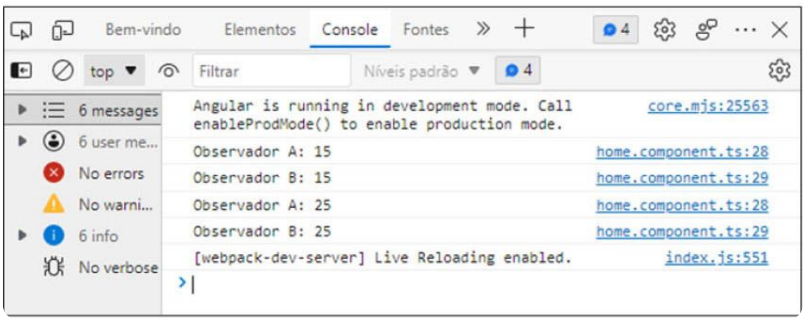
```typescript
import { Subject } from 'rxjs';

const subject = new Subject();

subject.subscribe((x)=>console.log(`Observador A: ${x}`));
subject.subscribe((x)=>console.log(`Observador B: ${x}`));
subject.next(15);
subject.next(25);
```

Todo componente **Subject** é também um **Observable**, possibilitando utilizar os métodos **next**, n**error** e **complete**, mas ele define um **canal central**, normalmente relacionado a algum **assunto**, que transmite as mensagens para todos os assinantes (**Observers**) simultaneamente. Note que as chamadas ao método next ocorrem fora de um bloco de definição, permitindo que o canal seja programado de forma externa com maior facilidade.

Vejamos o resultado da execução do código

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Exemplo de canal (Subject) com dois assinantes.

O último componente da biblioteca é o agendador (n**Scheduler**), que permite especificar quando teremos o fornecimento de dados pelo Observable. Um exemplo simples é o uso de agendador **assíncrono**, que inicia a execução após o término da tarefa corrente. Vamos observar um exemplo.

```
typescript

import { interval, map, observeOn, asyncScheduler } from 'rxjs';

const observable = interval(1000)
                   .pipe(map((x:number)=>x*x))
                   .pipe(observeOn(asyncScheduler));

const subscription = observable.subscribe(
      (x)=>console.log(`Valor direto: ${x}`));
setTimeout(()=>{subscription.unsubscribe()},10000);
```

Aqui temos uma fonte observável que emite números sequencialmente, em intervalos de um segundo, mas devido ao uso de **pipe**, os valores são transferidos para o mapeamento, elevando ao quadrado cada número gerado. Note como o uso do operador **map**, com base no paradigma funcional, facilita a implementação de transformações de dados.

Na sequência, outra chamada pipe envia os números elevados ao quadrado para o observador intermediário (**observeOn**) voltado para a temporização assíncrona (**asyncScheduler**). Feito isso, os números serão fornecidos de forma assíncrona, e como cancelamos a assinatura após dez segundos, não chegará a imprimir 81 (nove ao quadrado), como ocorreria na utilização da forma de agendamento padrão síncrona.

## Utilização de RxJS no Angular

Segundo a definição formal, **programação reativa** é um paradigma baseado em comportamento assíncrono, voltado para o fluxo de dados e a propagação de mudanças. Nesse contexto, **RxJS** é uma biblioteca que permite ao **Angular** utilizar programação reativa através de observáveis.

Tomando como base uma entidade **Animal**, bastante simples, com os atributos **nome** e **familia**, de acordo com a listagem seguinte, podemos definir um repositório assíncrono para ela.

```typescript
export class Animal {
    nome: string = ""; familia: string = "";
}
```

O serviço de gerenciamento do repositório, assumindo um comportamento **assíncrono** através da biblioteca RxJS, pode ser observado a seguir.

```typescript
@Injectable({ providedIn: 'root' })
export class AnimalService {
  private animais: Array = [];
  private canalAnimais = new Subject>();
  constructor() { }
  assinarAnimais(observer: any): void{
    this.canalAnimais.subscribe(observer);
  }
  adicionar(animal: Animal): void{
    this.animais.push(animal);
    this.canalAnimais.next(this.animais);
  }
}
```

A forma mais elegante de trabalhar com RxJS é através de **canais** (Subject), e o serviço exposto utiliza esse recurso na propagação das alterações do repositório para os componentes. Para que um componente receba as atualizações, ele deve fornecer um **observador** para **assinarAnimais**, e quando um animal é incluído no repositório, através do método **adicionar**, o conjunto atual é propagado no canal pelo método **next**.

Na listagem seguinte, temos um exemplo de componente que utilizaria o serviço.

```typescript
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  animais: Array = [];
  nome: string = "";
  familia: string = "";
  constructor(private animalServ: AnimalService){
    this.animalServ.assinarAnimais((animais: Array)=>{
       this.animais = animais;
    })
  }
  addAnimal(): void{
    this.animalServ.adicionar({nome:this.nome,familia:this.familia});
    this.familia = "";
    this.nome = "";
  }
}
```

Foi definido um atributo **animais** como uma **coleção**, e injetado **AnimalService** no construtor, além de ser implementada a **assinatura** do canal a partir do serviço, repassando para a coleção interna qualquer coleção

que seja oferecida pelo canal. Com a alteração do atributo interno, os elementos da página relacionados ao atributo serão atualizados.

Com relação ao método **addAnimal**, temos a chamada para o método **adicionar** do serviço, com a passagem dos dados no formato **JSON**. Em seguida, os dados de animal são limpos, o que irá refletir no formulário associado ao nível do modelo HTML.

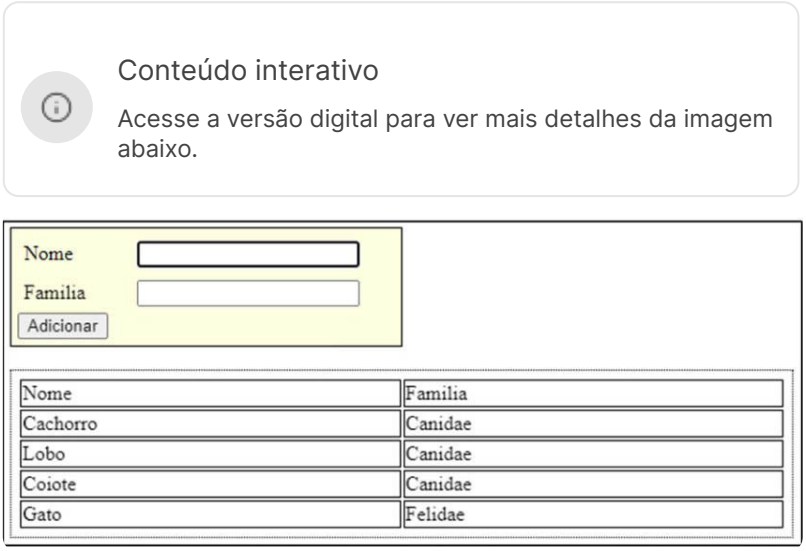
Um exemplo de modelo para o componente pode ser observado a seguir.

```
python



    Nome

    Familia

    Adicionar
```

| Nome | Familia |
| --- | --- |
| {{a.nome}} | {{a.familia}} |

A ligação **bidirecional** dos campos do formulário com o atributo **animal** é efetuada através da diretiva **ngModel**, enquanto o evento **submit** está associado ao método **adicionar**. Na tabela, é utilizada a diretiva **ngFor**, de forma a criar uma linha para cada animal da coleção.

Com uma folha de estilos adequada, teremos a tela a seguir.




Componente utilizando um canal RxJS.

Como a comunicação ocorre a partir do canal, poderíamos ter outros componentes associados, com as alterações sendo refletidas em tempo real. O processo seria o mesmo do anterior, com a injeção do serviço e obtenção da coleção a partir do observável, permitindo a composição da tela com base nos seletores dos componentes.

Existem diversas outras aplicações para os observadores RxJS no Angular, como no tratamento de eventos com baixo acoplamento, vejamos!

```typescript
import { fromEvent } from 'rxjs';

const el = document.getElementById('my-element')!;

const mouseMoves = fromEvent(el, 'mousemove');

const subscription = mouseMoves.subscribe(evt => {
  console.log(`Coordenadas: ${evt.clientX} X ${evt.clientY}`);
  if (evt.clientX < 40 && evt.clientY < 40) {
    subscription.unsubscribe();
  }
});
```

Criando um observável através de **fromEvent**, serão geradas informações a cada ocorrência do **evento** para o **elemento**, obtido da página através do método **getElementById**, onde o uso de **exclamação** no final indica ao compilador que não será retornado um valor nulo. No exemplo, estamos acompanhando o evento **mousemove**, do tipo **MouseEvent**, que permitirá emitir os dados de posicionamento a cada publicação.

Assinamos o observável, e utilizamos como observador apenas o tratamento simples, no modelo funcional, onde o atributo **evt**, referente ao evento, traz as coordenadas **clientX** e **clientY**, que serão impressas no console. Também temos o cancelamento da assinatura, com a chamada para o método **unsubscribe**, ao atingir a área superior esquerda do elemento.

Vamos considerar uma classe **Pessoa**, definida em termos de **nome**, cpf e **idade**.

```typescript
export class Pessoa {
   nome:string;
   cpf:string;
   idade:number;
   constructor(nome:string="", cpf:string="",idade:number=0){
      this.nome = nome;
      this.cpf = cpf;
      this.idade = idade;
   }
}
```

Outra utilização interessante da biblioteca é na manipulação de coleções via operadores.

```typescript
import { filter, map, of } from 'rxjs';
const  observable = of(new Pessoa("Ana","11111111111",25),
   new Pessoa("Luiz","22222222222",45),
   new Pessoa("Carlos","33333333333",32))
   .pipe( filter(p => p.idade > 26), map(p => p.nome) );
observable.subscribe((x)=>console.log(x));
```

No exemplo, temos o uso de **of**, para gerar o observável a partir de um conjunto de pessoas, com a repassagem via **pipe** para um filtro (**filter**), restringindo as pessoas com idade maior que 26, e o mapeamento (**map**), que efetua o retorno apenas do nome. A partir da fonte que foi definida, temos a assinatura (**subscribe**), levando à impressão dos nomes "Luiz" e "Carlos".

# Atividade 2

Questão 1

Através do paradigma da programação reativa, temos um modelo simples para o tratamento de elementos assíncronos, com base no padrão Observer, e a biblioteca RxJS é responsável pela viabilização desse modelo de programação para o Angular. Através da biblioteca, temos acesso a um conjunto de componentes, com papéis bem definidos na implementação de processos que devem ser tratados de forma assíncrona. Nesse contexto, qual componente representa a fonte de informações para o processamento assíncrono, sem possibilidade de assinantes múltiplos?

A Observer

B Subject

C Subscription

D Observable

E Operator

A alternativa D está correta.

Entre os componentes oferecidos na biblioteca RxJS, temos o Observable, como fonte da informação que será fornecida de forma assíncrona, com tratamento pontual. Se forem necessários múltiplos assinantes, devemos utilizar Subject. Em ambos os casos, deve ocorrer a assinatura da fonte (Subscription) pelo observador (Observer), ficando este último responsável pelo tratamento da informação recebida. Quanto aos operadores (Operator), são os elementos fornecidos pela biblioteca para efetuar operações comuns através do paradigma funcional.

# Acesso a recursos remotos

Quando lidamos com acesso remoto, é comum adotarmos estratégias assíncronas, pois a espera por uma resposta do servidor poderá parar todo o sistema, além do que não obter as vantagens do paralelismo em chamadas múltiplas. Nesse contexto, utilizar um observável RxJS pode ser uma estratégia muito interessante para simplificar o processo de comunicação assíncrona.

Neste vídeo, você acompanhará a criação de um servidor simulado e a de um serviço, e assim compreenderá como funciona um acesso a recursos remotos.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

## Criação de um servidor simulado

Devemos sempre lembrar que o Angular é uma tecnologia de **front-end**, sem recursos voltados para a persistência de dados, necessitando de acesso a um servidor. Mas o framework oferece uma ferramenta para gerar os serviços HTTP em memória, com interceptação das chamadas e fornecimento de respostas, atuando como um **mock server** (servidor simulado).

Como o servidor simulado trabalha apenas com a chave primária **id**, do tipo numérico, vamos precisar de uma entidade que siga esse padrão.

```typescript
export class Produto {
   id: number = 0;
   codigo: string = "";
   nome: string = "";
   quantidade: number = 0;
}
```

Para utilizar o mock server, é necessário instalar a biblioteca correta.

```typescript
npm install angular-in-memory-web-api --save
```

Com a biblioteca instalada, pode ser definido um serviço que implementará o servidor simulado.

```typescript
import { Injectable } from '@angular/core';
import { InMemoryDbService } from 'angular-in-memory-web-api';
import { Produto } from './produto';

@Injectable({
  providedIn: 'root'
})
export class ProdutoServidorService implements InMemoryDbService{
  createDb(){
    const produtos = [
      {id: 1, codigo: "A001", nome:"Banana", quantidade: 1000},
      {id: 2, codigo: "A002", nome:"Manga",  quantidade: 500 },
    ];
    return {produtos};
  }
  genId(produtos: Produto[]): number {
    return produtos.length > 0 ?
           Math.max(...produtos.map(p => p.id)) + 1 : 1;
  }
}
```

Ao implementar **InMemoryDbService**, tudo que precisamos é o fornecimento da base de dados inicial, através do método **createDb**, onde é retornada uma coleção no formato **JSON**. Embora não seja obrigatório, a geração de identificadores pode ser definida no método **genId**.

No método **genId**, a coleção é recebida como parâmetro, retornando o valor do identificador. Caso o tamanho da coleção (**length**) seja maior que zero, recuperamos o valor máximo (max) para o campo id, individualizado pelo método **map**, e retornamos o valor obtido acrescido de um, enquanto para uma coleção vazia apenas retornamos um.

Para que todos os artefatos necessários sejam gerados pela biblioteca, a partir do serviço que foi implementado, é necessáriz a divisão **imports** de **app.module.ts**, de acordo com o fragmento de código da listagem seguinte.

```typescript
imports: [
    BrowserModule,
    FormsModule,
    AppRoutingModule,
    HttpClientModule,
    HttpClientInMemoryWebApiModule.forRoot(
        ProdutoServidorService,
        { dataEncapsulation: false }
    )
  ],
```

Com a modificação, utilizamos **HttpClientInMemoryWebApiModule**, tendo como parâmetro o serviço criado anteriormente (**ProdutoServidorService**), definindo o modelo para geração dos artefatos de simulação e interceptação das chamadas HTTP. Também incluímos o módulo para criar as chamadas HTTP no restante do sistema, denominado **HttpClientModule**.

## Criação de um serviço

Algo interessante sobre o novo serviço é que não ocorre a adição na divisão **providers**, ou seja, o serviço não será disponibilizado para injeção de dependência no aplicativo, ocorrendo sempre acesso de forma **indireta**, por meio da interceptação das chamadas HTTP.

Com o servidor simulado pronto, pode ser criado um serviço para acessá-lo via chamadas HTTP.

```typescript
@Injectable({
  providedIn: 'root'
})
export class ProdutoService {
  private produtosUrl = "api/produtos";
  private http: HttpClient;
  httpOptions = {
    headers: new HttpHeaders({'Content-Type': 'application/json'})
  };
  constructor(http: HttpClient) { this.http = http; }

  private handleError(operation:string = 'operation', result?: T){
    return (error: any): Observable => {
      console.error(operation+" :: "+error);
      return of(result as T);
    };
  }
  adicionar(produto: Produto): Observable{
    return this.http.post(this.produtosUrl, produto,
                                  this.httpOptions).pipe(
        tap((novo: Produto) =>
             console.log(`Adicionado id=${novo.id}`)),
        catchError(this.handleError('adicionar'))
      );
  }
  obterTodos():Observable {
    return this.http.get(this.produtosUrl).pipe(
        tap(_ => console.log('Produtos recuperados')),
        catchError(this.handleError('obterTodos', []))
      );
  }
}
```

Temos os elementos para acesso ao servidor nos atributos de **ProdutoService**, onde **produtoUrl** representa o endereço inicial, http é o cliente que efetua as chamadas, e **httpOptions** inclui os cabeçalhos necessários para a comunicação, sendo aqui determinado o conteúdo **JSON**. Note que o cliente HTTP é obtido através da injeção de dependência no construtor.

Em seguida, foi definido um método genérico para tratamento de erros, onde **T** é substituído pelo **tipo esperado** no retorno. O método **handleError** emite uma mensagem no console, para permitir a análise do erro de comunicação ocorrido, e depois utiliza o operador n**of** para retornar um **Observable** para o chamador, impedindo seu bloqueio, mesmo que o valor seja nulo.

No método **adicionar**, recebemos um produto como parâmetro, visando à inclusão na base de dados do servidor, e temos o retorno de um **Observable** para **Produto**. Através de uma chamada para o **endereço de base**, via método **post**, com a passagem do valor fornecido, ocorre a inclusão do produto na coleção, sendo retornado o mesmo produto fornecido, mas agora com o atributo id preenchido. Através do método **pipe**, o retorno do servidor é enviado para o operador **tap**, que emite uma mensagem no console, sem interromper o fluxo de dados, e para **catchError**, que direciona o tratamento do erro para o método **handleError**.

Finalmente, no método **obterTodos**, temos o retorno de um **Observable** para a **coleção**, sendo utilizado o **endereço de base**, via método **get** do HTTP. Note que o operador **catchError**, nesse caso, inclui o retorno de uma coleção vazia, para evitar erros no retorno para o componente.

Um componente de exemplo pode ser observado na listagem seguinte.

```typescript
@Component({
  selector: 'app-produto',
  templateUrl: './produto.component.html',
  styleUrls: ['./produto.component.css']
})
export class ProdutoComponent implements OnInit {
  router: Router;
  servProd: ProdutoService;
  produtos: Array = [];
  produto: Produto = new Produto();
  enviando: boolean = false;

  constructor(router: Router, servProd: ProdutoService) {
    this.router = router;
    this.servProd = servProd;
  }

  incluir():void{
    this.enviando = true;
    this.servProd.adicionar(this.produto)
        .subscribe(_=> {
          this.enviando = false;
          this.obterTodos();
        });
  }
  private obterTodos(){
    this.servProd.obterTodos()
        .subscribe((produtos)=>this.produtos=produtos);
  }
  ngOnInit(): void { this.obterTodos(); }
}
```

Em termos da obtenção, temos um método privado, com o nome **obterTodos**, onde assinamos o método de mesmo nome no serviço, via **subscribe**, utilizando o retorno para o preenchimento da coleção interna do componente. Ao inicializarmos o componente, através do método **ngOnInit**, temos a chamada para o método interno **obterTodos**, atualizando a coleção interna.

Com relação à inclusão, definimos um **booleano** com o nome **enviando**, que será utilizado para indicar uma inclusão em andamento, iniciado com valor **false**, já que a comunicação ocorrerá apenas na implementação do método **incluir**. Ao nível do método **incluir**, temos a alteração de **enviando** para **true**, seguida de uma **assinatura** no método **adicionar** do serviço, passando o **produto** como parâmetro. Na recepção do **Observable**, alteramos o valor de **enviando** para false, já que a comunicação remota é concluída.

A atualização da coleção interna é garantida com uma nova chamada para **obterTodos**, logo após o retorno de **enviando** para o valor **false**.

# Atividade 3

Questão 1

Quando trabalhamos em sistemas Web, a responsabilidade sobre a persistência dos dados é da tecnologia servidora utilizada, com o fornecimento das informações necessárias através do protocolo HTTP e seus diferentes métodos. Por meio da inclusão de HttpClientModule, podemos efetuar a comunicação com o servidor de forma simples, a partir do front-end criado em Angular, onde o objeto injetado, do tipo HttpClient, oferece os métodos necessários atender a todos os modos de utilização do HTTP. Qual seria o método para solicitar a inclusão de um registro?

A post

B put

C delete

D get

E head

A alternativa A está correta.

Segundo o padrão mais comum utilizado, definido pelos princípios do REST, uma inclusão de dados deve ser efetuada pelo método post. Em geral, temos uma chamada para o endereço, via método post, e a passagem dos dados no corpo da requisição. Quanto aos demais, get é utilizado nas consultas, put para efetuar alterações, delete para a exclusão do registro e head para a obtenção de dados estruturais (metadados) acerca da fonte de informação.

# Projeto QuartoApp

Nesta prática, criaremos um projeto com o nome **QuartoApp**, no qual teremos um exemplo simples de tela baseada em componentes, com atualização efetuada de forma **automática**, devido ao uso de **canais assíncronos** da biblioteca **RxJS**.

Neste vídeo, você aprenderá, através de estudo de caso, a implementar o projeto QuartoApp.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Roteiro de Prática

1. Vamos criar um aplicativo com o nome **QuartoApp**, baseado em uma entidade **Pessoa**, executando os comandos.

```typescript
ng new QuartoApp
cd QuartoApp
ng g class Pessoa --skip-tests true
ng g s Pessoa --flat --skip-tests true
ng g c TabPessoa
ng g c ListPessoa
ng g c PanelPessoa
ng g c Exemplo1
```

Ao final, teremos um projeto com a classe Pessoa, serviço de gerenciamento PessoaService, e os componentes TabPessoa, ListPessoa e PanelPessoa. Também foi definido um componente com o nome Exemplo1, que irá conter um formulário para inclusão de pessoas.

2. Vamos criar a nossa classe **Pessoa** que será bastante simples, constituída apenas dos atributos **nome**, **cpf** e **idade**, sendo definida no arquivo **pessoa.ts**.

```typescript
export class Pessoa {
   nome:string;
   cpf:string;
   idade:number;
   constructor(nome:string="", cpf:string="",idade:number=0){
      this.nome = nome;
      this.cpf = cpf;
      this.idade = idade;
   }
}
```

Vamos implementar o serviço para gerenciamento de dados, que será baseado em canais do RxJS. O código de **PessoaService**, no arquivo **pessoa.service.ts**, deve ser alterado.

```typescript
import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';
import { Pessoa } from './pessoa';

@Injectable({
  providedIn: 'root'
})
export class PessoaService {
  private pessoas: Array = [];
  private canalPessoaAdd  = new Subject();
  private canalPessoaFull = new Subject>();

  constructor() { }

  assinarAdd(observer: any): void{
    this.canalPessoaAdd.subscribe(observer);
  }
  assinarFull(observer: any): void{
    this.canalPessoaFull.subscribe(observer);
  }
  adicionar(pessoa: Pessoa): void{
    this.pessoas.push(pessoa);
    this.canalPessoaAdd.next(pessoa);
    this.canalPessoaFull.next(this.pessoas);
  }
  atualizar(): void{
    this.canalPessoaFull.next(this.pessoas);
  }
}
```

3. Vamos disponibilizar o serviço no ambiente, sendo necessário acrescentá-lo aos provedores configurados em **app.module.ts**, de acordo com o fragmento de código seguinte. Já que iremos utilizar um formulário posteriormente, aproveitamos para importar **FormsModule**.

```typescript
Imports: [
    BrowserModule,
    FormsModule,
    AppRoutingModule
  ],
  providers: [PessoaService],
```

4. Vamos definir os componentes que visam apresentar os dados de pessoas nos formatos de lista, tabela, ou painel de visualização individual. Começaremos a programar por **ListPessoaComponent**, no arquivo **list-pessoa.component.ts**.

```typescript
import { Component, OnInit } from '@angular/core';
import { Pessoa } from '../pessoa';
import { PessoaService } from '../pessoa.service';

@Component({
  selector: 'app-list-pessoa',
  templateUrl: './list-pessoa.component.html',
  styleUrls: ['./list-pessoa.component.css']
})
export class ListPessoaComponent implements OnInit {
  pessoas: Array = [];
  servPessoa: PessoaService;
  constructor(servPessoa: PessoaService) {
    this.servPessoa = servPessoa;
  }
  ngOnInit(): void {
    this.servPessoa.assinarFull((pessoas: Array)=>{
      console.log("Array da lista atualizado");
      this.pessoas = pessoas;
    });
  }
}
```

5. Vamos alterar o modelo, no arquivo **list-pessoa.component.html.**

```python
```

6. Vamos alterar o conteúdo de **tab-pessoa.component.ts**, onde podemos observar que **TabPessoaComponent** é quase idêntico a **ListPessoaComponent**, ocorrendo diferenças apenas em termos da decoração da classe e da mensagem no console.

```typescript
import { Component, OnInit } from '@angular/core';
import { Pessoa } from '../pessoa';
import { PessoaService } from '../pessoa.service';

@Component({
  selector: 'app-tab-pessoa',
  templateUrl: './tab-pessoa.component.html',
  styleUrls: ['./tab-pessoa.component.css']
})
export class TabPessoaComponent implements OnInit {
  pessoas: Array = [];
  servPessoa: PessoaService;
  constructor(servPessoa: PessoaService) {
    this.servPessoa = servPessoa;
  }
  ngOnInit(): void {
    this.servPessoa.assinarFull((pessoas: Array)=>{
      console.log("Array da tabela atualizado");
      this.pessoas = pessoas;
    });
  }
}
```

7. Vamos modificar o arquivo **tab-pessoa.component.html**, referente ao **modelo**.

```python
| Nome | CPF | Idade |
|---|---|---|
| {{pessoa.nome}} | {{pessoa.cpf}} | {{pessoa.idade}} |
```

Vamos alterar o conteúdo de **tab-pessoa.component.css**, em que temos apenas o acréscimo de bordas na tabela.

```python
table td{ border: solid black 1px; }
```

8. Vamos alterar **PanelPessoaComponent**, no arquivo **panel-pessoa.component.ts**.

```typescript
import { Component, OnInit } from '@angular/core';
import { Pessoa } from '../pessoa';
import { PessoaService } from '../pessoa.service';

@Component({
  selector: 'app-panel-pessoa',
  templateUrl: './panel-pessoa.component.html',
  styleUrls: ['./panel-pessoa.component.css']
})
export class PanelPessoaComponent implements OnInit {
  pessoa: Pessoa = new Pessoa();
  servPessoa: PessoaService;

  constructor(servPessoa: PessoaService) {
    this.servPessoa = servPessoa;
   }
  ngOnInit(): void {
    this.servPessoa.assinarAdd((pessoa:Pessoa)=>this.pessoa=pessoa);
  }
}
```

9. Utilizaremos o conteúdo da listagem seguinte no arquivo **panel-pessoa.component.html**, para que o modelo reflita os dados obtidos através do canal (**Subject**), ao nível da classe.

```typescript



   Nome:{{pessoa.nome}}
   Idade:{{pessoa.idade}}
   Cpf:{{pessoa.cpf}}
```

10. Vamos definir uma folha de estilos adequada, no arquivo **panel-pessoa.component.css**.

```typescript
.pessoa{ display: flex; flex-direction: row; flex-wrap: wrap; }
.pessoa span{
  width: 150px; display: inline-block;
  border: dotted black 1px; background-color: lightyellow;
}
```

11. Nosso formulário será definido em Exemplo1Component, começando a implementação por sua classe, no arquivo exemplo1.component.ts.

```
typescript

import { Component, OnInit } from '@angular/core';
import { Pessoa } from '../pessoa';
import { PessoaService } from '../pessoa.service';

@Component({
  selector: 'app-exemplo1',
  templateUrl: './exemplo1.component.html',
  styleUrls: ['./exemplo1.component.css']
})
export class Exemplo1Component implements OnInit {
  novaPessoa: Pessoa = new Pessoa();
  servPessoa: PessoaService;
  constructor(servPessoa: PessoaService){
    this.servPessoa = servPessoa;
  }

  adicionar():void{
    const aux = new Pessoa(this.novaPessoa.nome,
      this.novaPessoa.cpf,this.novaPessoa.idade);
    this.servPessoa.adicionar(aux);
  }
  ngOnInit(): void {
  }
}
```

12. Vamos alterar, já que temos os elementos necessários para o modelo, o arquivo **exemplo1.component.html**.

```
python



      CPF:


      Nome:


      Idade:


      Incluir
```

A ligação **bidirecional** dos campos do formulário com o atributo n**novaPessoa** é efetuado através da diretiva **ngModel**, enquanto o evento **submit** está associado ao método **adicionar**.

13. Vamos alterar a folha de estilos, no arquivo **exemplo1.component.css**.

```
python

label, button{
  margin:5px; width: 100px; display: inline-block;
}
```

O componente principal será alterado apenas ao nível do **modelo**, com a utilização do conteúdo da listagem seguinte no arquivo **app.component.html**.

```python
{{title}}

---



Último adicionado:
```

14. Vamos executá-lo e inserir algumas pessoas, verificando na prática os efeitos do paradigma reativo.

Execução do sistema de exemplo QuartoApp.

# Atividade 4

Crie uma segunda versão para o aplicativo de gerenciamento de tarefas com persistência em memória, mantendo a funcionalidade original, de modo que cada tarefa tem um título, uma descrição e um marcador indicando se já foi efetuada.

A nova versão deverá utilizar três componentes:

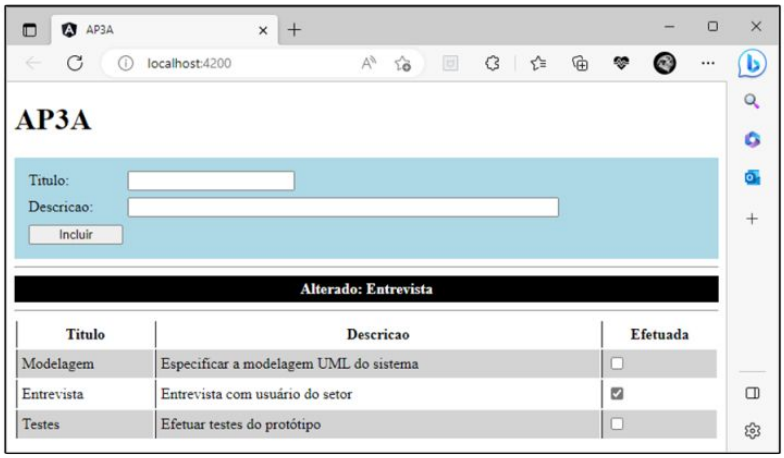
- Formulário para inclusão de tarefa.

- Tabela com listagem de tarefas.

- Painel com última operação efetuada.

Para promover a atualização dos componentes, utilize canais de comunicação assíncronos, com base na tecnologia RxJS.

Ao final, você deverá fornecer uma saída, semelhante à apresentada a seguir.




## Chave de resposta

Comandos de linha para criação e configuração do projeto.

```python
ng new AP3A
cd AP3A
ng g class Tarefa
ng g s Tarefa
ng g c TarefaForm
ng g c TarefaTable
ng g c TarefaPanel
```

Arquivo tarefa.ts.

```typescript
export class Tarefa {
    titulo: string = "";
    descricao: string = "";
    completa: boolean = false;
}
```

Arquivo tarefa.service.ts.

```typescript
import { Injectable } from '@angular/core';
import { Tarefa } from './tarefa';
import { Subject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class TarefaService {
  private tarefas: Array = [];
  private canalTarefas = new Subject>();
  private canalMensagens = new Subject();

  constructor() { }

  assinarTarefas(observer: any): void{
    this.canalTarefas.subscribe(observer);
  }
  assinarMsgs(observer: any): void{
    this.canalMensagens.subscribe(observer);
  }
  adicionar(tarefa: Tarefa): void{
    this.tarefas.push(tarefa);
    this.canalMensagens.next("Adicionado: "+tarefa.titulo);
    this.canalTarefas.next(this.tarefas);
  }
  alternar = (posicao: number) => {
    this.tarefas[posicao].completa = !this.tarefas[posicao].completa;
    this.canalMensagens.next("Alterado: "+
                             this.tarefas[posicao].titulo);
    this.canalTarefas.next(this.tarefas);
  }
}
```

Arquivo app.module.ts.

```typescript
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { TarefaFormComponent }
       from './tarefa-form/tarefa-form.component';
import { TarefaTableComponent }
       from './tarefa-table/tarefa-table.component';
import { TarefaPanelComponent }
       from './tarefa-panel/tarefa-panel.component';

import { TarefaService } from './tarefa.service';
import { FormsModule } from '@angular/forms';

@NgModule({
  declarations: [
    AppComponent,
    TarefaFormComponent,
    TarefaTableComponent,
    TarefaPanelComponent
  ],
  imports: [
    BrowserModule,
    FormsModule
  ],
  providers: [
    TarefaService
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Arquivo tarefa-form.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';
import { Tarefa } from '../tarefa';
import { TarefaService } from '../tarefa.service';

@Component({
  selector: 'app-tarefa-form',
  templateUrl: './tarefa-form.component.html',
  styleUrls: ['./tarefa-form.component.css']
})
export class TarefaFormComponent implements OnInit {
  tarefa = new Tarefa();
  constructor(private gestor: TarefaService) { }
  ngOnInit(): void { }
  incluir = () => {
    this.gestor.adicionar(this.tarefa);
    this.tarefa = new Tarefa();
  }
}
```

Arquivo tarefa-form.component.html.

```python

    Titulo:


    Descricao:


    Incluir
```

Arquivo tarefa-form.component.css.

```python
form{ width: auto; background: lightblue; padding: 10px; }
label, button{ margin:5px; width: 100px; display: inline-block; }
```

Arquivo tarefa-panel.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';
import { TarefaService } from '../tarefa.service';

@Component({
  selector: 'app-tarefa-panel',
  templateUrl: './tarefa-panel.component.html',
  styleUrls: ['./tarefa-panel.component.css']
})
export class TarefaPanelComponent implements OnInit {
  mensagem: string = "Nenhum movimento efetuado...";
  constructor(private gestor: TarefaService) { }
  ngOnInit(): void {
    this.gestor.assinarMsgs((msg: string)=>{this.mensagem=msg});
  }
}
```

Arquivo tarefa-panel.component.html.

```python
{{mensagem}}
```

Arquivo tarefa-panel.component.css.

```python
#msg{
    background: black; color: white; width:auto;
    text-align: center; padding: 5px; font-weight: bold;
}
```

Arquivo tarefa-table.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';
import { Tarefa } from '../tarefa';
import { TarefaService } from '../tarefa.service';

@Component({
  selector: 'app-tarefa-table',
  templateUrl: './tarefa-table.component.html',
  styleUrls: ['./tarefa-table.component.css']
})
export class TarefaTableComponent implements OnInit {

  tarefas = new Array();

  constructor(private gestor: TarefaService) { }

  ngOnInit(): void {
    this.gestor.assinarTarefas((lst: Array)=>this.tarefas=lst);
  }

  alternar = (posicao: number) => {
    this.gestor.alternar(posicao);
  }

}
```

Arquivo tarefa-table.component.html.

```python
| Titulo | Descricao | Efetuada |
|---|---|---|
| {{t.titulo}} | {{t.descricao}} | |
```

Arquivo tarefa-panel.component.css.

```python
table { width: 100%; }
td,th { border-left: 1px solid black; padding: 5px; }
th { font-weight: bold; }
tr:nth-child(even) { background: lightgray }
```

Arquivo app.component.html.

```python
{{title}}

---
```

## Projeto TerceiroAppRx

Nesta prática, criaremos um projeto com o nome **TerceiroAppRx**, onde fazemos uma releitura do projeto **TerceiroApp**, tendo como objetivo utilizar de **RxJS** para conexão **HTTP**, amparado por um servidor simulado (**mock server**).

Neste vídeo, você aprenderá, através de estudo de caso, a implementar o projeto TerceiroAppRx.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

### Roteiro de Prática

1. Inicialmente, vamos copiar o projeto **TerceiroApp**, renomeando o diretório para **TerceiroAppRx**, além de adicionar a **biblioteca** e o **serviço** para definir o servidor simulado no aplicativo no diretório TerceiroAppRx.

```typescript
npm install angular-in-memory-web-api –save
ng g s ProdutoServidor --flat --skip-tests true
```

2. Como o servidor simulado trabalha apenas com a chave primária id, do tipo numérico, vamos alterar a classe **Produto**, no arquivo **produto.ts**, passando a utilizar um atributo **id**.

```typescript
export class Produto {
   id: any;
   codigo: string;
   nome: string;
   quantidade: number;
   constructor(id: any = null, codigo: string = "",
               nome: string = "", quantidade: number = 0) {
      this.id = id;
      this.codigo = codigo;
      this.nome = nome;
      this.quantidade = quantidade;
   }
}
```

3. Vamos codificar nosso servidor simulado, no arquivo **produto-servidor.service.ts**.

```typescript
import { Injectable } from '@angular/core';
import { InMemoryDbService } from 'angular-in-memory-web-api';
import { Produto } from './produto';

@Injectable({
  providedIn: 'root'
})
export class ProdutoServidorService implements InMemoryDbService{

  createDb(){
    const produtos = [
      {id: 1, codigo: "A001", nome:"Banana", quantidade: 1000},
      {id: 2, codigo: "A002", nome:"Manga",  quantidade: 500 },
      {id: 3, codigo: "B001", nome:"Caneta", quantidade: 100 },
      {id: 4, codigo: "B002", nome:"Papel",  quantidade: 1200},
    ];
    return {produtos};
  }

  genId(produtos: Produto[]): number {
    return produtos.length > 0 ?
           Math.max(...produtos.map(p => p.id)) + 1 : 1;
  }
}
```

Na prática, o servidor simulado vai interceptar as chamadas HTTP, respondendo aos diversos métodos do protocolo, segundo o padrão REST. Como fonte de dados, será utilizada a coleção fornecida em **createDb**, enquanto **genId** fornece o próximo valor da chave primária, invocado durante a resposta ao POST, com a inserção de uma entidade.

4. Vamos à configuração na divisão **imports** de **app.module.ts**.

```typescript
imports: [
    BrowserModule, FormsModule, AppRoutingModule,
    HttpClientModule,
    HttpClientInMemoryWebApiModule.forRoot(
        ProdutoServidorService,
        { dataEncapsulation: false }
    )
  ],
```

A complementação de código do Visual Studio Code efetua a importação de bibliotecas de forma automática, mas apenas a título de informação, elas são apresentadas no fragmento de código da listagem seguinte, devendo estar presentes nas importações de **app.module.ts**.

```typescript
import { HttpClientModule } from '@angular/common/http';
import { HttpClientInMemoryWebApiModule }
       from 'angular-in-memory-web-api';
import { ProdutoServidorService } from './produto-servidor.service';
```

5. Vamos implementar a classe **ProdutoService**, no arquivo **produto.service.ts**, no qual utilizamos chamadas HTTP.

```typescript
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, Observable, of, tap } from 'rxjs';
import { Produto } from './produto';

@Injectable({
  providedIn: 'root'
})
export class ProdutoService {
  private produtosUrl = "api/produtos";
  private http: HttpClient;
  httpOptions = {
    headers: new HttpHeaders({'Content-Type': 'application/json'})
  };
  constructor(http: HttpClient) { this.http = http; }
  private handleError(operation:string = 'operation', result?: T){
    return (error: any): Observable => {
      console.error(operation+" :: "+error);
      return of(result as T);
    };
  }
  adicionar(produto: Produto): Observable{
    return this.http.post(this.produtosUrl, produto,
                                  this.httpOptions).pipe(
        tap((novo: Produto) =>
             console.log(`Adicionado id=${novo.id}`)),
        catchError(this.handleError('adicionar'))
      );
  }
  remover(id: number): Observable{
    const url = `${this.produtosUrl}/${id}`;
    return this.http.delete(url, this.httpOptions).pipe(
        tap(_ => console.log(`Removido id=${id}`)),
        catchError(this.handleError('remover'))
      );
  }
  obterTodos():Observable {
    return this.http.get(this.produtosUrl).pipe(
        tap(_ => console.log('Produtos recuperados')),
        catchError(this.handleError('obterTodos', []))
      );
  }
}
```

6. Vamos alterar **ProdutoComponent**, no arquivo **produto.component.ts**, de acordo com a listagem seguinte, com a injeção de **ProdutoService** e **Router** no construtor.

```typescript
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { Produto } from '../produto';
import { ProdutoService } from '../produto.service';

@Component({
  selector: 'app-produto',
  templateUrl: './produto.component.html',
  styleUrls: ['./produto.component.css']
})
export class ProdutoComponent implements OnInit {
  router: Router;
  servProd: ProdutoService;
  produtos: Array = [];
  constructor(router: Router, servProd: ProdutoService) {
    this.router = router;
    this.servProd = servProd;
  }
  incluir():void{ this.router.navigateByUrl("/novoprod"); }
  private obterTodos(){
    this.servProd.obterTodos()
        .subscribe((produtos)=>this.produtos=produtos);
  }
  excluir(id: number):void{
    this.servProd.remover(id).subscribe(_=>this.obterTodos());
  }
  ngOnInit(): void { this.obterTodos(); }
}
```

7. Com relação ao modelo, no arquivo **produto.component.html**, modificaremos a chamada para o método **excluir**, que agora utiliza o **id** do produto.

```python
Listagem de Produtos
                                                                    Novo Produto
---
{{produto.codigo}}

{{produto.nome}}

Quantidade: {{produto.quantidade}}
---
Excluir
```

8. Vamos alterar a classe **NovProdComponent**, no arquivo **nov-prod.component.ts**.

```typescript
import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { Produto } from '../produto';
import { ProdutoService } from '../produto.service';

@Component({
  selector: 'app-nov-prod',
  templateUrl: './nov-prod.component.html',
  styleUrls: ['./nov-prod.component.css']
})
export class NovProdComponent implements OnInit {
  router: Router;
  servProd: ProdutoService;
  produto: Produto = new Produto();
  enviando: boolean = false;

  constructor(router: Router, servProd: ProdutoService) {
    this.router = router;
    this.servProd = servProd;
  }

  incluir():void{
    this.enviando = true;
    this.servProd.adicionar(this.produto)
        .subscribe(_=> {
          this.enviando = false;
          this.router.navigateByUrl("/produto")
        });
  }

  ngOnInit(): void { }
}
```

9. Vamos à alteração do modelo, no arquivo **nov-prod.component.html**, que ocorre apenas na habilitação ou desabilitação do botão de envio.

```python
```

## Novo Produto

```
Código:

Nome:

Quantidade:

Incluir
```

10. Já podemos executar o projeto **TerceiroAppRx**, verificando que são mantidas as funcionalidades da versão original, em **TerceiroApp**, mas agora com uso da biblioteca **RxJS** e comunicação **HTTP**, além do padrão de desenvolvimento **Observer**.

# Atividade 5

Crie uma terceira versão para o aplicativo de gerenciamento de tarefas, agora direcionando a persistência para o protocolo HTTP, além de utilizar duas telas, uma para listagem e outra para inclusão, com menu de navegação utilizando o roteamento do Angular.

A nova versão deverá utilizar dois componentes, representando a área principal de cada tela:

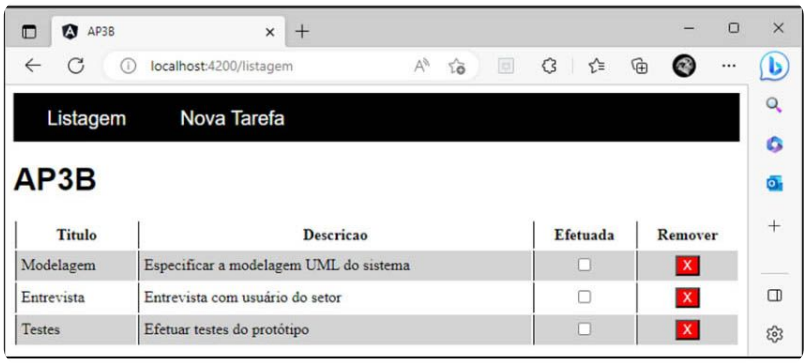
- Formulário para inclusão de tarefa.

- Tabela com listagem de tarefas.

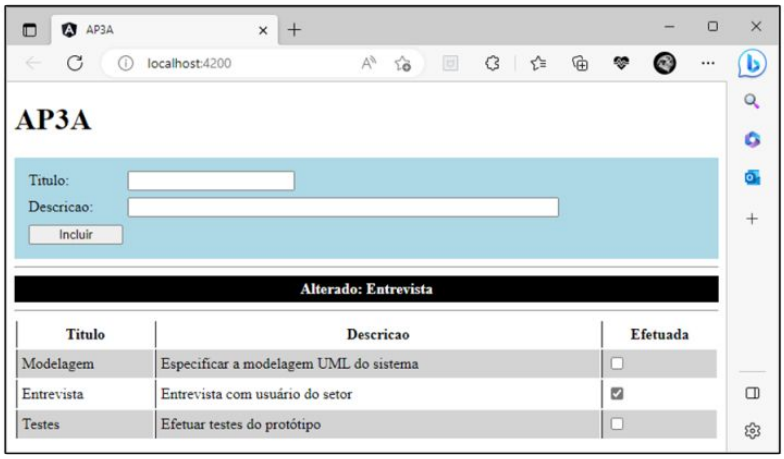
Também deverá ser utilizado um servidor HTTP simulado e acesso remoto via tecnologia RxJS.

Ao final, você deverá fornecer uma saída, semelhante à apresentada a seguir.

**Tela 1**




Tela 2




## Chave de resposta

Comandos de linha para criação e configuração do projeto.

```
python

ng new AP3B --routing
cd AP3B
ng g class Tarefa --skip-tests true
ng g s Tarefa --skip-tests true
ng g c TarefaForm
ng g c TarefaTable
npm install angular-in-memory-web-api@0.14.0 --save-dev
ng g s TarefaServidor --skip-tests true
```

Arquivo tarefa.ts.

```typescript
export class Tarefa {
   id: any;
   titulo: string;
   descricao: string;
   completa: boolean;
   constructor(id: any = null, titulo: string = "",
               descricao: string = "", completa: boolean = false) {
      this.id = id;
      this.titulo = titulo;
      this.descricao = descricao;
      this.completa = completa;
   }
}
```

Arquivo tarefa-servidor.service.ts.

```typescript
import { Injectable } from '@angular/core';
import { InMemoryDbService } from  'angular-in-memory-web-api';
import { Tarefa } from './tarefa';

@Injectable({
  providedIn: 'root'
})
export class TarefaServidorService implements InMemoryDbService{
  createDb(){
    const tarefas = [
      {id: 1, titulo: "Modelagem",
       descricao: "Especificar a modelagem UML do sistema",
       completa: false},
      {id: 2, titulo: "Entrevista",
       descricao: "Entrevista com usuário do setor",
       completa: false},
      {id: 3, titulo: "Testes",
       descricao: "Efetuar testes do protótipo",
       completa: false},
    ];
    return {tarefas};
  }
  genId(tarefas: Tarefa[]): number {
    return tarefas.length > 0 ?
           Math.max(...tarefas.map(t => t.id)) + 1 : 1;
  }
}
```

Arquivo tarefa.service.ts.

```typescript
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, Observable, of, tap } from 'rxjs';
import { Tarefa } from './tarefa';

@Injectable({
  providedIn: 'root'
})
export class TarefaService {
  private tarefasUrl = "api/tarefas";
  private http: HttpClient;
  httpOptions = {
    headers: new HttpHeaders({'Content-Type': 'application/json'})
  };
  constructor(http: HttpClient) { this.http = http; }
  private handleError(operation:string = 'operation', result?: T){
    return (error: any): Observable => {
      console.error(operation+" :: "+error);
      return of(result as T);
    };
  }
  adicionar(tarefa: Tarefa): Observable{
    return this.http.post(this.tarefasUrl, tarefa,
                                this.httpOptions).pipe(
        tap((novo: Tarefa) => console.log(`Adicionado id=${novo.id}`)),
        catchError(this.handleError('adicionar'))
      );
  }
  remover(id: number): Observable{
    const url = `${this.tarefasUrl}/${id}`;
    return this.http.delete(url, this.httpOptions).pipe(
        tap(_ => console.log(`Removido id=${id}`)),
        catchError(this.handleError('remover'))
      );
  }
  alterar(tarefa: Tarefa): Observable{
    return this.http.put(this.tarefasUrl, tarefa,
                               this.httpOptions).pipe(
        tap(_ => console.log(`Alterado id=${tarefa.id}`)),
        catchError(this.handleError('alterar'))
    );
  }
  obter(id: number): Observable{
    const url = `${this.tarefasUrl}/${id}`;
    return this.http.get(url).pipe(
        tap(_ => console.log(`Obtido id=${id}`)),
        catchError(this.handleError('obter'))
      );
  }
  obterTodos():Observable {
    return this.http.get(this.tarefasUrl).pipe(
        tap(_ => console.log('Produtos recuperados')),
        catchError(this.handleError('obterTodos', []))
      );
  }
}
```

Arquivo app-routing.module.ts.

```typescript
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { TarefaTableComponent }
         from './tarefa-table/tarefa-table.component';
import { TarefaFormComponent }
         from './tarefa-form/tarefa-form.component';

const routes: Routes = [
  { path: 'listagem', component: TarefaTableComponent },
  { path: 'dados', component: TarefaFormComponent },
  { path: '', redirectTo: '/listagem', pathMatch: 'full' },
 ];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Arquivo app.component.html.

```python


  Listagem
  Nova Tarefa

# {{title}}
```

Arquivo app.component.css.

```python
*{font-family: Arial, Helvetica, sans-serif;}
#menu {
    background-color: black; width:100%; height: 50px;
}
#menu a{
    color: white; text-decoration: none; width: 150px;
    margin-top: 15px; font-size: 20px;
    display: inline-block; text-align: center;
}
#menu a:hover{ text-decoration: underline; }
```

Arquivo app.module.ts.

```typescript
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { TarefaFormComponent }
         from './tarefa-form/tarefa-form.component';
import { TarefaTableComponent }
         from './tarefa-table/tarefa-table.component';
import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';
import { HttpClientInMemoryWebApiModule }
         from 'angular-in-memory-web-api';
import { TarefaServidorService } from './tarefa-servidor.service';
import { TarefaService } from './tarefa.service';

@NgModule({
  declarations: [
    AppComponent, TarefaFormComponent, TarefaTableComponent
  ],
  imports: [
    BrowserModule, AppRoutingModule, FormsModule,
    HttpClientModule,
    HttpClientInMemoryWebApiModule.forRoot(
      TarefaServidorService,{ dataEncapsulation: false })
  ],
  providers: [TarefaService],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Arquivo tarefa-form.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';
import { Tarefa } from '../tarefa';
import { TarefaService } from '../tarefa.service';

@Component({
  selector: 'app-tarefa-form',
  templateUrl: './tarefa-form.component.html',
  styleUrls: ['./tarefa-form.component.css']
})
export class TarefaFormComponent implements OnInit {
  tarefa = new Tarefa();
  enviando: boolean = false;

  constructor(private gestor: TarefaService) { }

  ngOnInit(): void { }

  incluir = () => {
    this.enviando = true;
    this.gestor.adicionar(this.tarefa).subscribe(
      _ => {
        this.tarefa = new Tarefa();
        this.enviando = false;
      }
    );
  }
}
```

Arquivo tarefa-form.component.html.

```python

    Titulo:


    Descricao:


    Incluir
```

Arquivo tarefa-form.component.css.

```python
form{ width: auto; background: lightblue; padding: 10px; }
label, button{ margin:5px; width: 100px; display: inline-block; }
```

Arquivo tarefa-table.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';
import { Tarefa } from '../tarefa';
import { TarefaService } from '../tarefa.service';

@Component({
  selector: 'app-tarefa-table',
  templateUrl: './tarefa-table.component.html',
  styleUrls: ['./tarefa-table.component.css']
})
export class TarefaTableComponent implements OnInit {

  tarefas = new Array();

  constructor(private gestor: TarefaService) { }

  ngOnInit(): void {
    this.gestor.obterTodos().subscribe(
      (tarefas) => this.tarefas = tarefas
    );
  }

  alternar = (id: number) => {
    this.gestor.obter(id).subscribe(
      (tarefa) => {
        tarefa.completa = !tarefa.completa;
        this.gestor.alterar(tarefa).subscribe(
          _ => this.gestor.obterTodos().subscribe(
            (tarefas) => this.tarefas = tarefas
          )
        )
      }
    );
  }

  remover = (id: number) => {
    this.gestor.remover(id).subscribe(
      _ => this.gestor.obterTodos().subscribe(
        (tarefas) => this.tarefas = tarefas
      )
    )
  }
}
```

Arquivo tarefa-table.component.html.

```python
| Titulo | Descricao | Efetuada | Remover |
|---|---|---|---|
| {{t.titulo}} | {{t.descricao}} | | X |
```

Arquivo tarefa-table.component.css.

```python
table { width: 100%; }
td,th { border-left: 1px solid black; padding: 5px; }
.oper { text-align: center; }
th { font-weight: bold; }
tr:nth-child(even) { background: lightgray }
button { background: red; color: white; }
```

# Tecnologia de back-end

Para criar um sistema completo, precisamos de um repositório de dados apropriado, como o **MongoDB**, e de uma tecnologia para o tratamento de requisições HTTP relacionadas às ações de persistência, sendo comum o uso de Web Services RESTful, os quais podem ser criados com base no **Express**, sobre o NodeJS. Outro ponto relevante é a preocupação com a usabilidade e com a acessibilidade, o que pode ser facilitado com a adoção de componentes do **Material Design**.

Este vídeo traz uma apresentação da tecnologia de back-end, abordando também o banco de dados MongoDB e o servidor REST com Express. Confira!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Banco de dados MongoDB

O termo **NoSQL**, que significa **Not Only SQL**, caracteriza uma família de repositórios que não são baseados apenas na sintaxe SQL, adotando diferentes modelos de armazenamento, e levando à classificação em diferentes grupos. Entre as opções existentes, podemos destacar as estratégias chave-valor, colunar, baseada em grafo e documental, além de algumas arquiteturas híbridas, que adotam combinações das anteriores.

Atenção

Enquanto no modelo relacional temos esquemas fixos para armazenagem, as estruturas de dados podem ser alteradas dinamicamente em bases NoSQL.

Quanto à arquitetura **documental**, temos sempre uma chave associada a um documento, o qual pode ser uma notação de texto, como **XML**, **JSON** e **YAML**, ou algum formato binário, como **PDF**. Uma característica desses documentos é a de que utilizam estruturas de metadados associadas, motivo pelo qual os dados são classificados como **semiestruturados**.

Um dos melhores exemplos de banco de dados no modelo documental é o **MongoDB**, uma opção de código aberto, multiplataforma e escrita em linguagem C++. Os valores armazenados no MongoDB são documentos no formato **JSON**, e as chaves podem ser geradas de forma automática ou especificadas de acordo com padrões específicos.

## Servidor REST com Express

Como estamos lidando com um banco de dados, precisamos de uma tecnologia de **back-end** para intermediar a comunicação com o aplicativo Angular. Considerando que o **NodeJS** define a plataforma na qual o Angular é definido, nada mais natural que utilizar a mesma tecnologia para criar o back-end, e uma boa opção é definir um servidor minimalista com base em **Express**.

A biblioteca Express permite criar um **servidor embarcado**, que executa sem a necessidade de produtos externos, como Apache ou JBoss, viabilizando uma utilização simples, sem qualquer tipo de configuração mais

complexa.

Através do processo de comunicação HTTP, via **Express**, podemos definir serviços de acordo com o padrão REST (Representational State Transfer), onde utilizamos os métodos do protocolo para gerenciar as operações de consulta e manipulação de dados. Vamos entender melhor!

- **GET**: consultas, com o retorno de entidades e coleções no formato JSON.

- **POST**: inserção de entidade, a partir dos dados no formato JSON.

- **PUT**: edição da entidade, com base na chave e nos dados em formato JSON.

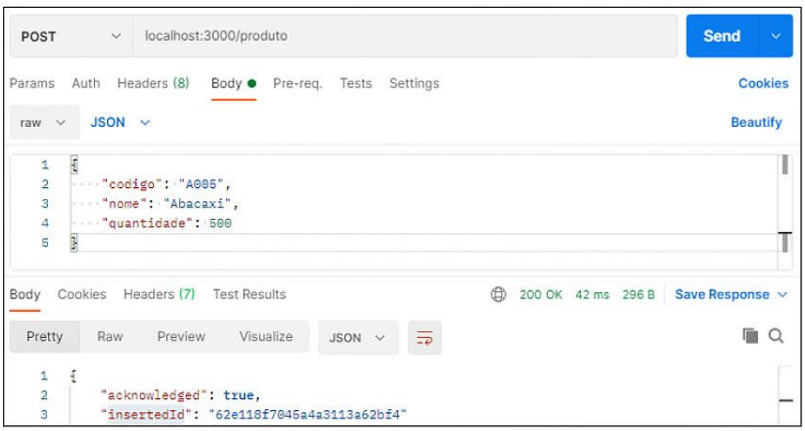
- **DELETE**: remoção da entidade, com base na chave primária.

De acordo com o REST, um endereço do tipo http://servidor/produto iria retornar todos os produtos da coleção, enquanto http://servidor/produto/62dfdc8ca0cc073daa10e815 retorna apenas o produto cujo atributo **_id** apresenta o valor numérico expresso ao final do endereço, em hexadecimal. Em ambos os casos, estamos nos referindo ao uso do método **GET**, por se tratar de **consulta**, e os dados referentes aos produtos são obtidos no formato **JSON**.

Para as chamadas nos modos **POST** e **DELETE**, será necessário utilizar algum aplicativo como o **Postman**, que permite definir as chamadas para todos os métodos do HTTP e a configuração dos parâmetros necessários, como visto a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Inclusão de produto através do Postman.

No exemplo apresentado, temos uma requisição para o endereço **localhost:3000/produto**, no modo **POST**, onde foram incluídos os dados do produto no corpo (**Body**), com as opções **raw** e **JSON**. Após configurar a chamada, podemos clicar no botão Send, obtendo a resposta no painel inferior, através da divisão **Body**.

Note que o retorno indica a efetivação da operação (**acknowledeged**) e o identificador gerado pelo MongoDB (**insertedId**). Essas informações serão úteis posteriormente, para o tratamento da resposta pela nossa

aplicação cliente, criada na plataforma Angular.

Devido ao uso de portas distintas, front-end e back-end estarão em domínios diferentes, e para que nosso servidor **Express** disponibilize o conteúdo para o cliente **Angular**, será necessária a configuração do **CORS** (Cross-Origin Resource Sharing), um mecanismo que permite oferecer os recursos restritos do servidor para endereços específicos. Esse é um ferramental de segurança que bloqueia acessos não previstos a qualquer rota definida em nosso sistema servidor.

Criptografia também é algo muito importante na implementação de um sistema, como para a guarda da senha, de forma que um acesso indevido ao banco de dados não exponha essa informação. Na comunicação, se quisermos proteger o canal existente entre cliente e servidor, devemos adicionar **TLS** (Transport Layer Security) ao protocolo HTTP, no ambiente de produção.

Para a criptografia de senhas, devemos utilizar o modelo **destrutivo**, em que não é possível efetuar a decriptografia, pois a senha não pode ser exposta em quaisquer circunstâncias, mesmo para o fornecedor do sistema. Como não é possível recuperar o valor original, precisamos comparar o valor armazenado na forma criptografada.

# Atividade 1

Questão 1

As tecnologias relacionais, que apresentam uma grande preocupação com a manutenção de relacionamentos entre as entidades, nem sempre oferecem resposta em tempo adequado para grandes massas de dados. Com o advento do big data, cuja a quantidade de dados é expressiva, opções ao armazenamento relacional passaram a ser utilizadas, popularizando os repositórios classificados como NoSQL, que por apresentarem grande heterogeneidade, são classificados em diferentes grupos, de acordo com a forma adotada para representar a informação. Uma opção de banco muito utilizada atualmente é o MongoDB, que é classificado como uma base do tipo:

A chave-valor.

B colunar.

C documental.

D baseada em grafos.

E relacional.

A alternativa C está correta.

A estrutura de armazenagem do MongoDB segue o modelo documental, onde os registros são documentos no formato JSON, com individualização a partir de um identificador padronizado, com o nome _id, que pode ser gerado automaticamente pelo banco. Quanto às demais opções, chave-valor define um modelo onde temos chaves associadas a valores simples, colunar é uma extensão do primeiro, com adição de

versionamento e agrupamento de campos, e os modelos baseados em grafos expressam ligações entre registros de forma gráfica. Já no modelo relacional, temos a contraposição aos modelos NoSQL, com relacionamentos sendo mantidos de forma consistente e robusta.

# Material design

As múltiplas plataformas da atualidade, incluindo sistemas móveis, web e desktop, levaram à necessidade de uma padronização maior das interfaces gráficas, além da inclusão de elementos relacionados à responsividade. Através de uma iniciativa do Google, vimos o surgimento do Material Design, disponível no endereço www.material.io, inicialmente focado no sistema Android, mas que hoje em dia é adotado como referencial para construção de interfaces em outras plataformas, incluindo Web, Flutter e iOS.

Neste vídeo, serão apresentadas as características gerais da tecnologia Material Design e a aplicação de seus componentes.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Características gerais

A utilização de uma abordagem minimalista é um ponto crucial para a garantia da responsividade, já que sistemas de projeção mais complexos não seriam representados adequadamente em dispositivos com menor poder computacional.

São incluídos elementos relacionados à **tipografia**, com a especificação de fontes apropriadas para os mais diversos ambientes, padronização de paletas de **cores**, fornecendo ferramentas que analisam o contraste, e definição de **dimensões** para garantia da **usabilidade** nas diversas plataformas.

Até mesmo os tipos de **animação** são formalizados pelo Material Design, pois devem oferecer significado para cada ação do usuário no contexto de utilização do aplicativo.

## Aplicando componentes

Os componentes abordados pelo Material Design apresentam diferentes objetivos, definindo os seguintes grupos. Confira!

### Grupo: Exibição

Objetivo: organização do conteúdo.

Componentes: Card, Lista, Sheet.

### Grupo: Navegação

Objetivo: alternância entre rotas ou telas.

Componentes: Tabs, NavigationDrawer.

Grupo: Ações
Objetivo: iniciar alguma operação.

Componentes: Button, FloatingActionButton.

Grupo: Entrada
Objetivo: entrada de dados e seleção de opções.

Componentes: TextField, Chips, Switch, Slider.

Grupo: Comunicação
Objetivo: emissão de mensagens.

Componentes: SnackBar, Banner, Dialog.

A seguir observamos o resultado da inclusão de **Cards** e **Tabs** em um **modelo** do Angular. Para disponibilizar os seletores, os módulos **MatCardModule** e **MatTabsModule**, da biblioteca **Angular Material**, devem ser incluídos nas importações do módulo principal.

Página de exemplo com Cards e Tabs.

Entre os componentes comuns do material design, o **Card** representa um painel com elevação, sendo definido pelo seletor **mat-card**, com divisão interna organizada por **mat-card-title-group** e **mat-card-content**, como na listagem a seguir.

python

Shiba Inu

Exemplo

Texto de exemplo apresentado por um componente Card

No cabeçalho, temos um seletor **mat-card-title**, com o título do cartão, **mat-card-subtitle**, com o subtítulo e uma imagem. Após a área de cabeçalho (título) do cartão, temos uma área de conteúdo, delimitada pelo seletor **mat-card-content**. A estrutura definida em um cartão é um padrão limpo e claro para a de informações, além de permitir a inclusão de botões de ação em uma terceira área, delimitada pelo seletor **mat-card-actions**.

Os elementos de navegação incluem as abas (**Tabs**), como no código de exemplo apresentado a seguir. Na troca de conteúdo, temos uma animação padrão, da direita para a esquerda.

```
python



    Placa de Som - PCI-E - Creative Sound Blaster Z SE

    Caixa de Som - Logitech Z607 - 5.1 Surround




    Placa de Vídeo - Nvidia - GeForce RTX série 30




    Processador Intel Core I7
```

A utilização de abas é muito simples, delimitando o conjunto em um seletor **mat-tab-group**, e cada área de conteúdo é definida em um seletor **mat-tab**, onde o atributo **label** irá especificar o título a aba. Podemos carregar dinamicamente o conteúdo, via tecnologia AJAX, permitindo a utilização de conteúdos mais complexos nas abas, sem que ocorra degeneração da performance em nosso front-end.

Componentes para **entrada de dados** foram amplamente modificados pelo Material Design, e passaram a trazer elementos associados para melhoria da **usabilidade** e **responsividade**, como podemos observar no código da listagem seguinte.

```
python



    Nome




  Assinar Canal

  Espaço de Armazenagem




  Enviar
```

Como podemos observar, campos de entrada do tipo Input são marcados como **matInput**, sendo associados à descrição pelo seletor **mat-label**, e os dois elementos são organizados pelo seletor **mat-form-field**. A descrição é apresentada no conteúdo do campo vazio, e quando ocorre a digitação do valor, ela se move para o topo da área de edição, além de termos uma marcação, na parte inferior, ativada quando o campo tem o foco para digitação.

Elementos **mat-checkbox** são caixas de marcação comuns, apenas com uma formatação padrão, enquanto um seletor **mat-slider** define um controle deslizante para seleção de valor. Os botões são criados como botões comuns, mas acrescidos do atributo **mat-raised-button**.

Temos, a seguir, o formulário do exemplo. Para criá-lo serão necessários n**MatInputModule**, **MatCheckboxModule**, **MatSliderModule** e **MatButtonModule**.

Componentes de entrada modificados pelo Material Design.

Outro componente amplamente utilizado é o **Floating Action Button**, ou FAB, que apresenta um botão, normalmente circular, posicionado acima das demais camadas do aplicativo. Ele sempre deve estar relacionado à função principal da tela.

No grupo de comunicação, temos as caixas de diálogo e outros meios comuns para apresentar mensagens ao usuário. Elas são exibidas durante a execução de um aplicativo, como no exemplo seguinte, com uma função de ativação na classe do componente.

```typescript
import { Component } from '@angular/core';

import { MatSnackBar } from '@angular/material/snack-bar';

@Component({

  selector: 'app-root',

  templateUrl: './app.component.html',

  styleUrls: ['./app.component.css']

})

export class AppComponent {

  title = 'Rascunho';

  constructor(private _snackBar: MatSnackBar) {}

  openSnackBar(message: string, action: string) {

    this._snackBar.open(message, action);

  }

}
```

Elementos do tipo **MatSnackBar** devem ser adicionados via **injeção de dependência**, e abertos através da chamada para o método **open**. Será necessário invocar o método de exibição a partir do clique em um botão do modelo, como no código apresentado a seguir.

```python


Clique Aqui
```

Ao clicar no botão, será apresentado o **SnackBar**, com a exibição da mensagem ocorrendo na parte inferior da tela, confira!

Exemplo com exibição de SnackBar.

# Atividade 2

Questão 1

Com o surgimento do Material Design, foi possível determinar um padrão para a criação de interfaces, envolvendo diferentes plataformas, incluindo preocupações com responsividade, usabilidade e acessibilidade.

São especificados os melhores padrões de cores, fontes, dimensões e unidades de medida, além de termos o conceito visual de componentes padronizados, que são implementados nas diferentes plataformas, trazendo homogeneidade para os sistemas. Para o Angular, seu uso é baseado na importação da biblioteca adequada e aplicação de atributos nos componentes comuns do HTML, bem como através de um conjunto de seletores próprios. Por exemplo, um botão poderia ser configurado para exibição elevada, ao mesmo tempo em que representa a opção negativa para uma pergunta, utilizando a configuração.

A mat-raised-button color="warn".

B mat-fab color="accent".

C mat-button color="warn".

D mat-raised-button color="primary".

E mat-button color="accent".

A alternativa A está correta.

As opções mais comuns para a formatação do botão são mat-button, onde temos o formato comum, sem elevação, mat-raised-button, que inclui a elevação, e mat-fab, para uma apresentação como botão flutuante, normalmente circular, representando uma ação principal. Já com relação ao esquema de cores, primary é a opção padrão, accent seria para um destaque simples, e warn (atenção) para uma opção que represente perigo. Considerando as definições apresentadas, a configuração correta seria **mat-raised-button color="warn"**.

# Persistência de dados na prática

Nesta prática, veremos como instalar o MongoDB e criar um repositório de produtos.

Neste vídeo, através do desenvolvimento de um estudo de caso, você aprenderá a implementar a persistência de dados.

Conteúdo interativo

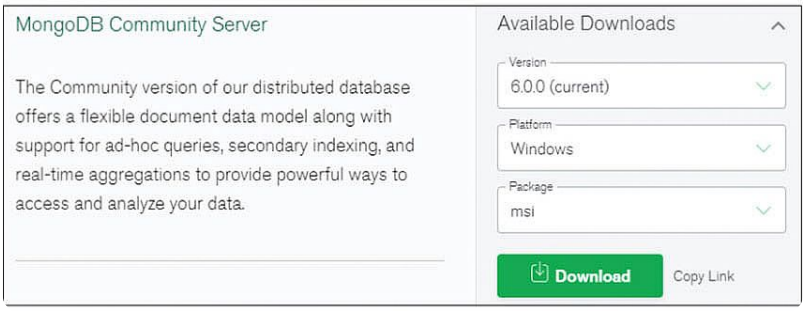
Acesse a versão digital para assistir ao vídeo.

## Roteiro de prática

1. Vamos baixar e instalar a **versão gratuita** no endereço https://www.mongodb.com/try/download/community. Escolha a versão adequada para seu sistema operacional e clique em download.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



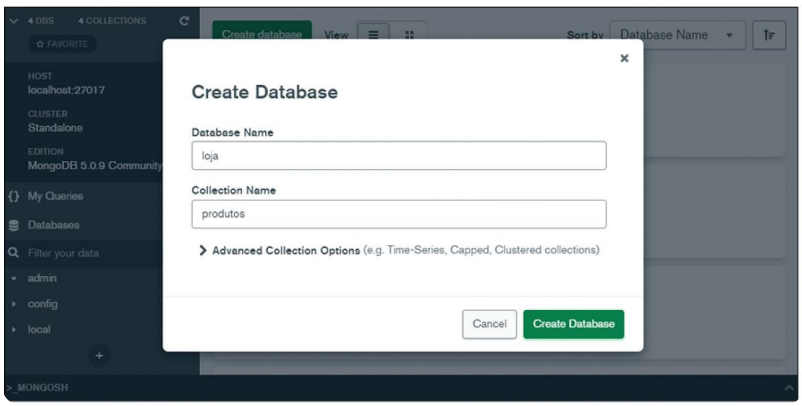

Download do MongoDB.

Ao final da instalação, teremos o banco de dados instalado como um **serviço**, além de um aplicativo para gerência com o nome **MongoDB Compass**.

2. Vamos abrir o gerenciador e clicar diretamente na opção **Connect**, abrindo uma conexão com o servidor de forma local e, efetuada a conexão, vamos criar um banco de dados com o nome **loja**, contendo uma coleção denominada **produtos**.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

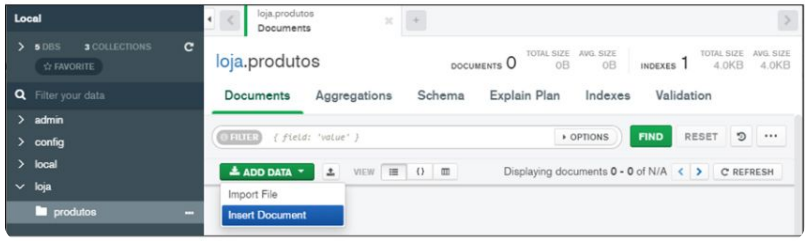



Criação da base no MongoDB através do gerenciador Compass.

3. Vamos inserir alguns documentos, no formato JSON, navegando até o banco criado, na lateral esquerda, e escolhendo a coleção **produtos**. Após selecionar a coleção, escolhemos a opção **Add Data**, seguida de **Insert Document**, abrindo a janela para digitação de documentos.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Inserção de registro através do Compass.

Utilizaremos o conjunto JSON, apresentado na listagem seguinte, para a definição de quatro produtos onde o campo _id não é citado, levando à geração automática pelo MongoDB.

```python
[
  {"codigo": "A001", "nome":"Banana", "quantidade": 1000},
  {"codigo": "A002", "nome":"Manga",  "quantidade": 500 },
  {"codigo": "B001", "nome":"Caneta", "quantidade": 100 },
  {"codigo": "B002", "nome":"Papel",  "quantidade": 1200}
]
```

4. Podemos verificar que os identificadores foram gerados automaticamente.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

Produtos criados com geração automática do identificador.

5. Na versão final de nosso aplicativo, será permitida a inclusão de usuários, para isso vamos criar a coleção **usuarios** no MongoDB. Devemos clicar no banco **loja**, a partir da árvore de navegação, na lateral esquerda, seguido do clique sobre o botão **Create Collection**, causando a abertura da janela para inserção do nome da coleção. Não vamos inserir dados na nova coleção.

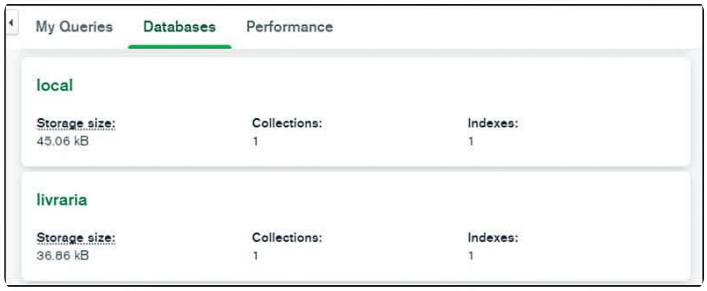
# Atividade 3

1. Defina uma nova coleção de documentos no MongoDB, com o nome tarefas, no banco de dados loja, criado na prática anterior.

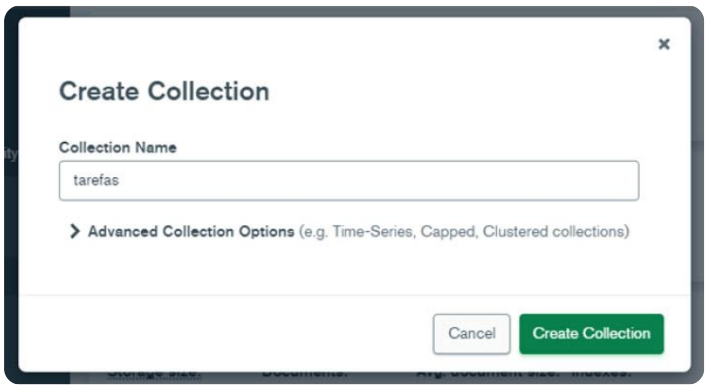
2. Insira três documentos na nova coleção, onde deverão ser fornecidos valores para os campos titulo, descricao e completa. O identificador, com nome _id, será gerado pelo MongoDB.

## Chave de resposta

Após conectar no MongoDB, através do MongoDB Compass, selecione o banco de dados loja, na aba Databases.

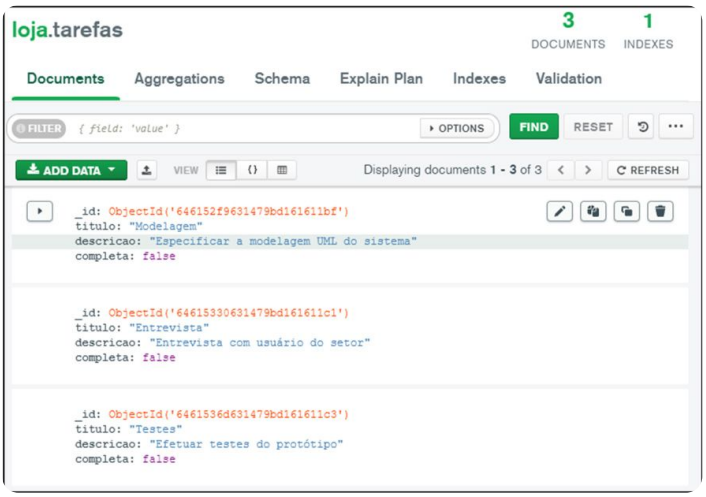



Com o banco **loja** selecionado, escolha a opção **Create Collection** e defina a coleção **tarefas**.




Finalmente, clique em **tarefas** e insira os documentos no formato **JSON**, um de cada vez.

```javascript
{
  "titulo": "Modelagem",
  "descricao": "Especificar a modelagem UML do sistema",
  "completa": false
}
{
  "titulo": "Entrevista",
  "descricao": "Entrevista com usuário do setor",
  "completa": false
}
{
  "titulo": "Testes",
  "descricao": "Efetuar testes do protótipo",
  "completa": false
}
```




## Tecnologia de back-end na prática

Nesta prática, veremos como criar uma **API** no estilo **REST**, baseada na tecnologia **Express**, com acesso à base de dados criada anteriormente no **MongoDB**.

Neste vídeo, através do desenvolvimento de um estudo de caso, você aprenderá a implementar a tecnologia de back-end.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Roteiro de Prática

1. Vamos instalar a biblioteca no ambiente de desenvolvimento.

```
typescript

npm install -g express-generator
```

2. Agora podemos criar nosso servidor, executando os comandos da listagem seguinte. Note que, após criar o projeto, entramos no diretório e instalamos a dependência **mongodb**, referente aos componentes de acesso ao MongoDB pelo NodeJS.

```
typescript

express -e ServidorLoja

cd ServidorLoja

npm install mongodb
```

3. Após abrir o diretório no Visual Studio Code, vamos iniciar a programação de nosso servidor com a **persistência dos dados**, criando o arquivo **GestorDB**.js, no diretório **dados**.

```typescript
const mongoClient = require("mongodb").MongoClient;

const ObjectId = require('mongodb').ObjectId;

mongoClient.connect("mongodb://127.0.0.1",

                    {useUnifiedTopology: true})

           .then(conn => global.conn = conn.db("loja"))

           .catch(err => console.log(err))

function findAll() {

  return global.conn.collection("produtos").find().toArray();

}
function findOne(chave) {

  return global.conn.collection("produtos")

               .findOne({_id: new ObjectId(chave)});

}

function insert(produto) {

  return global.conn.collection("produtos")

               .insertOne(produto);

}

function deleteOne(chave) {

  return global.conn.collection("produtos")

               .deleteOne({_id: new ObjectId(chave)});

}

module.exports = { findAll, insert, deleteOne, findOne }
```

4. Nosso gestor de persistência deve ficar disponível para todo o sistema, o que é feito com o acréscimo de uma linha no arquivo **www**, do tipo Java Script, dentro do diretório **bin**, com a definição da referência **global.db**. Todo o restante do código original do arquivo deverá ser mantido inalterado.

```typescript
#!/usr/bin/env node

/**
 * Module dependencies.
 */

global.db = require('../dados/GestorDB');
```

5. Vamos iniciar o processo de comunicação HTTP, via **Express**, de acordo com o padrão **REST**. Para tal, vamos alterar o arquivo **index.js**, dentro do diretório **routes**, substituindo todo o código de original pela listagem apresentada a seguir.

```typescript
var express = require('express');

var router = express.Router();

router.get('/produto', async (req, res) => {

  try {

    const docs = await global.db.findAll();

    res.send(docs);

  } catch (err) {

    res.send({resultado:"Erro ao Listar", mensagem: err});

  }

})

router.get('/produto/:id', async (req, res) => {

  try {

    const doc = await global.db.findOne(req.params.id);

    res.send(doc);

  } catch (err) {

    res.send({resultado:"Erro ao Consultar", mensagem: err});

  }

})

router.post('/produto', async (req, res) => {

  const codigo = req.body.codigo;

  const nome = req.body.nome;

  const quantidade = parseInt(req.body.quantidade);

  try {

    const doc = await global.db.insert({codigo, nome, quantidade});

    res.send(doc);

  } catch (err) {

    res.send({resultado:"Erro ao Inserir", mensagem: err});

  }

})

router.delete('/produto/:id', async (req, res) => {

  try {

    await global.db.deleteOne(req.params.id);

    res.send({resultado:"Removido"});
```

É importante saber que no corpo de uma função assíncrona, a instrução **await** permite aguardar a resposta da chamada para outra função com elementos assíncronos.

6. Vamos executar nosso servidor, utilizando o comando seguinte no diretório do projeto, ou através do Visual Studio Code, na opção **Run and Debug**, com escolha do tipo **NodeJS**.

```
typescript

npm start
```

7. Vamos realizar a chamada para **http://localhost:3000/produto**, pois a porta padronizada do Express tem valor 3000, a fim de termos o retorno da coleção completa de produtos no formato JSON.

```
python

[{"_id":"62dfdc8ca0cc073daa10e815","codigo":"A001",

"nome":"Banana","quantidade":1000},

{"_id":"62dfdc8ca0cc073daa10e816","codigo":"A002",

"nome":"Manga","quantidade":500},

{"_id":"62e20e481cde1213669d6635","codigo":"B001",

"nome":"Caneta","quantidade":100},
```

8. Para utilizar o recurso de CORS, precisamos instalar a biblioteca correta, executando o comando a seguir diretório do projeto.

```
typescript

npm install cors
```

9. Após instalar a biblioteca, precisamos alterar o arquivo **app.js**, na raiz do projeto, **adicionando** a configuração do CORS logo abaixo da definição da variável app, como no fragmento de código apresentado na listagem seguinte.

```
typescript

var app = express();

const cors = require('cors');

app.use(cors({

    origin: 'http://localhost:4200'

}));
```

10. Vamos adicionar as funções necessárias para gestão dos usuários em **GestorDB.js**, no diretório **dados**, de acordo com o fragmento de código da listagem seguinte, sem modificar o restante do arquivo. Note que a cláusula de **exportação** precisou ser alterada, para incluir as novas funções, relacionadas ao gerenciamento de usuários.

```typescript
function createUser(usuario){

  return global.conn.collection("usuarios")

               .insertOne(usuario);

}

function findUser(login,senha){

  return global.conn.collection("usuarios")

               .findOne({login, senha});

}

module.exports = { findAll, insert, deleteOne, findOne,

                   createUser, findUser }
```

11. Vamos modificar o arquivo users.js, no diretório routes, de acordo com a listagem apresentada a seguir (em um projeto Express, a rota users é criada automaticamente).

```typescript
var express = require('express');

var router = express.Router();

var crypto = require('crypto');

function getSenhaHex(senha) {

  const secret = "c4l4ng0p4ssw0rd";

  const hash = crypto.createHmac('sha256',secret)

                     .update(senha).digest();

  return hash;

}
router.post('/', async(req, res) => {

  const login = req.body.login;

  const senha = getSenhaHex(req.body.senha);

  const nome  = req.body.nome;

  const email = req.body.email;

  try {

    const doc = await global.db.createUser({login,senha,nome,email});

    res.send(doc);

  } catch (err) {

    res.send({resultado:"Erro ao Criar Usuario", mensagem: err});

  }

});

router.post('/login', async(req, res) => {

  const login = req.body.login;

  const senha = getSenhaHex(req.body.senha);

  try {

    const doc = await global.db.findUser(login,senha);

    res.send({login: doc.login, senha: "",

              nome: doc.nome, email: doc.email});

  } catch (err) {

    res.send({resultado:"Erro no Login", mensagem: err});

  }

});

module.exports = router;
```

# Atividade 4

Crie um servidor, baseado na tecnologia **Express** e acesso ao **MongoDB**, na forma de uma API REST, contemplando as operações sobre a coleção de **tarefas**, definida anteriormente.

O servidor deverá acrescentar suporte a CORS, com acesso permitido a partir da porta 4200.

| URL | Modo | Operação |
|---|---|---|
| localhost:3000/tarefas | GET | Obtenção de todas as tarefas |
| localhost:3000/tarefas/:id | GET | Obtenção da tarefa identificada pelo id |
| localhost:3000/tarefas | POST | Inclusão de tarefa, com dados fornecidos no corpo da requisição |
| localhost:3000/tarefas | PUT | Alteração da tarefa, com dados fornecidos no corpo da requisição |
| localhost:3000/tarefas/:id | DELETE | Remoção da tarefa identificada pelo id |

## Chave de resposta

Comandos de linha para criação e configuração do projeto.

```python
npm install -g express-generator
express -e AP4B
cd AP4B
npm install
npm install mongodb
npm install cors
```

Arquivo dados/GestorDB.js.

```javascript
const mongoClient = require("mongodb").MongoClient;
const ObjectId = require('mongodb').ObjectId;

mongoClient.connect("mongodb://127.0.0.1",
                    {useUnifiedTopology: true})
            .then(conn => global.conn = conn.db("loja"))
            .catch(err => console.log(err))

function findAll() {
  return global.conn.collection("tarefas").find().toArray();
}

function findOne(chave) {
  return global.conn.collection("tarefas")
               .findOne({_id: new ObjectId(chave)});
}

function insert(tarefa) {
  return global.conn.collection("tarefas")
               .insertOne(tarefa);
}

function updateOne(chave, tarefa) {
  return global.conn.collection("tarefas")
               .updateOne({_id: new ObjectId(chave)},
                          {$set: tarefa});
}

function deleteOne(chave) {
  return global.conn.collection("tarefas")
               .deleteOne({_id: new ObjectId(chave)});
}

module.exports = { findAll, findOne, insert, updateOne, deleteOne }
```

Disponibilize o gestor de forma global, importando na parte inicial do arquivo bin/www.

```
#!/usr/bin/env node

/**
 * Module dependencies.
 */

global.db = require('../dados/GestorDB');

var app = require('../app');
```

Arquivo **routes/index.js**.

```javascript
var express = require('express');
var router = express.Router();

router.get('/tarefa', async (req, res) => {
  try {
    const docs = await global.db.findAll();
    res.send(docs);
  } catch (err) {
    res.send({resultado:"Erro ao Listar", mensagem: err});
  }
})
router.get('/tarefa/:id', async (req, res) => {
  try {
    const doc = await global.db.findOne(req.params.id);
    res.send(doc);
  } catch (err) {
    res.send({resultado:"Erro ao Consultar", mensagem: err});
  }
})
router.post('/tarefa', async (req, res) => {
  const titulo = req.body.titulo;
  const descricao = req.body.descricao;
  const completa = Boolean(req.body.completa);
  try {
    const doc = await global.db.insert({titulo, descricao, completa});
    res.send(doc);
  } catch (err) {
    res.send({resultado:"Erro ao Inserir", mensagem: err});
  }
})
router.put('/tarefa', async (req, res) => {
  const chave = req.body._id;
  const titulo = req.body.titulo;
  const descricao = req.body.descricao;
  const completa = Boolean(req.body.completa);
  try {
    const doc = await global.db.updateOne(chave,
                              {titulo, descricao, completa});
    res.send(doc);
  } catch (err) {
    res.send({resultado:"Erro ao Alterar", mensagem: err});
  }
})
router.delete('/tarefa/:id', async (req, res) => {
  try {
    await global.db.deleteOne(req.params.id);
    res.send({resultado:"Removido"});
  } catch (err) {
    res.send({resultado:"Erro ao Remover", mensagem: err});
  }
})
module.exports = router;
```

Adicione o suporte a **CORS** no arquivo app.js, logo após a criação da instância de **express**.

```
10  var app = express();
11
12  const cors = require('cors');
13  app.use(cors({
14      origin: 'http://localhost:4200'
15  }));
16
17  // view engine setup
```

Comando de linha para executar o servidor.

```
javascript

npm start
```

O teste pode ser feito com acesso ao endereço http://localhost:3000/tarefa.

# Versão final do aplicativo

Nesta prática, iremos finalizar o nosso aplicativo, efetuando as modificações necessárias para acesso ao back-end definido com tecnologias Express e MongoDB, além de adicionar alguns dos componentes do Material Design, deixando a interface mais rica.

Neste vídeo, através do desenvolvimento de um estudo de caso, você aprenderá a implementar a tecnologia de back-end.

### Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Roteiro de Prática

1. Vamos começar duplicando o diretório do projeto e renomeando a cópia para **TerceiroAppFinal**. Como não iremos utilizar o servidor em **memória**, adotado anteriormente, vamos **remover** a biblioteca do projeto, executando o comando a seguir, no diretório **TerceiroAppFinal**.

```
typescript

npm uninstall angular-in-memory-web-api
```

2. Vamos **remover** o arquivo **produto-servidor.service.ts** clicando sobre o arquivo, no **Visual Studio Code**, e apertando a tecla **Delete**, ou utilizando o botão direito e escolhendo a opção de menu **Delete**. Feito isso, as referências devem ser removidas do módulo, no arquivo **app.module.ts**, eliminando as linhas de importação para **HttpClientInMemoryWebApiModule** e **ProdutoServidorService**, além de modificar a divisão imports para o conteúdo apresentado a seguir, com a exclusão da referência ao servidor em memória e serviço associado.

```typescript
imports: [
    BrowserModule,
    FormsModule,
    AppRoutingModule,
    HttpClientModule,
  ]
```

3. Devido ao identificador do MongoDB ser um número muito grande, que é tratado como texto no envio, iremos modificar a classe **Produto**, trocando o atributo **id** para **_id**, do tipo **string**, de acordo com a listagem apresentada a seguir, referente ao arquivo **produto.ts**.

```typescript
export class Produto {
  _id: string;
  codigo: string;
  nome: string;
  quantidade: number;
  constructor(_id: string="", codigo: string="",
              nome: string="", quantidade: number=0) {
    this._id = _id;
    this.codigo = codigo;
    this.nome = nome;
    this.quantidade = quantidade;
  }
}
```

4. Precisamos mudar o **clique** do botão no painel de produto, na tela de listagem de produtos, já que o **nome** do atributo foi modificado. A alteração é pontual, de acordo com o fragmento de código da listagem seguinte, e deve ser efetuada no arquivo **produto.component.html**.

```
python


  {{produto.codigo}}


  {{produto.nome}}


  Quantidade: {{produto.quantidade}}
---


Excluir
```

5. Note que acusará erro de tipo ao efetuar a mudança, e isso se deve ao fato de que ainda não alteramos a classe para o novo **tipo** do atributo, portanto vamos modificar o método **excluir**, de acordo com o fragmento de código seguinte, no arquivo **produto.component.ts**.

```
typescript
excluir(_id: string):void{
    this.servProd.remover(_id)
        .subscribe(_=>this.obterTodos());
  }
```

6. Precisamos alterar **ProdutoService**, no arquivo **produto.service.ts**, adequando-o ao novo tipo do atributo chave e modificando o endereço do servidor.

```typescript
export class ProdutoService {

  private produtosUrl = "http://localhost:3000/produto";

  // Importante! O restante do código não é alterado.

  adicionar(produto: Produto): Observable{

    return this.http.post(this.produtosUrl, produto,

                                  this.httpOptions).pipe(

      tap((novo: any) =>

           console.log(`Adicionado id=${novo.insertedId}`)),

      catchError(this.handleError('adicionar'))

    );

  }

  remover(_id: string): Observable{

    const url = `${this.produtosUrl}/${_id}`;

    return this.http.delete(url, this.httpOptions).pipe(

      tap(_ => console.log(`Removido id=${_id}`)),

      catchError(this.handleError('remover'))

    );

  }
```

7. Vamos executar nosso aplicativo, que agora busca os **produtos** a partir do **MongoDB**, por intermédio de **ServidorLoja**. Como o login ainda não foi alterado, ocorrendo a validação apenas em **memória**, devemos ter as mesmas funcionalidades.

8. Vamos criar um formulário para cadastro de usuários, exibido na forma de diálogo, com a ajuda dos componentes do **Material Design**, os quais serão adicionados com a execução do comando seguinte, no diretório do projeto. Durante a instalação, são feitas algumas perguntas, mas basta pressionar **Enter** para aceitar o valor padrão.

```typescript
ng add @angular/material
```

9. Diversas alterações são feitas de forma automática nos arquivos principais, como index.html, mas algumas configurações irão exigir nossa interferência, portanto vamos começar alterando a folha de estilos global, no arquivo styles.css, diretório src.

```
typescript

@import

"../node_modules/@angular/material/prebuilt-themes/indigo-pink.css";

html, body { height: 100%; }

body {margin: 0; font-family: Roboto, "Helvetica Neue", sans-serif;}
```

10. Alguns módulos também precisam de importação manual, no arquivo **app.module.ts**, mas como vamos precisar de outros módulos, pertencentes ao núcleo do Angular, o conteúdo completo do arquivo é listado a seguir.

```typescript
import { NgModule } from '@angular/core';

import { BrowserModule } from '@angular/platform-browser';

import { FormsModule, ReactiveFormsModule } from '@angular/forms';

import { HttpClientModule } from '@angular/common/http';

import { BrowserAnimationsModule }

       from '@angular/platform-browser/animations';

import { AppRoutingModule } from './app-routing.module';

import { AppComponent } from './app.component';

import { RodapeComponent } from './rodape/rodape.component';

import { LateralComponent } from './lateral/lateral.component';

import { HomeComponent } from './home/home.component';

import { ProdutoComponent } from './produto/produto.component';

import { NovProdComponent } from './nov-prod/nov-prod.component';

import { ProdutoService } from './produto.service';

import { LoginService } from './login.service';

import { LoginComponent } from './login/login.component';

import { MatFormFieldModule } from '@angular/material/form-field';

import { MatInputModule } from '@angular/material/input';

import { MatButtonModule } from '@angular/material/button';

import { MatDialogModule } from '@angular/material/dialog';

@NgModule({

  declarations: [

    AppComponent,      RodapeComponent,   LateralComponent,

    HomeComponent,     ProdutoComponent,  NovProdComponent,

    LoginComponent

  ],

  imports: [

    BrowserModule,     FormsModule,       ReactiveFormsModule,

    AppRoutingModule,  HttpClientModule,  BrowserAnimationsModule,

    MatInputModule,    MatButtonModule,   MatFormFieldModule,

    MatDialogModule

  ],
  providers: [ProdutoService, LoginService],
```

11. Agora que as configurações estão completas, vamos adicionar o componente para cadastro de usuário, com o nome **CadastroUsuComponent**, através do comando a seguir.

```
typescript

ng g c CadastroUsu
```

12. Antes de começar a programação do novo componente, precisamos modificar n**LoginService**, no arquivo **login.service.ts**.

```typescript
import { HttpClient, HttpHeaders } from '@angular/common/http';

import { Injectable } from '@angular/core';

import { ActivatedRouteSnapshot, CanActivate, Router,

         RouterStateSnapshot, UrlTree } from '@angular/router';

import { catchError, concat, from, map, Observable, of, tap }

         from 'rxjs';

import { Usuario } from './usuario';


@Injectable({

  providedIn: 'root'

})

export class LoginService implements CanActivate{

  private usersUrl = "http://localhost:3000/users";

  private http: HttpClient;

  httpOptions = {

    headers: new HttpHeaders({ 'Content-Type': 'application/json' })

  };

  router: Router;

  isAuthenticated: boolean = false;

  usuarioLogado: Usuario = new Usuario();


  constructor(router: Router, http: HttpClient) {

     this.router = router;

     this.http = http;

  }

  private handleError(operation:string = 'operation', result?: T) {

    return (error: any): Observable => {

      console.error(operation+" :: "+error);

      return of(result as T);


    };

  }

  canActivate(route: ActivatedRouteSnapshot,

              state: RouterStateSnapshot):

        boolean | UrlTree | Observable |
```

13. Após a alteração para o padrão Observer, com base em RxJS, precisamos alterar o componente de login, mas antes vamos codificar a classe **CadastroUsuComponent**, de acordo com a listagem seguinte, no arquivo **cadastro-usu.component.ts**. Faremos dessa forma, pois teremos na tela de login a opção de cadastro de usuário.

```typescript
import { Component, OnInit } from '@angular/core';

import { Validators, FormBuilder, FormGroup } from '@angular/forms';

import { LoginService } from '../login.service';

import { Usuario } from '../usuario';

@Component({

  selector: 'app-cadastro-usu',

  templateUrl: './cadastro-usu.component.html',

  styleUrls: ['./cadastro-usu.component.css']

})

export class CadastroUsuComponent implements OnInit {

  formUsu: FormGroup;

  servLogin: LoginService;

  constructor(private formBuilder: FormBuilder,

              servLogin: LoginService) {

    this.servLogin = servLogin;

    this.formUsu = this.formBuilder.group({

      nome:  ['', [Validators.required]],

      email: ['', [Validators.required, Validators.email]],

      login: ['', [Validators.required]],

      senha: ['', [Validators.required, Validators.minLength(6)]]

    });

  }

  getError(campo: string): string {

    if(this.formUsu.get(campo)?.hasError('required'))

       return "Campo Obrigatório";

    if(this.formUsu.get(campo)?.hasError('email'))

       return "E-mail inválido";

    if(this.formUsu.get(campo)?.hasError('minlength')){

       const erro = this.formUsu.get(campo)?.errors?.['minlength'];

       return `Mínimo de ${erro.requiredLength} caracteres`;

    }

    return "Erro no campo";

  }
```

Nosso novo formulário é do tipo **reativo**, que permite o tratamento de **erros** enquanto ocorre o **preenchimento** dos campos. Para o modelo adotado, é necessário um **FormGroup**, aqui com o nome **formUsu**, definindo um conjunto de elementos **FormControl**, onde cada um traz o valor inicial e um grupo de **validadores**. Podemos observar que temos a **injeção** de um **FormBuilder**, permitindo a criação de **formUsu**, através do método **group**, sendo incluídos os controles para cada um dos campos do **usuário**, à exceção do identificador gerado pelo MongoDB.

Validadores são utilizados para a validação do campo durante o preenchimento, veja os mais comuns!

Validators.required
Campos obrigatórios do formulário.

Validators.email
Verifica se o e-mail está no formato correto.

Validators.maxLength
Determina o tamanho máximo do texto.

Validators.minLength
Determina o tamanho mínimo do texto.

Validators.pattern
Utiliza uma expressão regular para validação.

Validators.max
Determina o valor máximo que pode ser utilizado.

Validators.min
Determina o valor mínimo que pode ser utilizado.

Validators.requiredTrue
Exige que a caixa de checagem seja marcada.

14. Vamos modificar o modelo do componente, no arquivo **cadastro-usu.component.html**.

python

# Cadastro de Usuário

Nome

{{getError("nome")}}

E-mail

{{getError("email")}}

Login

{{getError("login")}}

Senha

{{getError("senha")}}

Cancelar

15. Ainda teremos que efetuar diversas alterações em LoginComponent, que será responsável pela abertura do diálogo com o formulário de cadastro, mas podemos verificar o resultado apresentado a seguir.

Formulário de Cadastro de Usuário sendo apresentado.

16. Não é difícil perceber que precisamos de um ponto de ativação para o diálogo, o qual estará na tela de login, sendo necessário alterar **LoginComponent**, no arquivo **login.component.ts**, de acordo com a listagem seguinte, onde temos o conteúdo completo da classe.

```typescript
import { Component, OnInit } from '@angular/core';

import { MatDialog } from '@angular/material/dialog';

import { Router } from '@angular/router';

import { CadastroUsuComponent }

       from '../cadastro-usu/cadastro-usu.component';

import { LoginService } from '../login.service';

@Component({

  selector: 'app-login',

  templateUrl: './login.component.html',

  styleUrls: ['./login.component.css']

})

export class LoginComponent implements OnInit {

  router: Router;

  servLogin: LoginService;

  dialogo: MatDialog;

  login: string = "";

  senha: string = "";

  mensagem: string = "";

  enviando: boolean = false;

  constructor(router: Router, servLogin: LoginService,

              dialogo: MatDialog) {

    this.router = router;

    this.servLogin = servLogin;

    this.dialogo = dialogo;

  }

  signin(): void{

    this.enviando = true;

    this.servLogin.signin(this.login,this.senha)

        .subscribe(resultado=>{

          this.enviando = false;

          if(resultado){

            this.router.navigateByUrl("/produto");

          }
```

17. Vamos alterar o modelo, no arquivo **login.component.hmtl**, de acordo com a listagem seguinte, na qual temos a inclusão do botão para abertura do diálogo e modificação dos campos de entrada para o padrão do Material Design.

python

# Acesso Autenticado

{{mensagem}}

Login

Senha

Login

Não é cadastrado?

Clique aqui!

18. Vamos realizar um pequeno **acréscimo** na folha de estilos, no arquivo **login.component.css**, de acordo com o fragmento de código seguinte, **incluindo** a classe dos painéis ao final, sem modificar o restante do conteúdo original do arquivo.

```
python
.pnlInterno{width: 49%; display: flex; flex-wrap: wrap; float: left;}
```

19. Também iremos alterar o arquivo **lateral.component.css**, visando deixar a área de visualização do aplicativo dinâmica, modificando o estilo para o identificador lateral.

```
python
#lateral{
  display: flex; flex-wrap: wrap; width: 20%; min-height: 300px;
  height: calc(100% - 165px);
  justify-content: center; background-color: lightblue;
}
```

20. O nosso rodapé terá a altura de 15 pixels, fixada com a modificação do identificador, no arquivo **rodape.component.css**.

```
python
#rodape{
  background-color: red; color: white; font-weight: bold;
  text-align: center; padding: 5px; border: black dashed;
  height: 15px;
}
```

21. Com relação ao cabeçalho, vamos fixar a altura em 130 pixels, acrescentando seu identificador, ao final do arquivo n**app.component.css**, de acordo com o fragmento de código apresentado a seguir, sem modificar o restante do conteúdo.

```
python
#cabecalho{ height:130px; }
```

22. Agora nosso aplicativo está com suporte melhor ao redimensionamento da página, o que o torna mais responsivo, e já podemos testar a nova versão da tela de autenticação.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

Versão final da tela de autenticação.

Após tentar acessar a listagem de produtos, será apresentada a tela de autenticação, onde será necessário clicar no botão com o texto "Clique aqui!", abrindo o cadastro de usuários, e inserir ao menos um usuário no MongoDB, já que precisaremos dele para o acesso autenticado. Para nosso teste, vamos criar o usuário Administrador, com e-mail admin@calango.net, login como admin e senha 123456.

Podemos utilizar, em seguida, o novo usuário, obtendo a autenticação do sistema e acessando a tela de listagem de produtos.

23. Vamos verificar o novo usuário no **MongoDB**, por meio do gerenciador **Compass**, observe!

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

Consulta ao usuário do aplicativo através do MongoDB Compass.

24. Faremos uma modificação final na tela de listagem de produtos, para utilizar os componentes do Material Design, alterando o conteúdo do arquivo **produto.component.html**.

python

```
Listagem de Produtos

+

{{produto.codigo}}
{{produto.nome}}

Quantidade: {{produto.quantidade}}

Excluir
```

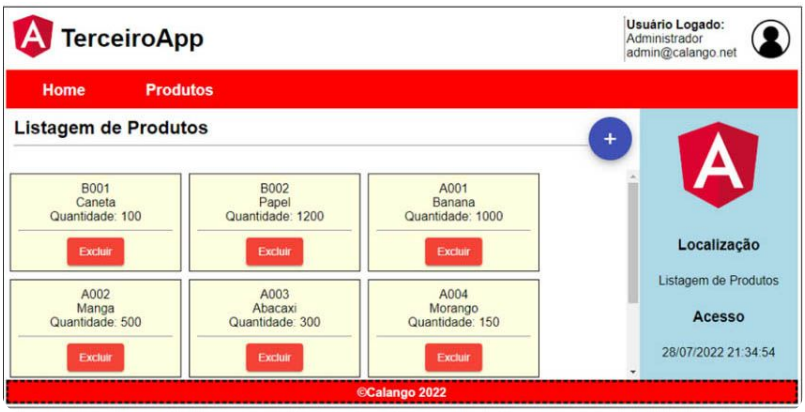
25. Com todas as modificações até aqui, a área de exibição dos produtos não acompanhará mais o espaço disponível, sendo necessária uma pequena alteração na folha de estilos. Devemos mudar o conteúdo de **produto.component.css**.

```
python
*{font-family: Arial, Helvetica, sans-serif;}

#conteudo{

  display: flex; flex-wrap: wrap; width: 80%;

  height:calc(100% - 165px);

  float: left; justify-content: center; align-items: center;

}

.container{

  display: flex; flex-direction: row; flex-wrap: wrap; width: 100%;

  overflow-y: auto; height: calc(100% - 80px);

}

.pnlProduto{

  width: 200px; height: 105px; background-color: lightyellow;

  border: black solid 1px; text-align: center;

  margin: 5px; padding: 10px;

}
```

26. Vamos executar a versão final da listagem de produtos, conforme apresentado a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.




Versão final da tela para listagem de produtos.

27. Aproveitando o fato de que estamos adotando o Material Design em todas as telas anteriores, vamos modificar o arquivo **nov-prod.component.html**.

```python
Novo Produto
---
Código
Nome
Quantidade
Incluir
```

28. As modificações efetuadas podem ser observadas a seguir.

Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

Versão final da tela para cadastro de produto.

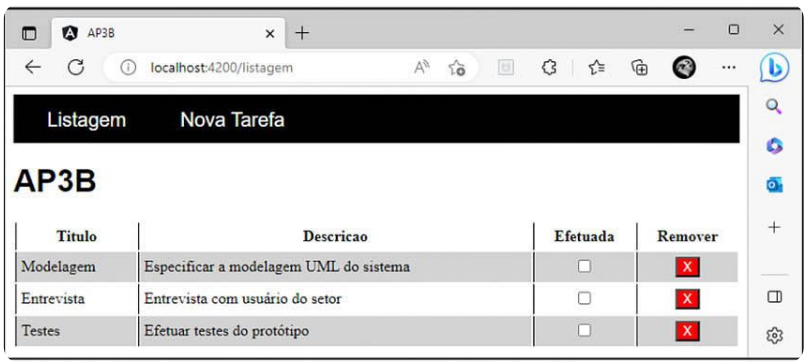
Chegamos ao final do nosso desenvolvimento, com um aplicativo totalmente funcional, baseado em Angular, Express e MongoDB, além de utilizar componentes do Material Design.

# Atividade 5

Copie a última versão do aplicativo de gerenciamento de tarefas para um novo diretório, renomeie e adapte para a chamada ao servidor criado na última atividade prática.

Lembre-se de remover o servidor simulado.

A saída da versão correta do aplicativo de gerenciamento de tarefas é replicada a seguir.




## Chave de resposta

Copie o diretório AP3B e renomeie para AP4C, através do gerenciador de arquivos, ou utilize os comandos de linha apresentados a seguir.

```
python


md AP4C
xcopy AP3B AP4C /s
```

Comandos de linha para configuração do novo projeto.

```python

cd AP4C
npm uninstall angular-in-memory-web-api@0.14.0
```

Exclua o arquivo **tarefa-servidor.service.ts** e modifique o atributo **title**, em **app.component.ts**, para o valor "**AP4C**". Os demais arquivos modificados são listados na sequência.

Arquivo app.module.ts.

```typescript
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { TarefaFormComponent }
         from './tarefa-form/tarefa-form.component';
import { TarefaTableComponent }
         from './tarefa-table/tarefa-table.component';

import { FormsModule } from '@angular/forms';
import { HttpClientModule } from '@angular/common/http';
import { TarefaService } from './tarefa.service';

@NgModule({
  declarations: [
    AppComponent,
    TarefaFormComponent,
    TarefaTableComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule,
    HttpClientModule,
  ],
  providers: [TarefaService],
  bootstrap: [AppComponent]
})

export class AppModule { }
```

Arquivo tarefa.ts.

```typescript
export class Tarefa {
   _id: string;
   titulo: string;
   descricao: string;
   completa: boolean;
   constructor(_id: string = "", titulo: string = "",
               descricao: string = "", completa: boolean = false) {
      this._id = _id;
      this.titulo = titulo;
      this.descricao = descricao;
      this.completa = completa;
   }
}
```

Arquivo tarefa.service.ts.

```typescript
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, Observable, of, tap } from 'rxjs';
import { Tarefa } from './tarefa';

@Injectable({
  providedIn: 'root'
})
export class TarefaService {
  private tarefasUrl = "http://localhost:3000/tarefa";
  private http: HttpClient;
  httpOptions = {
    headers: new HttpHeaders({'Content-Type': 'application/json'})
  };

  constructor(http: HttpClient) { this.http = http; }

  private handleError(operation:string = 'operation', result?: T){
    return (error: any): Observable => {
      console.error(operation+" :: "+error);
      return of(result as T);
    };
  }

  adicionar(tarefa: Tarefa): Observable{
    return this.http.post(this.tarefasUrl, tarefa,
                                  this.httpOptions).pipe(
        tap((novo: any) =>
             console.log(`Adicionado id=${novo.insertedId}`)),
        catchError(this.handleError('adicionar'))
      );
  }

  remover(_id: string): Observable{
    const url = `${this.tarefasUrl}/${_id}`;
    return this.http.delete(url, this.httpOptions).pipe(
        tap(_ => console.log(`Removido id=${_id}`)),
        catchError(this.handleError('remover'))
      );
  }

  alterar(tarefa: Tarefa): Observable{
    return this.http.put(this.tarefasUrl, tarefa,
                                 this.httpOptions).pipe(
        tap(_ => console.log(`Alterado id=${tarefa._id}`)),
        catchError(this.handleError('alterar'))
    );
  }

  obter(_id: string): Observable{
    const url = `${this.tarefasUrl}/${_id}`;
    return this.http.get(url).pipe(
        tap(_ => console.log(`Obtido id=${_id}`)),
        catchError(this.handleError('obter'))
      );
  }

  obterTodos():Observable {
    return this.http.get(this.tarefasUrl).pipe(
        tap(_ => console.log('Produtos recuperados')),
        catchError(this.handleError('obterTodos', []))
      );
  }
}
```

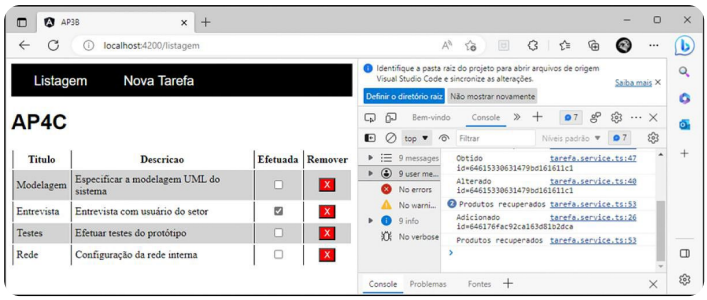
Arquivo tarefa-table.component.ts.

```typescript
import { Component, OnInit } from '@angular/core';

import { Tarefa } from '../tarefa';
import { TarefaService } from '../tarefa.service';

@Component({
  selector: 'app-tarefa-table',
  templateUrl: './tarefa-table.component.html',
  styleUrls: ['./tarefa-table.component.css']
})
export class TarefaTableComponent implements OnInit {

  tarefas = new Array();

  constructor(private gestor: TarefaService) { }

  ngOnInit(): void {
    this.gestor.obterTodos().subscribe(
                            (tarefas) => this.tarefas = tarefas);
  }
  alternar = (_id: string) => {
    this.gestor.obter(_id).subscribe(
      (tarefa) => {
        tarefa.completa = !tarefa.completa;
        this.gestor.alterar(tarefa).subscribe(
          _ => this.gestor.obterTodos().subscribe(
            (tarefas) => this.tarefas = tarefas
          )
        )
      }
    );
  }
  remover = (_id: string) => {
    this.gestor.remover(_id).subscribe(
      _ => this.gestor.obterTodos().subscribe(
        (tarefas) => this.tarefas = tarefas
      )
    )
  }
}
```

Arquivo tarefa-table.component.html.

```python
| Titulo | Descricao | Efetuada | Remover |
|---|---|---|---|
| {{t.titulo}} | {{t.descricao}} | | X |
```

Execute o servidor, com **npm start**, e o cliente, com ng serve --open, deixando-os em execução simultaneamente. A interface cliente deverá funcionar como antes.


AP3B
localhost:4200/listagem
Listagem
Nova Tarefa
AP4C
Titulo
Descricao
Efetuada
Remover
Modelagem
Especificar a modelagem UML do sistema
Entrevista
Entrevista com usuário do setor
Testes
Efetuar testes do protótipo
Rede
Configuração da rede interna
Identifique a pasta raiz do projeto para abrir arquivos de origem Visual Studio Code e sincronize as alterações.
Saiba mais
Definir o diretório raiz
Não mostrar novamente
Bem-vindo
Console
Filtrar
Níveis padrão
9 messages
9 user me...
No errors
No warni...
9 info
No verbose
Obtido
tarefa.service.ts:47
Alterado
tarefa.service.ts:40
Produtos recuperados
tarefa.service.ts:53
Adicionado
tarefa.service.ts:26
Produtos recuperados
tarefa.service.ts:53
Problemas
Fontes

# Considerações finais

- Arquitetura do framework Angular.
- Linguagem typescript.
- Roteamento no angular.
- Construção de componentes no Angular.
- Serviços no angular.
- Padrões de desenvolvimento de software.
- A Biblioteca RxJS.
- Acesso a recursos remotos.
- Tecnologia de back-end.
- Tecnologia Material Design.

## Explore +

- Acesse o artigo **Um overview sobre o framework Angular**, disponibilizado por Geek Hunter, com uma boa descrição geral da arquitetura do Angular.
- Leia o artigo **Persistência com Spring Data JPA**, disponibilizado no site DevMedia.
- Acesse o artigo **How to Use Reactive Forms in Angular**, disponibilizado por Digital Ocean, com um bom tutorial sobre criação de formulários reativos no Angular.

- Verifique a documentação oficial do **RxJS – Reactive Extensions Library for Java Script**, navegue pelos menus, e acesse vários exemplos de código com uso da biblioteca.

- Acesse o artigo **MongoDB - Guia completo para iniciantes**, disponibilizado por Tips Code, com um tutorial muito completo acerca do uso de MongoDB e Compass.

- Verifique a documentação oficial dos componentes do Material Angular, navegue pelos menus, e acesse diversos exemplos de utilização em projetos Angular.

## Referências


CHODOROW, K; BRADSHAW, S. **Mongodb**: the definitive guide. 3. ed. USA: O'Reilly, 2019.

KOTARU, V. **Angular for material design**. 1. ed. USA: Apress, 2019.

MEW, K. **Aprendendo Material Design**. 1. ed. São Paulo: Novatec, 2016.

MOHAMMED, Z. **Angular projects**. 1. ed. Reino Unido: Packt, 2019.

NORING, C. **Architecting angular applications**. 1. ed. Reino Unido: Packt, 2018.

O'HANLON, P. **Advanced typescript programming**. 1. ed. Reino Unido: Packt, 2019.

ROZENTALS, N. **Mastering typescript 3**. 3. ed. Reino Unido: Packt, 2019.

SESHADRI, S. **Angular**: up and running. 1. ed. USA: O'Reilly, 2018.

WILSON, M. **Construindo aplicações node**. 1. ed. São Paulo: Novatec, 2013.